System and Method for Enterprise Application Portability

Provisional Patent Application

This provisional application describes a system for:

- Extracting application genomes from enterprise systems and documents
- Representing applications as canonical, platform-neutral models
- Orchestrating transformation workflows using hybrid AI and deterministic pipelines
- Enabling incremental migration and cross-platform reconstruction
- Optimizing AI-driven transformation through context hydration and reusable transformation recipes

The invention establishes a new category of software:

Enterprise Application Portability.

Patent Technical Disclosure: Application Genome Extraction

Title

System and Method for Automated Application Genome Extraction and Structured Taxonomy-Based Repository Storage for Enterprise Application Portability

1. Field of Invention

This invention relates to enterprise software migration, application portability, and automated configuration analysis. Specifically, it describes a system and method for automatically decomposing enterprise applications into a canonical structured representation ("application genome") and storing the results in a version-controlled repository with a prescribed hierarchical taxonomy.

2. Background and Prior Art Gaps

Enterprise application migration between platforms (e.g., Platform A to Platform B) currently requires extensive manual effort. Existing approaches include:

- Manual mapping: Consultants manually document application objects, fields, workflows, and relationships -- a process that takes weeks to months per application.
- Platform-specific export tools: Vendor tools (e.g., platform configuration exports, vendor metadata APIs) export raw configurations but do not normalize them into a vendor-agnostic format usable for cross-platform migration.
- Code analysis tools: Static analysis tools examine source code but do not capture configuration-driven applications (e.g., platform catalog items configured via UI, not code).
- Documentation-based approaches: Migration teams rely on written documentation which is often incomplete, outdated, or inconsistent with the actual deployed application.

Gaps in prior art:

1. No existing system automatically decomposes enterprise applications into a canonical, vendor-agnostic genome comprising objects, fields, workflows, and relationships.
2. No existing system produces both a flat document representation (for backward compatibility and simple serialization) AND a structured graph representation (for advanced traversal and field-to-object binding) from a single extraction.
3. No existing system stores extracted genomes in a hierarchical taxonomy (tenant/vendor/product_area/module) within a version-controlled repository, enabling bulk operations, cross-tenant discovery, and auditability.
4. No existing system employs deterministic heuristic field-to-object binding (prefix matching) that is reproducible, LLM-free, and efficient.
5. No existing system implements a multi-pass pipeline (scan -> normalize -> build -> commit) with graceful degradation at each stage.

---

3. Summary of Invention

The present invention provides an automated pipeline that:

1. Connects to source enterprise platforms via configured integrations
2. Discovers available applications, catalogs, and modules through live API queries
3. Deploys a context-aware extractor at the source platform that extracts application objects at a negotiated scan depth
4. Normalizes the vendor-specific payload into a vendor-agnostic intermediate representation (tables, fields, workflows, relationships)
5. Builds a dual-schema genome:
   - GenomeDocument: Flat lists of object names, field names, workflow names, and relationship strings (backward compatible)
   - GenomeGraph: Structured graph with typed objects owning their fields, ID-based relationship references, and workflow-to-object bindings via deterministic heuristic matching
6. Commits all artifacts to a GitHub repository using a prescribed taxonomy: genomes/tenants/{tenant}/vendors/{vendor}/{product_area}/{module}/ with separate files for genome YAML, graph YAML, per-item structure YAML, configuration, and raw vendor payload

The system operates deterministically (no LLM required for core extraction), supports graceful degradation (failures in

intermediate steps do not halt the pipeline), and produces idempotent results via source signature hashing.

---

4. Detailed Description

4.1 System Architecture

The system comprises six cooperating services:

1. OYExtractorRegistryService -- Deploys and manages extractors at source platforms; performs context-aware extraction
2. Normalizer -- Converts vendor-specific payloads into a vendor-agnostic intermediate format
3. GenomeBuilder -- Applies vendor-specific parsers to produce a GenomeDocument (flat schema)
4. GenomeGraphBuilder -- Transforms GenomeDocument into GenomeGraph (structured schema) using deterministic heuristic field binding
5. OYGenomeBuilderService -- Produces a canonical genome representation with summary statistics
6. OYGenomeGitHubService -- Commits all artifacts to GitHub with taxonomized directory structure

4.2 Data Flow

```
Source Enterprise Platform
       v API calls
[OYExtractorRegistryService]
  - Deploy extractor with context (tenant, vendor, application, scan_depth)
  - Extract application objects
  - Flatten vendor-specific response wrappers
       v raw vendor payload
[Normalizer (_normalize_for_genome_builder)]
  - Infer tables from catalog items
  - Extract fields from item variables
  - Derive workflows from item workflow associations
  - Infer relationships from category hierarchies and variable containment
  - Deduplicate by name (set-based, order-preserving)
       v normalized payload {tables[], fields[], workflows[], relationships[]}
[GenomeBuilder]
  - Select vendor-specific parser from registry
  - Parse objects, workflows, fields, relationships
```

```
  - Output: GenomeDocument (flat string lists)
       v
[GenomeGraphBuilder]
  - Phase 1: Create GenomeObject for each object name
  - Phase 2: Bind fields to objects using longest-prefix-match heuristic
  - Phase 3: Parse relationship strings ("A -> B") into typed GenomeRelationship
  - Phase 4: Attach workflows to objects by name containment
  - Phase 5: Create synthetic "global" object for unmatched fields
  - Phase 6: Assemble GenomeGraph preserving discovery order
       v
[OYGenomeGitHubService]
  - Build file tree: genome.yaml, graph.yaml, structure/, config/, data/
  - Scrub secrets from all content
  - Ensure GitHub repo exists (create if needed)
  - Commit with structured message including tenant, vendor, application, depth
       v
GitHub Repository (versioned, searchable, auditable)
```

4.3 Dual-Schema Genome Representation

The system produces two complementary representations from a single extraction:

GenomeDocument (Flat Schema):

```
objects: ["request", "approval", "task", "asset"]
workflows: ["request submission", "manager approval", "procurement"]
fields: ["request_id", "requested_by", "item_type", "cost_center"]
relationships: ["request -> approval", "request -> task", "task -> asset"]
```

GenomeGraph (Structured Schema):

```
objects:
  - id: request
    name: request
    type: table
    fields:
      - id: request_id
        name: request_id
        object_name: request
        type: string
        required: true
      - id: requested_by
        name: requested_by
        object_name: request
        type: reference
        reference: user
    workflows: ["wf_request_submission"]
    relationships: ["rel_0", "rel_1"]
workflows:
  - id: wf_request_submission
    name: request submission
```

```
    trigger: ""
    steps: []
relationships:
  - id: rel_0
    source_object: request
    target_object: approval
    relationship_type: reference
    cardinality: "1:N"
```

4.4 Deterministic Field-to-Object Binding Algorithm

The GenomeGraphBuilder employs a heuristic prefix-matching algorithm to bind fields to objects without requiring LLM inference:

1. Construct a lookup dictionary mapping normalized object name fragments to object IDs
2. For each field name, attempt to match using progressively shorter prefixes:
   - Field "request_id" -> try "request" -> matches object "request" [x]
   - Field "assignedtouser" -> try "assignedtouser" -> "assigned_to" -> "assigned" -> no match -> falls to global
3. Fields that cannot be matched are assigned to a synthetic "global" virtual object
4. The algorithm is deterministic, reproducible, and requires no external API calls

4.5 Vendor-Specific Parser Registry

The GenomeBuilder maintains a pluggable parser registry supporting multiple vendors. The following illustrates the registry pattern with example vendor identifiers:

```
# Illustrative examples of supported platform parsers
_VENDOR_PARSERS = {
    "vendor_a": _parse_vendor_a,
    "vendor_b": _parse_vendor_b,
    "vendor_c": _parse_vendor_c,
    "vendor_d": _parse_vendor_d,
    "vendor_e": _parse_vendor_e,
}
```

Each parser handles vendor-specific payload formats while producing the same GenomeDocument output. Parsers accept multiple alternate key names for the same concept (e.g., ["flows", "workflows", "business_rules"] for vendor workflow representations) to handle payload format variations from the same vendor.

4.6 Taxonomized GitHub Storage

The system commits genome artifacts to GitHub using a prescribed hierarchical taxonomy:

```
genomes/
  tenants/
    {tenant_id}/                    <- multi-tenant isolation
      vendors/
        {vendor}/                   <- vendor grouping
          {product_area}/           <- product area (service_catalog, itsm, etc.)
            {module}/               <- specific module (technical_catalog, hr_catalog)
              genome.yaml           <- canonical normalized genome
              graph.yaml            <- structured GenomeGraph
              structure/
                {item_slug}.yaml    <- per-item structure decomposition
              config/
                catalog_config.yaml <- pricing and workflow configuration
              data/
                raw_vendor_payload.json <- original vendor response (scrubbed)
              transformations/      <- created only by translations/re-prompts
```

This structure enables:

- Multi-tenant isolation: Each tenant's genomes are separated
- Cross-vendor discovery: Browse all genomes from a specific vendor
- Granular version control: Per-item YAML files enable meaningful diffs
- Audit trail: Raw vendor payload preserved for lineage traceability
- Bulk operations: Directory-based queries across product areas

4.7 Context-Aware Extraction

The extractor negotiates scan depth with the source platform:

- Deploy phase: Registers an extractor at the vendor with a unique key derived from the target name
- Extract phase: Passes context parameters (tenant, vendor, application, scope, scan_depth) that influence the shape and depth of the extracted payload
- Fallback chain: If the primary self-deploy extractor is unavailable, the system falls back to vendor-specific alternative endpoints

4.8 Idempotency and Deduplication

- Source signatures: SHA-256 hash of the raw vendor payload stored on the ExtractionPayload record, preventing re-extraction of identical payloads

- Name-based deduplication: Within a single extraction, duplicate object/field/workflow names are deduplicated using set-based tracking while preserving discovery order
- Extraction status tracking: ExtractionPayload records track status (pending -> processing -> completed/failed), preventing concurrent processing of the same payload

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented method for automated application genome extraction comprising:

(a) connecting to a source enterprise platform via a configured integration;

(b) deploying a context-aware extractor at the source platform with negotiated scan depth parameters;

(c) extracting application objects, fields, workflows, and relationships from the source platform;

(d) normalizing the vendor-specific extraction payload into a vendor-agnostic intermediate representation;

(e) constructing a GenomeDocument comprising flat lists of object names, field names, workflow names, and relationship strings;

(f) constructing a GenomeGraph comprising typed objects with bound fields, typed relationships with cardinality, and workflows attached to objects;

(g) committing the GenomeDocument, GenomeGraph, and supporting artifacts to a version-controlled repository using a hierarchical taxonomy structure.

Claim 2. A system for automated application genome extraction and storage comprising:

(a) an extractor registry service configured to deploy and manage extractors at source enterprise platforms;

(b) a normalizer configured to convert vendor-specific payloads into a vendor-agnostic format;

(c) a genome builder configured to apply vendor-specific parsers to produce a flat GenomeDocument;

(d) a genome graph builder configured to transform GenomeDocument into a structured GenomeGraph using deterministic heuristic field-to-object binding;

(e) a repository service configured to commit genome artifacts to a version-controlled repository with a prescribed taxonomized directory structure.

Claim 3. A method for deterministic field-to-object binding in an application genome comprising:

(a) constructing a lookup dictionary mapping normalized object name fragments to object identifiers;

(b) for each field name, attempting to match using progressively shorter prefixes of the field name against the lookup dictionary;

(c) assigning matched fields to their corresponding objects;

(d) assigning unmatched fields to a synthetic virtual object.

Dependent Claims

Claim 4. The method of Claim 1 wherein the hierarchical taxonomy structure comprises: tenants/{tenantid}/vendors/{vendor}/{productarea}/{module}/.

Claim 5. The method of Claim 1 wherein the GenomeGraph comprises GenomeObject entities that own their GenomeField entities, with GenomeRelationship and GenomeWorkflow entities stored at the graph level and referenced by identifier from participating objects.

Claim 6. The method of Claim 1 further comprising generating per-item structure files wherein each application item is decomposed into an individual YAML file containing its name, category, description, active status, and typed variables.

Claim 7. The method of Claim 1 wherein the normalizer infers relationships from category hierarchies and variable containment patterns in the vendor payload.

Claim 8. The method of Claim 1 further comprising a fallback chain wherein if a primary extractor is unavailable, the system automatically attempts alternative extraction endpoints.

Claim 9. The method of Claim 1 further comprising source signature computation using a cryptographic hash of the raw vendor payload, enabling idempotent extraction.

Claim 10. The system of Claim 2 wherein the genome builder maintains a pluggable parser registry mapping vendor identifiers to vendor-specific parsing functions, each accepting multiple alternate key names for the same semantic concept.

Claim 11. The system of Claim 2 further comprising an extraction payload store that tracks extraction status, preventing concurrent processing of identical payloads.

Claim 12. The method of Claim 3 wherein the progressively shorter prefix matching operates by splitting the field name on delimiter characters and attempting matches from the longest prefix to shortest.

Claim 13. The method of Claim 3 wherein the synthetic virtual object is created on demand only when unmatched fields exist, and is marked with a type annotation of "virtual" to distinguish it from extracted objects.

Claim 14. The method of Claim 1 further comprising scrubbing secrets from all content prior to committing to the version-controlled repository.

Claim 15. The method of Claim 1 wherein the pipeline implements graceful degradation such that failure at any intermediate stage does not halt the pipeline, and partial results are returned with status information.

Claim 16. The system of Claim 2 wherein the GenomeGraph preserves discovery order of objects, fields, workflows, and relationships throughout all processing stages.

Claim 17. The method of Claim 1 wherein the repository service automatically creates the target repository if it does not exist.

Claim 18. The system of Claim 2 further comprising a discovery service that queries source platform APIs in real-time to identify available applications, catalogs, and modules prior to extraction.

Claim 19. The method of Claim 1 wherein the GenomeRelationship entities include cardinality metadata indicating the nature of the relationship between source and target objects.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

---

6. Advantages Over Prior Art

1. Automation: Eliminates weeks of manual application mapping by automating the entire extraction pipeline
2. Vendor agnosticism: Canonical genome format enables cross-platform migration planning regardless of source or target vendor
3. Dual representation: GenomeDocument provides simple serialization while GenomeGraph enables advanced queries -- both from a single extraction
4. Deterministic reproducibility: Field-to-object binding uses heuristic matching (no LLM), ensuring identical inputs produce identical outputs
5. Graceful degradation: Pipeline continues through intermediate failures, returning partial results rather than failing entirely
6. Auditability: Version-controlled storage with raw payload preservation provides complete lineage traceability
7. Extensibility: Adding new vendor support requires only implementing a parser function; all downstream processing is vendor-agnostic
8. Efficiency: No LLM tokens consumed for core extraction -- deterministic algorithms throughout

---

7. Drawings/Figures Description

Figure 1: System Architecture Overview -- Block diagram showing the six cooperating services (Extractor Registry, Normalizer, GenomeBuilder, GenomeGraphBuilder, OYGenomeBuilderService, GitHubService) with data flow arrows.

Figure 2: Multi-Pass Pipeline Flow -- Sequence diagram showing Pass 1 (Scan: Deploy -> Extract -> Normalize -> Build) and Pass 2 (Expand: Commit to GitHub).

Figure 3: Dual-Schema Genome Representation -- Side-by-side comparison of GenomeDocument (flat lists) and GenomeGraph (structured objects with bound fields, relationships, workflows).

Figure 4: Field-to-Object Binding Algorithm -- Flowchart showing the progressively shorter prefix matching logic with examples.

Figure 5: Taxonomized GitHub Directory Structure -- Tree diagram showing the hierarchical folder structure (tenants/vendors/product_area/module/) with file types at each level.

Figure 6: Vendor Parser Registry -- Diagram showing how vendor-specific parsers are selected from the registry and produce uniform GenomeDocument output.

Figure 7: Graceful Degradation Chain -- Decision tree showing fallback behavior when individual pipeline stages fail.

Figure 8: GenomeGraph Data Model -- UML class diagram showing GenomeObject, GenomeField, GenomeRelationship, GenomeWorkflow, and GenomeGraph with their relationships and cardinalities.

---

8. Inventors

[To be completed by filing attorney]

9. Filing Notes

- Priority date should be established as soon as possible given the novelty of the dual-schema genome representation and deterministic field binding algorithm
- Consider international filing under PCT given the global enterprise software migration market
- The vendor-agnostic design and pluggable parser registry may warrant additional continuation claims as new vendor parsers are added
- The taxonomized GitHub storage structure may warrant a separate design patent filing

---

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Video Genome Extraction

Title

System and Method for Multi-Agent Video Analysis-Based Application Genome Extraction Using Computer Vision, Audio Transcription, and Structural Synthesis

---

1. Field of Invention

This invention relates to computer vision, application reverse engineering, multi-agent AI systems, and enterprise software migration. Specifically, it describes a system and method for extracting the complete structural genome of enterprise applications from video recordings using a coordinated multi-agent pipeline that combines visual frame analysis, audio transcription, UI reverse engineering, and structural synthesis.

---

2. Background and Prior Art Gaps

Current approaches to understanding enterprise application structure for migration purposes require:

- API access: Direct connection to source platform APIs, which may be restricted by security policies, licensing, or technical constraints
- Source code access: Reading configuration files, scripts, or source code, which may not be available for SaaS applications
- Manual documentation: Subject matter experts manually documenting application structure -- error-prone, time-consuming, and often incomplete
- Screen recording for documentation only: Video walkthroughs are recorded but never machine-analyzed; they serve only as reference material for human reviewers

Gaps in prior art:

1. No existing system extracts application structural metadata (objects, fields, workflows, relationships) from video recordings of application walkthroughs.

2. No existing system uses multi-agent AI architecture to decompose video analysis into specialized parallel tasks (frame analysis, UI extraction, audio transcription, structural synthesis).
3. No existing system performs per-screen UI reverse engineering from video frames, producing HTML/CSS skeletons, color palettes, component hierarchies, and typography analysis.
4. No existing system correlates audio narration with visual content by timestamp to enrich structural extraction with spoken context (feature descriptions, business rules, workflow explanations).
5. No existing system produces design tokens (colors, typography, spacing patterns) alongside structural data, enabling both functional and visual replication of the source application.
6. No existing system performs perceptual hash-based frame deduplication with scene-change detection to optimize vision API costs while maintaining coverage.

---

3. Summary of Invention

The present invention provides a multi-agent pipeline that takes video recordings of enterprise applications and extracts their complete structural genome through five specialized AI agents operating in a coordinated directed acyclic graph (DAG):

1. Frame Intelligence Agent -- Extracts frames from video using adaptive intervals, computes perceptual hashes (8x8 grayscale average-hash), and deduplicates near-identical frames using Hamming distance thresholding, producing a set of unique screens with scene identifiers.

2. UI Extraction Agent -- Analyzes each unique screen using a vision LLM to extract HTML/CSS skeletons, color palettes (hex codes with usage context), typography patterns, component hierarchies, interactive elements, and data element labels.

3. Audio/Speech Agent -- Extracts the audio track via ffmpeg, transcribes using the Whisper API with segment-level timestamps, and correlates transcript segments to the nearest video frame by timestamp midpoint matching.

4. Application Structure Agent -- Synthesizes UI analysis and audio transcript into a complete application genome: typed objects with descriptions and key fields, fields with types and validation rules, workflows with steps and triggers, relationships with cardinality, navigation structure, and role/permission observations.

5. Synthesis & Validation Agent -- Merges all agent outputs into a final genome document, validates consistency, resolves conflicts, assigns per-section confidence scores, and produces both backward-compatible flat lists and enriched structured representations.

The pipeline executes Agents 2 and 3 in parallel (as they are independent), streams progress to the client via Server-Sent Events (SSE), and stores extraction results with full metadata for later retrieval and GitHub commit.

---

4. Detailed Description

4.1 Pipeline Architecture (Agent DAG)

```
                 +----------------------+
                 |  Video File Input     |
                 +----------+-----------+
                            v
                 +----------------------+
                 |  Agent 1: Frame      |
         Stage 1 |  Intelligence        |   Sequential
                 |  (scene detection,   |
                 |   deduplication)      |
                 +----------+-----------+
                       +----+----+
                       v         v
            +-------------+ +-------------+
    Stage 2 | Agent 2: UI | | Agent 3:    |   Parallel
            | Extraction  | | Audio/Speech|   (asyncio.gather)
            +------+------+ +------+------+
                   +-------+-------+
                           v
                 +----------------------+
         Stage 3 |  Agent 4: App        |   Sequential
                 |  Structure           |   (depends on 2+3)
                 +----------+-----------+
                            v
                 +----------------------+
         Stage 4 |  Agent 5: Synthesis  |   Sequential
                 |  & Validation        |   (depends on 4)
                 +----------+-----------+
                            v
                 +----------------------+
                 |  Final Genome Output  |
                 |  + Design Tokens      |
                 |  + UI Analysis        |
                 |  + Audio Transcript   |
                 +----------------------+
```

4.2 Agent 1: Frame Intelligence

Purpose: Extract video frames and deduplicate near-identical frames to produce a minimal set of unique screens.

Algorithm:

1. Dense frame extraction: Use ffmpeg to extract frames at a configurable interval (default: every 2 seconds), up to a maximum frame count (default: 40):

```
ffmpeg -i {video} -vf fps=1/{interval} -frames:v {max} -q:v 2 frame_%04d.jpg
```

2. Perceptual hash computation (average-hash algorithm):
   - Convert frame to 8x8 grayscale image using PIL
   - Compute average pixel value across all 64 pixels
   - Generate 64-bit hash: each pixel -> "1" if above average, "0" if below
   - Encode as hexadecimal string
   - Fallback: MD5 hash of raw bytes if PIL unavailable
3. Hamming distance deduplication:
   - Compare each frame's perceptual hash to the most recent unique frame
   - Compute Hamming distance: XOR the two hashes, count set bits
   - If distance > threshold (default: 6), frame is unique -> new scene
   - If distance <= threshold, frame is a duplicate -> skip
4. Scene ID assignment: Each unique frame receives an incrementing scene ID, enabling temporal grouping.

Output: list[FrameInfo] where each FrameInfo contains: index, timestampsec, imagebytes, sceneid, isunique, fingerprint.

Key innovation: Perceptual hashing provides content-aware deduplication that is robust to minor visual changes (cursor movement, loading indicators) while detecting genuine screen transitions. This reduces vision API costs by 40-70% compared to sending all extracted frames.

4.3 Agent 2: UI Extraction

Purpose: Analyze each unique screen to extract detailed UI structure, design tokens, and HTML/CSS skeletons.

Algorithm:

1. Batch construction: Group frames into batches of up to 8 for efficient API usage
2. Content block assembly: For each batch, construct vision LLM content blocks:
   - Text instruction block describing the analysis task
   - Image blocks (base64-encoded JPEG) for each frame
   - Per-frame context labels: [Screen {index} at {timestamp}s -- Scene {scene_id}]
   - Closing instruction block requesting structured JSON output
3. Vision LLM call: Send content blocks with specialized system prompt requesting:
   - components: hierarchical UI component list with positions
   - color_palette: hex codes with usage context (e.g., {"hex": "#1a73e8", "usage": "primary button background"})
   - typography: font sizes, weights, and usage locations
   - layout: page structure type, column count, description
   - html_skeleton: semantic HTML with inline CSS representing the page layout
   - text_labels: all visible text (headings, buttons, menus)
   - data_elements: table columns, form field labels, dropdown options
   - interactive_elements: buttons, inputs, toggles with types and labels
4. Response parsing: Extract JSON from LLM response, handling markdown fences and embedded prose

Output: list[ScreenAnalysis] with full UI decomposition per screen.

Key innovation: Per-screen HTML/CSS skeleton generation from video frames enables visual replication of the source application -- information not available through API-based extraction. Design tokens (colors, typography) capture the application's visual identity.

4.4 Agent 3: Audio/Speech

Purpose: Extract and transcribe audio narration, correlating spoken content with specific video frames.

Algorithm:

1. Audio detection: Use ffprobe to check for audio streams:

```
ffprobe -v error -select_streams a -show_entries stream=codec_type -of csv=p=0 {video}
```

2. Audio extraction: Extract audio track to 16kHz mono WAV:

```
ffmpeg -i {video} -vn -acodec pcm_s16le -ar 16000 -ac 1 -y {output.wav}
```

3. Transcription: Send WAV to OpenAI Whisper API with segment-level timestamps:
   - Model: whisper-1
   - Response format: verbose_json with segment granularity
   - Returns: full text, language detection, per-segment timestamps
4. Frame correlation algorithm (nearest-timestamp midpoint matching):
   - For each transcript segment, compute midpoint: mid = (segment.start + segment.end) / 2
   - Find the frame with the nearest timestamp: argmin(|mid - frame_timestamp|)
   - Build correlation map: {frameindex: [textsegments]}

Output: TranscriptResult with segments, full text, duration, language; plus dict[int, list[str]] correlation map.

Key innovation: Timestamp-based correlation between audio and video frames enables the system to associate spoken descriptions ("this is the approval workflow") with the specific screen being discussed, enriching structural extraction with context not visible in the UI.

4.5 Agent 4: Application Structure

Purpose: Synthesize UI analysis and audio transcript into a complete application genome.

Algorithm:

1. Context assembly: Build a structured prompt containing:
   - Per-screen UI analysis (components, data elements, interactive elements, text labels, layout)
   - Full audio transcript
   - Transcript-to-screen correlations (which words were spoken about which screen)
2. LLM synthesis: Send assembled context to LLM with specialized system prompt requesting:
   - objects: typed entities with descriptions and key fields (e.g., {"name": "incident", "description": "Tracks IT disruptions", "key_fields": ["number", "priority"]})
   - fields: typed fields with object binding, validation rules, and optionality (e.g., {"name": "priority", "type": "choice", "object": "incident", "options": [...]})
   - workflows: named processes with steps, triggers, and descriptions

- relationships: typed connections with cardinality (e.g., {"from": "incident", "to": "change", "type": "1:N"})
- navigation: application menu structure
- roles_permissions: observed access control patterns
- integrations: external system references

3. Response parsing: Extract structured JSON, populate AppStructureResult dataclass

Output: AppStructureResult with typed objects, fields, workflows, relationships, navigation, roles, integrations, confidence score, and reasoning chain.

Key innovation: Cross-referencing visual UI elements with spoken audio descriptions produces a more complete genome than either signal alone. Objects mentioned in narration but not visible on screen are still captured (with lower confidence).

4.6 Agent 5: Synthesis & Validation

Purpose: Merge all agent outputs into a final validated genome with both backward-compatible and enriched representations.

Algorithm:

1. Input aggregation:
   - Application structure (objects, fields, workflows, relationships) from Agent 4
   - Aggregated design tokens (colors, typography, layouts) from Agent 2
   - Audio context summary from Agent 3
2. LLM-based merge and validation: Send aggregated inputs to LLM requesting:
   - Consistency validation (cross-reference UI observations with structural claims)
   - Duplicate resolution (merge overlapping objects/fields from different screens)
   - Confidence scoring (per-section reliability assessment)
   - Backward-compatible output: genome_document with flat string lists (objects, fields, workflows, relationships)
   - Enriched output: structuredobjects, structuredfields, structuredworkflows, structuredrelationships with typed details
   - Design tokens: unified color palette with CSS variable suggestions, typography patterns, layout observations
3. Fallback merge: If LLM synthesis fails, perform mechanical merge:

- Extract names from structured objects/fields/workflows
- Build flat relationship strings from typed relationship objects
- Aggregate color palettes from all screen analyses
- Return with lower confidence score

Output: SynthesisResult with final genome dict, design tokens, validation notes, confidence score, and reasoning chain.

Key innovation: The fallback mechanical merge ensures the system always produces usable output even when the synthesis LLM call fails, providing graceful degradation.

4.7 Pipeline Orchestration and Progress Streaming

The orchestrator coordinates the five agents using Python's asyncio:

1. Stage 1 (sequential): Frame Intelligence
2. Stage 2 (parallel): UI Extraction + Audio/Speech via asyncio.gather()
3. Stage 3 (sequential): Application Structure (depends on Stages 1+2)
4. Stage 4 (sequential): Synthesis & Validation (depends on Stage 3)

Progress is streamed to the client via Server-Sent Events (SSE):

- Each agent emits running and done (or error) events with metadata
- Events are bridged from the async pipeline to the SSE response via asyncio.Queue
- The client renders a real-time progress panel showing each agent's status

4.8 Extraction Metadata

Each extraction produces comprehensive metadata:

```
{
  "_extraction_metadata": {
    "pipeline": "multi-agent-v1",
    "frame_count": 18,
    "unique_screens": 12,
    "has_audio": true,
    "audio_duration_sec": 45.3,
    "total_latency_ms": 67000,
    "stage_latencies": {
      "frame_intelligence": 2100,
      "ui_extraction_and_audio": 38000,
      "app_structure": 15000,
```

```
      "synthesis": 12000
    }
  }
}
```

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented method for extracting an application genome from a video recording comprising:

(a) extracting a plurality of frames from the video recording at adaptive intervals;

(b) computing a perceptual hash for each extracted frame;

(c) deduplicating frames by comparing perceptual hashes using Hamming distance thresholding to produce a set of unique screens;

(d) analyzing each unique screen using a vision language model to extract UI components, color palettes, typography patterns, layout structure, and an HTML/CSS skeleton;

(e) extracting and transcribing audio from the video recording with segment-level timestamps;

(f) correlating audio transcript segments to video frames by timestamp midpoint matching;

(g) synthesizing the UI analysis and correlated audio transcript into a structured application genome comprising typed objects, fields, workflows, and relationships;

(h) validating and merging the synthesized genome into a final output with confidence scoring.

Claim 2. A multi-agent system for video-based application genome extraction comprising:

(a) a frame intelligence agent configured to extract frames, compute perceptual hashes, and deduplicate frames using Hamming distance;

(b) a UI extraction agent configured to analyze frames using a vision language model and produce per-screen HTML/CSS skeletons, color palettes, and component hierarchies;

(c) an audio/speech agent configured to extract audio, transcribe with timestamps, and correlate transcript segments to frames;

(d) an application structure agent configured to synthesize UI analysis and audio transcript into typed application structure;

(e) a synthesis agent configured to merge all agent outputs, validate consistency, and produce a final genome;

(f) an orchestrator configured to execute agents (b) and (c) in parallel and stream progress via server-sent events.

Claim 3. A method for perceptual hash-based video frame deduplication for vision API cost optimization comprising:

(a) extracting frames at regular intervals from a video;

(b) for each frame, converting to grayscale, downscaling to an NxN grid, computing average pixel value, and generating a binary hash where each bit indicates whether a pixel exceeds the average;

(c) computing Hamming distance between consecutive frame hashes;

(d) retaining only frames whose Hamming distance from the most recent retained frame exceeds a threshold;

(e) assigning scene identifiers to retained frames.

Dependent Claims

Claim 4. The method of Claim 1 wherein the perceptual hash is an average-hash computed over an 8x8 grayscale downscale of the frame.

Claim 5. The method of Claim 1 wherein the UI extraction produces design tokens comprising color hex codes with usage descriptions, typography patterns, and CSS variable suggestions.

Claim 6. The method of Claim 1 wherein the audio transcript correlation uses nearest-neighbor matching of transcript segment midpoint timestamps to frame extraction timestamps.

Claim 7. The method of Claim 1 wherein the application structure synthesis cross-references spoken audio descriptions with visual UI observations, capturing objects mentioned in narration but not visible on screen.

Claim 8. The system of Claim 2 wherein the orchestrator uses an asyncio queue to bridge agent progress events to the SSE response stream.

Claim 9. The method of Claim 1 further comprising a fallback mechanical merge that produces a valid genome output without LLM synthesis when the synthesis agent fails.

Claim 10. The method of Claim 1 wherein the final genome output comprises both backward-compatible flat string lists and enriched structured representations with typed fields, relationships with cardinality, and workflows with steps.

Claim 11. The system of Claim 2 further comprising a persistent extraction record storing the genome, UI analysis, audio transcript, design tokens, agent progress, and extraction metadata.

Claim 12. The method of Claim 1 wherein the HTML/CSS skeleton generation produces semantic HTML with inline styles sufficient to recreate the visual layout of each screen.

Claim 13. The method of Claim 3 wherein a fallback hash is computed using MD5 of the raw frame bytes when the image processing library is unavailable.

Claim 14. The method of Claim 1 further comprising streaming per-agent progress events to the client in real-time during extraction.

Claim 15. The system of Claim 2 wherein the UI extraction agent processes frames in configurable batches to stay within vision API token limits.

Claim 16. The method of Claim 1 wherein the video recording requires no API access to the source application, enabling genome extraction from any recorded application walkthrough.

Claim 17. The method of Claim 1 further comprising extracting interactive elements (buttons, inputs, toggles) with their labels and visual styles from each screen.

Claim 18. The system of Claim 2 wherein extraction results are stored with mandatory taxonomy fields (vendor, product area) and optional module field, enabling organized repository storage.

Claim 19. The method of Claim 1 further comprising per-section confidence scoring in the final genome output, indicating reliability of each extracted component.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

---

6. Advantages Over Prior Art

1. No API access required: Extracts application structure from video recordings, enabling genome extraction from any application that can be screen-recorded -- including applications with restricted API access
2. Multi-signal correlation: Combines visual (UI layout, colors, components) and auditory (narration, feature descriptions) signals for more complete extraction than either alone
3. UI design capture: Extracts visual design details (color palettes, typography, HTML/CSS patterns) that are not available through API-based extraction
4. Cost-optimized vision analysis: Perceptual hash deduplication reduces vision API costs by 40-70% by eliminating near-duplicate frames
5. Parallel agent execution: UI extraction and audio transcription run concurrently, reducing total pipeline latency
6. Graceful degradation: Fallback mechanical merge ensures output even when LLM synthesis fails; audio agent degrades gracefully if no audio track or no Whisper API key

7. Real-time progress visibility: SSE streaming provides users with live feedback on each agent's status
8. Backward compatibility: Output includes both flat string lists (compatible with existing systems) and enriched structured representations

---

7. Drawings/Figures Description

Figure 1: Multi-Agent Pipeline DAG -- Directed acyclic graph showing the five agents with sequential and parallel edges, data flow between agents, and stage numbering.

Figure 2: Perceptual Hash Computation -- Step-by-step diagram showing: original frame -> 8x8 grayscale -> pixel values -> average computation -> binary hash -> hex encoding.

Figure 3: Hamming Distance Deduplication -- Timeline showing extracted frames, their perceptual hashes, computed Hamming distances, and which frames are retained vs. skipped.

Figure 4: Audio-to-Frame Correlation -- Timeline showing audio segments with timestamps aligned to video frames, with correlation arrows from segment midpoints to nearest frames.

Figure 5: Per-Screen UI Extraction Output -- Example screen analysis showing the original frame alongside extracted components, color palette swatches, HTML skeleton, and interactive elements.

Figure 6: Multi-Signal Synthesis -- Diagram showing how UI analysis (visual signal) and audio transcript (auditory signal) are combined to produce richer structural output than either alone.

Figure 7: SSE Progress Streaming Architecture -- Sequence diagram showing the asyncio.Queue bridge between pipeline agents and the SSE response stream.

Figure 8: Final Genome Output Structure -- Diagram showing the dual output: backward-compatible flat lists alongside enriched structured representations with design tokens.

---

8. Inventors

[To be completed by filing attorney]

9. Filing Notes

- The multi-agent architecture with parallel execution and SSE streaming is a strong system claim
- The perceptual hashing algorithm for frame deduplication may warrant independent continuation claims
- The audio-to-frame correlation algorithm (nearest-timestamp midpoint matching) is a novel method claim
- Consider international filing given the global enterprise software market
- Prior art search should focus on: video analysis for UI testing (different purpose), screen recording tools (no structural extraction), automated documentation (typically from code, not video)

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Amortized AI Transformation System with Reusable Translation Recipes

Title

System and Method for Amortized AI-Driven Application Transformation Through Reusable Translation Recipes with Compressed Transformation Pattern Encoding and Measurable Computational, Financial, and Environmental Cost Reduction

1. Field of Invention

This invention relates to AI transformation cost amortization, computational resource optimization, enterprise application migration, reusable transformation pattern encoding, and sustainable computing. Specifically, it describes a system and method for:

1. Capturing transformation intent from interactive AI-mediated transformation sessions and encoding it as reusable translation recipes -- compressed representations of multi-step reasoning that can be reapplied across multiple datasets in a single AI inference call.

2. Amortizing AI transformation cost by replacing repeated iterative prompting sessions with single-call recipe application, achieving measurable 3-4x efficiency gains in token consumption, latency, financial cost, and energy expenditure at portfolio scale.

3. Compressing multi-step transformation reasoning into structured hybrid instruction encodings that combine structured metadata (source type, target platform, output schema) with natural language transformation logic, producing a compact representation that preserves the full fidelity of multi-turn interactive sessions.

4. Optimizing transformation efficiency at portfolio scale by enabling a single transformation pattern capture to serve an unlimited number of structurally similar applications, with per-application marginal cost approaching a fixed floor regardless of transformation complexity.

2. Background and Prior Art Gaps

Enterprise application transformation between platforms requires converting application configurations, data models, and workflows from a source format to a target format. Current approaches include:

- Manual transformation: Developers manually rewrite configurations for the target platform -- slow, error-prone, and expensive.
- Iterative LLM interaction: Users interact with AI assistants in conversational sessions, iteratively refining transformations through multiple prompt-response cycles -- each cycle consumes LLM tokens and computational resources. For a single application, this typically requires 3-8 iterative sessions. For a portfolio of N similar applications, this cost scales linearly: N applications x 3-8 sessions x tokens per session.
- One-off scripts: Custom migration scripts written per-application -- not reusable across similar applications.
- Platform-specific migration tools: Vendor-provided tools that handle specific migration paths but require separate tooling for each source-target pair.

The fundamental inefficiency in current AI-mediated transformation is that transformation intelligence is discarded after each session. When a user successfully transforms Application A through iterative prompting, the reasoning, decisions, refinements, and validated patterns from that session are lost. When the user transforms structurally similar Application B, they must repeat the entire iterative process from scratch. This represents a massive waste of computational resources, user time, and energy.

Gaps in prior art:

1. No existing system captures transformation intent from interactive AI sessions and reapplies it across multiple datasets to reduce computational cost.
2. No existing system reverse-engineers interactive transformation sessions into stored, parameterized recipes that encode the user's transformation intent as compressed representations of multi-step reasoning.
3. No existing system encodes transformation logic as a structured + natural language hybrid instruction format that combines machine-parseable metadata with LLM-consumable natural language instructions.
4. No existing system achieves single-call transformation of complex application artifacts by applying pre-captured transformation patterns, replacing 3-8 iterative prompting sessions with a single inference call.
5. No existing system measures, tracks, and reports computational resource consumption reduction (tokens, energy, emissions, cost) achieved through transformation pattern reuse.
6. No existing system preserves original source integrity while storing transformations in a

separate namespace, enabling non-destructive iteration.

7. No existing system provides a recipe marketplace model where transformation recipes developed for one application can be discovered, shared across tenants, and applied to structurally similar applications.

8. No existing system compresses multi-step AI reasoning into reusable instruction artifacts that preserve the full transformation fidelity of iterative sessions in a single-call-consumable format.

---

3. Summary of Invention

The present invention provides an amortized AI transformation system that captures transformation patterns from interactive sessions and reapplies them across multiple datasets with dramatically reduced computational cost. The system operates on the principle that transformation intelligence should be captured once and applied many times -- analogous to compiling a function once and calling it repeatedly, but in the domain of AI-mediated reasoning.

The system comprises:

1. Interactive Transformation Studio: Users transform application genome content through conversational prompts in a studio environment. The LLM receives full repository context (file tree, configuration files, current genome) and produces a filesystem plan (new branch, folder structure, file contents). Each interactive session typically requires 3-8 iterative prompt-response cycles to achieve a satisfactory transformation.

2. Transformation Pattern Capture (Recipe Generation): The system automatically reverse-engineers completed interactive transformation sessions into reusable translation recipes. A specialized AI call analyzes the original genome content, the output files produced, and the user's conversation context to produce a compressed representation of the multi-step reasoning that drove the transformation. This compression captures the essential transformation logic while discarding session-specific artifacts (false starts, corrections, conversational scaffolding).

3. Hybrid Instruction Encoding: Recipes are stored in a structured + natural language hybrid format that combines:

   - Structured metadata: Source type, target platform, expected output structure (folders and files), status, version -- machine-parseable fields that enable recipe discovery, matching, and validation.

   - Natural language instructions: Complete, self-contained transformation logic expressed in

natural language that an LLM can follow -- the compressed encoding of the multi-step reasoning from the original interactive session.

This hybrid encoding is the core innovation: structured fields enable programmatic recipe management while natural language instructions preserve the full expressive power of the original transformation reasoning.

4. Single-Call Recipe Application: Stored recipes are applied to new genomes in a single LLM inference call. The system loads the full repository context, injects the recipe's hybrid instructions into the prompt, and produces a complete filesystem plan -- replacing what would otherwise require 3-8 iterative sessions. The single-call approach is possible because the recipe's compressed instructions encode the cumulative reasoning of the original multi-step session, providing the LLM with pre-validated transformation logic rather than requiring it to discover the transformation through iteration.

5. Non-Destructive Storage: All transformation outputs are written to a transformations/ subfolder within the source genome's directory, preserving the original genome files. Each transformation creates a new Git branch, never committing to main.

6. Efficiency Tracking and Reporting: The system tracks token consumption, latency, and estimated cost for every transformation call (interactive and recipe-based), enabling measurement and reporting of:
   - Per-recipe amortized savings (tokens saved per application)
   - Portfolio-level cumulative savings (total tokens, cost, energy, emissions avoided)
   - Efficiency ratio (interactive cost / recipe cost) per recipe
   - Projected savings for planned portfolio migrations

7. Recipe Marketplace: Recipes are tenant-scoped but can be published to a shared marketplace where transformation patterns developed by one tenant can be discovered and applied by other tenants with structurally similar applications, multiplying the amortization benefit across the entire user base.

---

4. Detailed Description

4.1 System Architecture

```
+-------------------------------------------------------------+
|                     Genome Studio                           |
```

```
|                                                                  |
|  +----------+    +--------------+    +-------------------+   |
|  |  Chat     |    |  Transform   |    |  Translation      |   |
|  |  (Q&A)    |    |  (Modify)    |    |  (Apply Recipe)   |   |
|  +----+-----+    +------+-------+    +-------+-----------+   |
|       |                 |                     |               |
|       v                 v                     v               |
|  +-----------------------------------------------------+    |
|  |              LLM Call (call_llm)                     |    |
|  |  Provider-agnostic: Anthropic / OpenAI / other       |    |
|  |  Full repo context: file tree + config files         |    |
|  +----------------------+------------------------------+    |
|                         v                                    |
|  +-----------------------------------------------------+    |
|  |              Filesystem Plan                         |    |
|  |  { branch_name, base_path, folders, files[] }        |    |
|  +----------------------+------------------------------+    |
|                         v                                    |
|  +-----------------------------------------------------+    |
|  |              Version-Controlled Commit                |    |
|  |  New branch -> create files -> non-destructive storage  |    |
|  |  transformations/ subfolder                          |    |
|  +-----------------------------------------------------+    |
|                         <->                                  |
|  +-----------------------------------------------------+    |
|  |           Efficiency Tracking Layer                   |    |
|  |  Token counts, latency, cost, energy per call        |    |
|  |  Cumulative savings per recipe + per portfolio        |    |
|  +-----------------------------------------------------+    |
+-------------------------------------------------------------+
                         <->
+-------------------------------------------------------------+
|               Recipe Lifecycle                               |
|                                                              |
|  +--------------+  +--------------+  +-------------------+ |
|  | Capture       |  | Store        |  | Apply             | |
|  | (reverse-     |->->| Translation  |->->| to New Genome     | |
|  |  engineer     |  | Record       |  | (single call)     | |
|  |  session)     |  | (hybrid      |  |                   | |
|  |              |  |  encoding)   |  | Track savings     | |
|  +--------------+  +------+-------+  +-------------------+ |
|                           v                                  |
|  +-----------------------------------------------------+    |
|  |           Recipe Marketplace                          |    |
|  |  Publish -> Discover -> Apply across tenants          |    |
|  |  Usage analytics, ratings, savings leaderboard        |    |
|  +-----------------------------------------------------+    |
+-------------------------------------------------------------+
```

4.2 The Studio Transform Flow

When a user transforms genome content interactively:

1. Context loading: The system automatically reads the full repository file tree and key configuration files (up to configurable limits) for context
2. Prompt construction: User's instruction is combined with repository context, current file path, and a generated branch name
3. LLM call: The transform system prompt instructs the LLM to return a structured response with:
   - reasoning: step-by-step thought process
   - explanation: what was done and why
   - filesystem_plan: exact files and folders to create
   - diff: summary of changes
   - preview: short preview of the result
4. Filesystem plan validation: System ensures branchname, basepath, files, and folders are present
5. Non-destructive rule: All new files MUST be created under a transformations/ folder within the base_path. Original genome files are never overwritten.
6. Version-controlled commit: Creates a new branch, writes each file, returns commit status
7. Efficiency tracking: Records token counts (input + output), latency, estimated cost, and estimated energy for the call

4.3 Transformation Pattern Capture (Recipe Generation via Reverse Engineering)

The recipe generation process reverse-engineers an interactive transformation session into a reusable translation recipe -- a compressed representation of the multi-step reasoning that produced the successful transformation.

Input:

- Original genome content (source)
- Output files produced by the transformation
- Chat context (summary of user's requests and refinements across all iterative turns)
- Source type and target platform

Process:

1. A specialized system prompt instructs the LLM to analyze the complete transformation context and produce a reusable recipe
2. The LLM examines:

- What the original genome looked like (structure, entities, complexity)
- What files were produced (output structure, content patterns)
- What the user asked for across all iterative turns (transformation intent)
- What refinements were applied (corrections, additions, format changes)
- What the final validated output looks like (quality benchmark)

3. The LLM generates a compressed instruction set:
   - instructions: Complete, self-contained prompt that another LLM can follow to reproduce the transformation on a different genome of the same type -- this is the compressed representation of the entire multi-turn session
   - output_structure: Expected folders and files the recipe produces
   - suggested_description: One-sentence description of the transformation pattern

Key design principle -- compression of multi-step reasoning: The generated instructions encode the cumulative result of the user's iterative refinement process. A 5-turn interactive session where the user said "transform this," then "add workflow steps," then "fix the schema format," then "add API routes," then "include validation rules" is compressed into a single instruction set that says "transform this genome into [target] including workflow steps, properly formatted schema, API routes, and validation rules." The instructions preserve the transformation's final quality while eliminating the iterative discovery process.

Key design principle -- vendor-specific but instance-agnostic: The generated instructions describe the transformation pattern (e.g., "convert enterprise catalog genome to web application") at the type level, not the specific data. The instructions reference structural patterns ("for each catalog variable, create a form field") rather than specific values ("create a form field for 'Employee Name'").

4.4 Hybrid Instruction Encoding (Recipe Storage)

Recipes are stored as structured records combining machine-parseable metadata with natural language transformation logic:

```
Translation Record:
  id:               unique identifier
  tenant_id:        owning tenant
  name:             human-readable recipe name
  description:      one-sentence description
  source_vendor:    source system type
  source_type:      source artifact category
  target_platform:  target system or format
  instructions:     [NATURAL LANGUAGE] complete transformation logic
  output_structure: [STRUCTURED] expected folders and files
  status:           active | draft | archived
  version:          recipe version number
```

```
  marketplace:      [STRUCTURED] publication and sharing metadata
  efficiency_stats: [STRUCTURED] tracked savings metrics
```

The instructions field is the core innovation -- it encodes the complete transformation logic as natural language instructions that an LLM can follow. This is the compressed representation of multi-step reasoning: what required 3-8 interactive turns to discover and refine is encoded in a single instruction block that can be injected into a single prompt.

The hybrid encoding serves dual purposes:

- Structured fields enable programmatic recipe management: discovery by source/target type, version control, marketplace search, efficiency tracking, validation of output structure
- Natural language instructions preserve the full expressive power of the original transformation reasoning: complex conditional logic, domain-specific rules, quality expectations, and output formatting requirements that would be difficult or impossible to encode in a purely structured format

4.5 Single-Call Recipe Application

The recipe application endpoint applies a stored recipe to a new genome in a single LLM inference call, replacing 3-8 iterative sessions:

1. Load recipe: Retrieve the Translation record with its hybrid-encoded instructions
2. Load context: Read full repository file tree + key configuration files (same as interactive transform)
3. Construct single prompt: Combine:
   - Repository file tree
   - Key configuration file contents (up to configurable limits)
   - Currently selected file content
   - Complete recipe metadata (name, source type, target platform, output structure)
   - Recipe instructions (the compressed transformation logic)
4. Single LLM call: Send the complete prompt and receive the full transformation result
5. Parse result: Extract filesystem plan from response
6. Same output format: Returns reasoning, explanation, filesystem_plan, diff, preview -- identical to interactive transform
7. Efficiency tracking: Records tokens used and computes savings versus interactive baseline

Why single-call application works: The recipe's instructions encode the cumulative reasoning of the original multi-turn session.

The LLM does not need to iteratively discover the transformation through trial and refinement because the recipe provides:

- Pre-validated transformation rules: Every instruction in the recipe was validated against a successful prior transformation
- Comprehensive coverage: The recipe includes all refinements from the original session (schema formatting, workflow steps, API routes, validation rules) in a single instruction block
- Quality benchmarks: The expected output structure tells the LLM what a successful result looks like
- Elimination of false starts: The original session's dead ends, corrections, and abandoned approaches are not encoded in the recipe -- only the final validated logic

4.6 Transformation Pattern Compression

Translation recipes function as compressed representations of multi-step AI reasoning chains. This section describes the compression mechanism and its implications.

4.6.1 The Compression Model

An interactive transformation session is a multi-step reasoning process:

```
Session (uncompressed):
  Turn 1: User: "Transform this genome into a web app"
           LLM: [8,000 input tokens -> 4,000 output tokens] -> partial result
  Turn 2: User: "Add the workflow steps from the genome"
           LLM: [10,000 input -> 4,000 output] -> improved result
  Turn 3: User: "Fix the schema format to match standard"
           LLM: [12,000 input -> 4,000 output] -> corrected result
  Turn 4: User: "Add API routes for each operation"
           LLM: [12,000 input -> 5,000 output] -> near-final result
  Turn 5: User: "Include validation rules from business rules"
           LLM: [14,000 input -> 5,000 output] -> final result

  Total: 56,000 input + 22,000 output = 78,000 tokens
  Reasoning steps: 5 sequential LLM calls with accumulating context
```

The recipe generation process compresses this into:

```
Recipe (compressed):
  Instructions: "Transform the genome into a web application including:
    1. Screen specifications derived from catalog variables with form fields
    2. Workflow steps preserving the genome's process flow with actors
    3. Data schema in [standard format] with types and constraints
    4. API routes for each CRUD operation plus workflow transitions
    5. Validation rules derived from business rules in the genome
```

```
    Expected output: master_prompt.md, schema.json, api_routes.yaml"

  Instruction length: ~500-2,000 tokens
  Application cost: ~12,000 input + 4,000 output = 16,000 tokens
  Compression ratio: 78,000 / 16,000 = 4.9x
```

The recipe compresses multi-step reasoning by:

1. Eliminating iterative discovery: The user's progressive refinement ("add workflows," "fix schema," "add APIs") is collapsed into a single comprehensive instruction set
2. Removing conversational scaffolding: Acknowledgments, clarifications, and status updates from the interactive session are discarded
3. Discarding dead ends: Failed approaches, corrections, and abandoned strategies from the original session are not encoded
4. Preserving decision outcomes: Only the final validated transformation logic is retained -- the "what to do" without the "how we figured out what to do"

4.6.2 Compression Fidelity

The compressed recipe preserves the transformation's output quality because:

- The natural language instructions encode the cumulative result of all refinement turns
- The expected output structure provides a quality benchmark that the applying LLM can validate against
- The instructions are generated by an LLM that analyzed both the input and the final successful output, ensuring the instructions are sufficient to reproduce the output quality

4.6.3 Compression as Amortization

The compression enables amortization: the cost of discovering the transformation pattern (78,000 tokens in the example) is paid once during the initial interactive session + recipe generation. Each subsequent application pays only the compressed cost (16,000 tokens). The amortized cost per application approaches 16,000 tokens as the number of applications increases, yielding a 4-5x reduction compared to repeated interactive sessions.

4.7 Seeded Recipe Examples

The system ships with production-ready recipes for common transformation paths:

Recipe 1: Enterprise Catalog -> Web Application

```
Instructions: "You are translating an enterprise service catalog genome into a
```

```
web application. Given the genome YAML content, produce:
1. A master_prompt.md -- comprehensive prompt for building the app
2. A catalog_summary.json -- structured JSON summary of the catalog item
3. A config file -- project configuration

The master prompt should describe the UI, API routes, data models, and workflows
that replicate the catalog item as a standalone web application.
Include all fields, validation rules, and approval workflows from the genome."

Output structure: {
  folders: ["transformations/web-app"],
  files: ["master_prompt.md", "catalog_summary.json", "config"]
}
```

Recipe 2: Enterprise Catalog -> Repository Structure

```
Instructions: "You are translating an enterprise service catalog genome into a
repository structure. Given the genome YAML content, produce:
1. A README.md -- project overview, setup instructions, architecture notes
2. A schema.json -- data model schema derived from genome objects and fields
3. A migration_plan.md -- step-by-step migration guide

The README should explain what the original application does,
its key workflows, and how the migrated version preserves functionality."

Output structure: {
  folders: ["transformations/repo-structure"],
  files: ["README.md", "schema.json", "migration_plan.md"]
}
```

4.8 Non-Destructive Transformation Storage

All transformation outputs are stored in a transformations/ subfolder:

```
genomes/tenants/{tenant}/vendors/{vendor}/{product_area}/{module}/
  genome.yaml                    <- ORIGINAL (never modified)
  graph.yaml                     <- ORIGINAL
  structure/                     <- ORIGINAL
  config/                        <- ORIGINAL
  data/                          <- ORIGINAL
  transformations/               <- All modifications go here
    web-app/
      master_prompt.md
      catalog_summary.json
      config
    repo-structure/
      README.md
      schema.json
      migration_plan.md
```

This is enforced at three levels:

1. System prompt: LLM instructed to create files under transformations/ only
2. Branch convention: Each transformation creates a new Git branch (never commits to main)
3. Code validation: Filesystem plan validation ensures base_path is within the genome directory

4.9 Token Consumption Analysis and Efficiency Gains

4.9.1 Per-Application Cost Model

Interactive Studio Approach (without recipes):

A typical transformation requires 3-8 iterative LLM calls as the user refines the output:

Session Step	Input Tokens	Output Tokens	Total
Initial transform prompt	~8,000	~4,000	12,000
Refinement 1 ("add workflow steps")	~10,000	~4,000	14,000
Refinement 2 ("fix the schema format")	~12,000	~4,000	16,000
Refinement 3 ("add API route details")	~12,000	~5,000	17,000
Chat Q&A (2 questions)	~6,000	~2,000	8,000
Total per application			**~67,000**

Recipe Approach:

Step	Input Tokens	Output Tokens	Total
First application (interactive, includes	~67,000	+ ~8,000 (recipe gen)	75,000
Each subsequent application (single pass)	~12,000	~4,000	16,000

4.9.2 Portfolio-Level Efficiency Gains

For a portfolio of N similar applications:

Portfolio Size (N)	Interactive (tokens)	Recipe-Based (tokens)	Reduction	Efficiency Multiplier
1	67,000	75,000	-12% (investment)	0.9x
5	335,000	139,000	59%	2.4x
10	670,000	219,000	67%	**3.1x**
20	1,340,000	379,000	72%	**3.5x**
50	3,350,000	859,000	74%	**3.9x**
100	6,700,000	1,659,000	75%	**4.0x**

500	33,500,000	8,059,000	76%	**4.2x**

Key efficiency characteristics:

- Breakeven point: The recipe approach becomes more efficient at N=2 (total cost parity)
- 3x efficiency: Achieved at portfolio sizes of approximately N=10
- 4x efficiency: Achieved at portfolio sizes of approximately N=100
- Asymptotic limit: As N->inf, the per-application cost approaches 16,000 tokens (recipe application only), compared to 67,000 tokens (interactive) -- a theoretical maximum of 4.2x reduction
- Marginal cost: Each additional application beyond the first costs only 16,000 tokens -- a fixed floor regardless of transformation complexity

4.9.3 Financial Cost Reduction

Using representative LLM pricing (varies by provider and model):

Portfolio Size	Interactive Cost	Recipe-Based Cost	Savings
10 applications	~$20.10	~$6.57	$13.53 (67%)
100 applications	~$201.00	~$49.77	$151.23 (75%)
1,000 applications	~$2,010.00	~$481.77	$1,528.23 (76%)

For enterprise customers with large application portfolios (100-1,000+ applications to migrate), the financial savings are substantial.

4.9.4 Latency Reduction

Beyond token cost, recipe application reduces end-to-end transformation latency:

- Interactive: 5 sequential LLM calls x ~15-30 seconds each + user think time = 10-30 minutes per application
- Recipe: 1 LLM call x ~15-30 seconds = under 1 minute per application
- Latency improvement: 10-60x for the transformation itself (excluding recipe creation)

4.10 Environmental Impact Analysis

4.10.1 Energy and Emissions Model

Using published estimates for LLM inference energy consumption:

- Typical LLM inference: ~0.004 kWh per 1,000 tokens (varies by model and provider)
- US grid average CO2 intensity: 0.39 kg CO2e per kWh

For a 100-application migration portfolio:

Approach	Total Tokens	Energy (kWh)	CO2e (kg)
Interactive (no recipes)	6,700,000	26.8	10.5
Recipe-based	1,659,000	6.6	2.6
Savings	**5,041,000**	**20.2**	**7.9**

4.10.2 Scaled Environmental Impact

For large enterprise migrations (1,000 applications across an organization):

Approach	Total Tokens	Energy (kWh)	CO2e (kg)
Interactive	67,000,000	268.0	104.5
Recipe-based	16,075,000	64.3	25.1
Savings	**50,925,000**	**203.7**	**79.4**

The recipe-based approach reduces electricity consumption by 76% and greenhouse gas emissions by 76% for large migration portfolios. When scaled across multiple enterprises using the recipe marketplace, the cumulative environmental benefit compounds further.

4.11 Recipe Marketplace

The recipe marketplace enables transformation pattern sharing across tenants, multiplying the amortization benefit:

4.11.1 Marketplace Architecture

- Publication: A tenant publishes a recipe to the marketplace, making it discoverable by other tenants. Publication includes recipe metadata, description, expected input/output, and efficiency statistics.
- Discovery: Tenants search the marketplace by source type, target platform, or keyword. Recipes are ranked by usage count, success rate, and efficiency metrics.
- Application: A tenant applies a marketplace recipe to their genome. The recipe instructions execute in the applying tenant's context with their genome data -- no data is shared between tenants.
- Feedback: Tenants rate recipes and report issues, creating a quality signal that improves

marketplace discovery.

4.11.2 Cross-Tenant Amortization

When a recipe is shared via the marketplace, the amortization benefit extends beyond the originating tenant:

- Tenant A creates a recipe through interactive transformation (cost: 75,000 tokens)
- Tenant A applies the recipe to 9 additional applications (cost: 144,000 tokens)
- Tenant B discovers and applies the same recipe to 20 applications (cost: 320,000 tokens, saving 1,020,000 tokens versus interactive)
- Tenant C applies to 50 applications (cost: 800,000 tokens, saving 2,550,000 tokens)

Total tokens consumed: 1,339,000

Total tokens that would have been consumed interatively: 5,360,000

Cross-tenant efficiency: 4.0x

The marketplace converts recipe creation from a per-tenant cost into a shared infrastructure benefit.

4.11.3 Marketplace Analytics

The marketplace tracks and displays:

- Total applications transformed per recipe
- Total tokens saved per recipe (cumulative across all tenants)
- Total estimated CO2e avoided per recipe
- Average efficiency ratio (interactive cost / recipe cost)
- Success rate (percentage of applications where the recipe produced a valid result)

4.12 Efficiency Tracking and Reporting

The system tracks computational resource consumption at every transformation call and provides reporting at multiple levels:

4.12.1 Per-Call Tracking

Each LLM call records:

- Input token count
- Output token count

- Total token count
- Call latency (milliseconds)
- Estimated financial cost (based on provider pricing)
- Estimated energy consumption (kWh)
- Estimated CO2e emissions (kg)
- Call type (interactive / recipe-generation / recipe-application)
- Recipe ID (if recipe-based)

4.12.2 Per-Recipe Reporting

Each recipe maintains cumulative statistics:

- Number of times applied
- Total tokens consumed via recipe application
- Estimated tokens that would have been consumed via interactive approach
- Token savings (absolute and percentage)
- Financial savings (absolute and percentage)
- Energy savings (kWh) and emissions savings (CO2e kg)
- Average efficiency ratio

4.12.3 Portfolio-Level Dashboard

For migration portfolios, the system provides:

- Total applications transformed (interactive vs. recipe-based)
- Cumulative token consumption and savings
- Cumulative financial cost and savings
- Cumulative energy and emissions impact
- Efficiency trend over time (as more recipes are created and applied)
- Projected savings for remaining portfolio (based on current recipe coverage)

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented method for amortized AI-driven application transformation comprising:

(a) recording an interactive transformation session wherein a user transforms application content through one or more AI-mediated prompt-response cycles, each cycle consuming computational resources;

(b) reverse-engineering the recorded transformation session into a reusable translation recipe by analyzing the original content, the output produced, and the user's transformation requests, wherein the recipe is a compressed representation of the multi-step reasoning performed across the interactive session;

(c) encoding the translation recipe in a structured + natural language hybrid format comprising structured metadata fields and natural language transformation instructions;

(d) applying the stored translation recipe to a different application dataset of the same type in a single AI inference call, producing a transformation output equivalent in quality to the multi-step interactive session;

(e) thereby reducing computational resource consumption per transformation by amortizing the pattern capture cost across multiple applications.

Claim 2. A system for amortized AI transformation with reusable translation recipes comprising:

(a) a transformation studio configured to execute interactive AI-mediated transformations of application content with contextual awareness of the application's structure;

(b) a recipe generator configured to reverse-engineer interactive transformation sessions into reusable translation recipes that encode multi-step reasoning as compressed natural language instructions;

(c) a recipe store configured to persist translation recipes in a hybrid encoding comprising structured metadata and natural language instructions;

(d) a recipe executor configured to apply stored recipes to new application datasets in a single AI inference call, replacing multiple iterative sessions;

(e) an efficiency tracking layer configured to measure, record, and report computational resource consumption (tokens, latency, cost, energy, emissions) for each transformation call;

(f) a non-destructive storage mechanism configured to write transformation outputs to a designated subfolder while preserving original source files.

Claim 3. A method for reducing computational resource consumption in AI-mediated data transformation comprising:

(a) capturing a transformation pattern from an interactive AI session as a reusable recipe, wherein the recipe encodes the cumulative reasoning of multiple interactive turns as a compressed instruction set;

(b) applying the captured recipe to N additional similar datasets, each requiring a single AI inference call instead of multiple iterative calls;

(c) tracking the computational resources consumed by each recipe application and comparing against the estimated cost of the equivalent interactive approach;

(d) thereby reducing total AI inference token consumption by a factor approaching (interactivetokens / recipetokens) as N increases, achieving efficiency gains of at least 3x for portfolios of 10 or more similar datasets.

Claim 4. A computer-implemented method for capturing transformation intent from an interactive AI session and reapplying it across multiple datasets to reduce computational cost, comprising:

(a) monitoring an interactive session in which a user directs an AI system to transform a first dataset through a sequence of prompt-response cycles, wherein the sequence comprises at least two cycles and produces a validated transformation output;

(b) generating a transformation intent record by analyzing the first dataset, the validated transformation output, and the sequence of user directives, wherein the transformation intent record encodes the cumulative effect of the sequence as a self-contained instruction set consumable by an AI system in a single inference call;

(c) storing the transformation intent record in a persistent store;

(d) for each of one or more additional datasets structurally similar to the first dataset, applying the stored transformation intent record by providing it as input to an AI system along with the additional dataset, producing a transformation output in a single inference call without iterative refinement;

(e) wherein the total computational cost of transforming the first dataset plus the one or more additional datasets using the transformation intent record is less than the computational cost of transforming each dataset independently through interactive multi-cycle sessions.

Dependent Claims

Claim 5. The method of Claim 1 wherein the translation recipe instructions are type-specific but instance-agnostic, enabling application to any dataset from the same structural type without modification.

Claim 6. The method of Claim 1 wherein the recipe application includes loading contextual information about the target dataset's structure alongside the recipe instructions, enabling the AI system to adapt the transformation pattern to the specific dataset.

Claim 7. The system of Claim 2 wherein the AI inference interface is provider-agnostic, supporting multiple AI providers through a unified dispatch function.

Claim 8. The method of Claim 1 further comprising generating an expected output structure (folders and files) as part of the recipe, enabling automated validation of recipe application results against the expected structure.

Claim 9. The system of Claim 2 wherein the non-destructive storage mechanism enforces preservation of original source files through: (i) AI system prompt instructions, (ii) version control branch conventions, and (iii) filesystem plan validation.

Claim 10. The method of Claim 1 wherein the interactive transformation session produces a filesystem plan comprising: branch name, base path, folder list, and file list with content.

Claim 11. The system of Claim 2 further comprising seeded recipe templates for common transformation paths that are available to all tenants without requiring an initial interactive session.

Claim 12. The method of Claim 1 wherein recipe generation uses a specialized AI system prompt that instructs the model to produce type-specific but instance-agnostic instructions by analyzing the structural patterns of the transformation rather than the specific data values.

Claim 13. The method of Claim 3 wherein for a portfolio of N similar datasets, the total token consumption is approximately: Cinitial + (N-1) x Crecipe, where Cinitial is the cost of the first interactive session plus recipe generation, and Crecipe is the cost of a single recipe application, and wherein C_recipe is at most one-fourth of the interactive session cost for transformations requiring three or more interactive turns.

Claim 14. The system of Claim 2 further comprising an efficiency reporting module that computes and presents: per-recipe token savings (absolute and percentage), portfolio-level cumulative savings, financial cost reduction, and projected savings for planned transformations.

Claim 15. The method of Claim 1 further comprising a recipe editor allowing users to manually create or modify translation recipes with fields for name, description, source type, target platform, instructions, output structure, and version.

Claim 16. The method of Claim 1 wherein the transformation studio auto-reads repository content (file tree and configuration files) for AI context without requiring the user to manually specify which files to include.

Claim 17. The system of Claim 2 wherein recipe application respects configurable resource limits (maximum tokens per call) to control per-call resource consumption.

Claim 18. The method of Claim 3 wherein the computational resource reduction is at least 67% for portfolios of 10 or more similar applications, and at least 75% for portfolios of 100 or more similar applications.

Claim 19. The method of Claim 1 further comprising a chat mode that enables conversational Q&A about application content without producing filesystem modifications, using a separate system prompt optimized for analysis rather than transformation.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

Claim 21. The method of Claim 1 further comprising tracking token consumption for each transformation call and generating a token reduction report that includes: tokens consumed by the recipe-based approach, estimated tokens that would have been consumed by an equivalent interactive approach, absolute token savings, percentage reduction, and cumulative savings across all applications of the recipe.

Claim 22. The method of Claim 3 further comprising computing and reporting the environmental impact reduction achieved through recipe-based transformation, including: estimated energy savings in kilowatt-hours derived from token reduction, estimated greenhouse gas emission reduction in CO_2-equivalent kilograms derived from energy savings and grid carbon

intensity, and cumulative environmental impact across all recipe applications.

Claim 23. The system of Claim 2 further comprising a recipe marketplace configured to enable cross-tenant sharing of translation recipes, the marketplace comprising: a publication mechanism allowing a tenant to make a recipe discoverable to other tenants, a discovery mechanism allowing tenants to search for recipes by source type, target platform, or keyword, an application mechanism allowing a tenant to apply a marketplace recipe to their own dataset without sharing data between tenants, and a feedback mechanism for quality signals that improve marketplace discovery ranking.

Claim 24. The method of Claim 4 wherein the transformation intent record encodes the cumulative reasoning of the interactive session as a compressed instruction set that is between 5x and 50x smaller in token count than the aggregate token consumption of the original interactive session, while preserving the transformation's output quality as measured by structural equivalence of the output.

Claim 25. The method of Claim 4 wherein the transformation intent record is shareable across organizational boundaries through a marketplace mechanism, and wherein each application of the shared record to a new dataset contributes to a cumulative reduction in total computational resources consumed across all organizations, with the per-organization marginal cost of transformation approaching a fixed floor regardless of transformation complexity.

Claim 26. The method of Claim 1 wherein the compressed representation of multi-step reasoning preserves transformation fidelity by encoding only the cumulative validated transformation logic and excluding: iterative discovery steps, conversational scaffolding, failed approaches, corrections, and session-specific artifacts from the original interactive session.

Claim 27. The system of Claim 2 further comprising a portfolio-level optimization dashboard that displays: total applications transformed (interactive vs. recipe-based), cumulative token consumption and savings, cumulative financial cost and savings, cumulative energy and emissions impact, efficiency trend over time, and projected savings for remaining planned transformations based on current recipe coverage.

Claim 28. The method of Claim 3 wherein the efficiency gains are achieved through transformation pattern compression in which the recipe's natural language instructions encode the cumulative effect of K interactive turns (where K >= 2) in a single instruction block consumable in one AI inference call, and wherein the compression ratio (interactive tokens / recipe application tokens) increases with K.

---

6. Advantages Over Prior Art

1. Amortized transformation cost: Recipe development cost is incurred once; application cost is constant regardless of recipe complexity, achieving 3-4x efficiency at portfolio scale
2. Transformation pattern compression: Multi-step interactive reasoning (3-8 turns) is compressed into a single-call instruction set that preserves transformation quality while

eliminating iterative discovery cost

3. Measurable resource reduction: 67-76% reduction in AI token consumption for portfolios of 10-500+ applications, with precise tracking and reporting
4. Financial cost reduction: Proportional reduction in AI inference spending -- potentially saving thousands of dollars for large enterprise migrations
5. Environmental sustainability: 75-76% reduction in electricity consumption and greenhouse gas emissions for large migration portfolios, with per-recipe and portfolio-level environmental impact tracking
6. Single-call quality: Recipe instructions produce equivalent-quality output in one inference call because they encode pre-validated, comprehensive transformation logic
7. Latency reduction: 10-60x reduction in end-to-end transformation time per application (single call vs. multi-turn interactive session)
8. Higher quality output: Recipe instructions are validated (derived from successful transformations) vs. ad-hoc user prompts
9. Non-destructive iteration: Original source files are always preserved; transformations stored separately
10. Cross-application portability: Same recipe applied to structurally similar applications without modification
11. Cross-tenant sharing: Recipe marketplace multiplies amortization benefit across the entire user base
12. Full context awareness: Recipe application includes automatic repository context loading
13. Provider agnosticism: Recipes work across AI providers without modification
14. Portfolio-level optimization: Dashboard enables data-driven decisions about recipe investment and migration planning
15. Audit trail: Version-controlled branching ensures all transformations are versioned and reversible
16. Hybrid encoding: Structured metadata enables programmatic management while natural language instructions preserve expressive power

---

7. Drawings/Figures Description

Figure 1: System Architecture -- Block diagram showing Studio, Recipe Generator, Recipe Store, Recipe Executor, Efficiency Tracker, and Recipe Marketplace with data flow arrows and AI call interface.

Figure 2: Interactive Transform Flow -- Sequence diagram showing: user prompt -> context loading -> AI call -> filesystem plan -> branch creation -> file commit -> efficiency tracking.

Figure 3: Transformation Pattern Compression -- Diagram showing a 5-turn interactive session (78,000 tokens) being compressed into a single recipe instruction block (~1,500 tokens) that produces equivalent output in a single call (16,000 tokens). Annotated with what is preserved (transformation logic) and what is discarded (iterative scaffolding).

Figure 4: Recipe Generation Flow (Reverse Engineering) -- Diagram showing: original content + output files + chat context -> recipe generation AI call -> Translation record creation with hybrid encoding.

Figure 5: Single-Call Recipe Application Flow -- Diagram showing: stored recipe + new genome + repo context -> single AI call -> filesystem plan -> commit. Contrasted with multi-call interactive flow.

Figure 6: Token Consumption Comparison -- Bar chart comparing total tokens for interactive approach vs. recipe approach across portfolio sizes (N=1, 5, 10, 20, 50, 100, 500).

Figure 7: Efficiency Multiplier Curve -- Graph showing the efficiency multiplier (interactive cost / recipe cost) increasing from 1x at N=1 to 4.2x as N->inf, with the 3x and 4x thresholds annotated.

Figure 8: Non-Destructive Storage Structure -- Directory tree showing original genome files alongside transformations/ subfolder with recipe outputs.

Figure 9: Hybrid Instruction Encoding -- Diagram showing the Translation record with structured metadata fields (machine-parseable) and natural language instructions (LLM-consumable) annotated side-by-side.

Figure 10: Environmental Impact Graph -- Line chart showing cumulative CO2e emissions for interactive vs. recipe approaches as portfolio size increases, with breakeven point and savings area highlighted.

Figure 11: Recipe Marketplace Architecture -- Diagram showing publish, discover, apply, and feedback flows across multiple tenants with data isolation boundaries.

Figure 12: Portfolio-Level Optimization Dashboard -- Wireframe showing cumulative savings metrics, efficiency trends, recipe coverage, and projected savings for planned migrations.

Figure 13: Asymptotic Cost Curve -- Graph showing per-application token cost approaching the recipe application floor (16,000 tokens) as N increases, compared to the flat interactive cost line (67,000 tokens).

Figure 14: Cross-Tenant Amortization -- Diagram showing how a single recipe creation (Tenant A) yields compounding savings as Tenants B, C, and D apply the same recipe via the marketplace.

Figure 15: Compression Fidelity Model -- Comparison showing that the compressed recipe instruction set (500-2,000 tokens) produces output structurally equivalent to the uncompressed multi-turn session output (78,000 tokens).

---

8. Inventors

[To be completed by filing attorney]

9. Filing Notes

- Amortized AI cost is the strongest framing: Position this as a fundamental innovation in AI computational efficiency, not merely a migration tool. The principle -- capture transformation reasoning once, apply many times -- is broadly applicable beyond enterprise migration.
- Claim 4 is the broadest claim: It covers any system that captures transformation intent from an interactive AI session and reapplies it to reduce cost. This claim does not depend on genome-specific terminology and should survive attempts to design around the genome-specific claims.
- Transformation pattern compression is independently novel: The concept of compressing multi-step AI reasoning into a single-call instruction set (Section 4.6) is a distinct innovation from recipe reuse. Prior art search should focus on: AI reasoning chain compression, prompt optimization, few-shot learning, macro recording systems.
- Reverse-engineering interactive sessions is the strongest defensive claim: No prior art captures AI transformation reasoning from completed sessions. Prior art search should focus on: programming by demonstration, workflow recording, session replay.
- Environmental and efficiency claims strengthen the application: Patent offices increasingly favor inventions with demonstrable sustainability benefits. The detailed token, energy, and emissions analysis (Sections 4.9-4.10) should be validated with production data before filing.
- Marketplace claims create platform lock-in: The cross-tenant recipe marketplace (Claims 23, 25) establishes network effects that make the system more valuable as adoption grows.
- Hybrid instruction encoding is independently novel: The structured + natural language hybrid format (Section 4.4) is distinct from purely structured automation (macros, scripts) and purely natural language instruction (prompt engineering). Prior art should focus on: hybrid prompt formats, structured few-shot templates.
- Portfolio-level optimization is the enterprise sale: The dashboard and projected savings capabilities (Claim 27) address enterprise procurement requirements for ROI measurement.
- The token consumption analysis should be validated with actual production data before filing.
- Consider filing as a continuation of Patent 1 (Genome Extraction) and cross-referencing Patent 5 (Application Genome Modeling) to establish a comprehensive patent family.
- The non-destructive storage mechanism (transformations/ subfolder with branch conventions) may warrant independent claims in a continuation application.

Appendix A: Seeded Translation Recipes (Illustrative Examples)

Recipe 1: Enterprise Catalog -> Web Application

- Source type: service_catalog
- Target platform: web application
- Output: masterprompt.md, catalogsummary.json, config
- Typical efficiency: 3.5x at N=10

Recipe 2: Enterprise Catalog -> Repository Structure

- Source type: service_catalog
- Target platform: repository
- Output: README.md, schema.json, migration_plan.md
- Typical efficiency: 3.2x at N=10

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Document-Based Application Genome Extraction

Title

System and Method for Automated Application Genome Extraction from Unstructured Product Documentation Using Multi-Agent AI Analysis with Confidence Scoring

1. Field of Invention

This invention relates to natural language processing, enterprise application migration, and automated knowledge extraction. Specifically, it describes a system and method for extracting the complete structural genome of enterprise applications from unstructured product documentation (PDF, DOCX, plain text, markdown) using a coordinated multi-agent AI pipeline that parses documents, identifies application structure, and validates extraction quality through a weighted confidence scoring algorithm.

2. Background and Prior Art Gaps

Enterprise applications are frequently documented in product specification documents, capability assessments, migration readiness reports, and technical design documents. Current approaches to extracting actionable application intelligence from these documents include:

- Manual reading and interpretation: Subject matter experts read documentation and manually create data models, workflow diagrams, and integration maps -- a process that takes days to weeks per document and is limited by the reader's domain expertise.
- Keyword-based extraction: Simple text mining tools extract keywords but cannot identify structural relationships between entities, workflows, and data models.
- Named entity recognition (NER): NLP tools identify entity names but cannot determine application-specific semantics (e.g., that "HR Case" is a data object with specific fields, not just a noun phrase).

- Document summarization: AI summarization tools produce prose summaries but not machine-actionable structural representations.
- Manual template mapping: Migration teams create spreadsheet templates and manually map documentation to rows -- error-prone and not scalable.

Gaps in prior art:

1. No existing system automatically converts unstructured product documentation into a canonical, machine-actionable application genome comprising typed objects, fields, workflows, and relationships.
2. No existing system employs a multi-agent pipeline with specialized agents for document parsing, structure extraction, and validation to decompose documentation analysis into coordinated subtasks.
3. No existing system produces a weighted confidence score based on extraction completeness (presence and richness of objects, fields, workflows, relationships, application metadata) that quantifies how rebuildable the extracted genome is.
4. No existing system supports multi-format document parsing (PDF, DOCX, plain text, markdown) with automatic section detection using heading styles, markdown syntax, and typographic patterns.
5. No existing system provides an iterative refinement loop where users can supply additional context and re-extract with enriched instructions, progressively improving the genome without starting from scratch.
6. No existing system performs referential integrity validation on extracted genomes, verifying that fields reference valid objects and relationships reference valid endpoints.
7. No existing system streams real-time agent progress to the client during multi-step extraction, enabling users to observe which pipeline stage is executing and its intermediate results.

---

3. Summary of Invention

The present invention provides a multi-agent pipeline that takes product documentation files and extracts their complete structural genome through three specialized AI agents:

1. Document Parser Agent -- Parses uploaded documents (PDF via text extraction libraries, DOCX via document object model traversal, plain text/markdown via heading and section detection), producing structured sections with titles, content, and page references. Handles

multiple document formats through a pluggable parser architecture with automatic format detection.

2. Structure Extraction Agent -- Analyzes the parsed document text using a large language model (LLM) with a specialized system prompt to extract: typed objects (tables, entities, records), fields (with parent object binding, type, and required status), workflows (with triggers and ordered steps), and relationships (with source/target objects, type, and description). Limits input to token-safe boundaries and produces a canonical JSON genome structure.

3. Synthesis & Validation Agent -- Validates the extracted genome for referential integrity (fields reference valid objects, relationships reference valid endpoints), computes a weighted confidence score based on extraction completeness, and attaches validation notes. The confidence algorithm uses six weighted components: objects (0.25), fields (0.20), workflows (0.20), relationships (0.15), application name (0.10), summary (0.10), with richness bonuses for depth.

The pipeline supports Server-Sent Events (SSE) streaming for real-time progress, iterative refinement through re-extraction with additional user context, and commits extracted genomes to a version-controlled repository with taxonomized directory structure.

---

4. Detailed Description

4.1 System Architecture

The system comprises three cooperating AI agents orchestrated by a pipeline coordinator:

1. Document Parser Agent -- Format-aware text extraction with section detection
2. Structure Extraction Agent -- LLM-based genome extraction from text
3. Synthesis & Validation Agent -- Referential integrity checking and confidence scoring

Supporting services:

- Upload Service -- Accepts and stores document files with format validation
- SSE Streaming Service -- Delivers real-time agent progress to the client
- GitHub Commit Service -- Commits extracted genomes to version-controlled repository
- LLM Configuration Service -- Resolves tenant-specific LLM provider settings

4.2 Data Flow

```
User uploads document (PDF, DOCX, TXT, MD)
       |
[Upload Service]
  * Validate format and size
  * Store to disk with unique identifier
       |
[Document Parser Agent]
  * Detect format from file extension
  * Parse document using format-specific parser
  * Extract sections with titles and content
  * Compute page count and word count
       | parsed text + sections
[Structure Extraction Agent]
  * Build LLM prompt with document text (token-limited)
  * Include vendor, product area, module context
  * LLM extracts: objects, fields, workflows, relationships
  * Parse JSON response, ensure required fields
       | raw genome
[Synthesis & Validation Agent]
  * Validate field-to-object references
  * Validate relationship endpoints
  * Compute weighted confidence score
  * Attach validation notes
       | validated genome + confidence
[GitHub Commit Service]
  * Build file tree: genome.yaml, summary.md, structure/*.json
  * Commit to repository with taxonomy path
       |
Version-controlled repository
```

4.3 Multi-Format Document Parsing

The Document Parser Agent supports four document formats through a pluggable parser architecture:

PDF Parser:

- Uses text extraction libraries to extract per-page text content
- Each page becomes a section with page number reference
- Handles multi-column layouts and embedded tables

DOCX Parser:

- Traverses the document object model (paragraphs, styles)
- Detects headings by style name (Heading 1, Heading 2, etc.) as section boundaries
- Groups body paragraphs under their preceding heading

- Estimates page count from character count

Plain Text / Markdown Parser:

- Detects markdown headings (lines starting with #) as section boundaries
- Detects ALL-CAPS lines (3-100 characters) as section boundaries
- Groups content under detected headings
- Estimates page count from character count

Format Detection:

- File extension determines parser selection (.pdf, .docx, .txt, .md)
- Unknown extensions fall back to plain text parser
- Each parser produces identical output structure: fulltext, sections[], pagecount, word_count

4.4 Structure Extraction via LLM

The Structure Extraction Agent uses a specialized system prompt that instructs the LLM to:

1. Analyze the document text as product documentation describing an enterprise application
2. Extract objects with name, label, description, and type
3. Extract fields with name, label, type, parent object reference, required status, and description
4. Extract workflows with name, description, trigger, and ordered steps
5. Extract relationships with source object, target object, type, and description
6. Return a canonical JSON structure with mandatory sections

The agent limits input document text to approximately 80,000 characters to stay within LLM token boundaries while maximizing coverage.

JSON Response Parsing:

The agent implements robust JSON extraction that:

- Strips markdown code fences from LLM output
- Attempts direct JSON parsing
- Falls back to brace-matching extraction (finding outermost { } boundaries)
- Ensures all required genome sections exist (defaulting to empty arrays)

4.5 Weighted Confidence Scoring Algorithm

The Synthesis & Validation Agent computes a confidence score using a weighted multi-factor formula:

```
Score = sum of:
  0.25  if objects exist (at least 1)
  0.20  if fields exist (at least 1)
  0.20  if workflows exist (at least 1)
  0.15  if relationships exist (at least 1)
  0.10  if application_name is non-empty
  0.10  if summary is non-empty

Richness bonuses:
  +0.05  if objects >= 5 (capped at 1.0)
  +0.05  if fields >= 10 (capped at 1.0)
```

This produces a score from 0.0 (empty extraction) to 1.0 (complete extraction with rich content), enabling automated quality assessment and user guidance on whether additional documentation should be provided.

4.6 Referential Integrity Validation

The validation agent performs three integrity checks:

1. Object name validation: Every object must have a non-empty name
2. Field-to-object validation: Every field's object reference must match an existing object name (case-insensitive)
3. Relationship endpoint validation: Every relationship's fromobject and toobject must match existing object names

Violations are recorded as validation notes (not errors), allowing the genome to be used while informing the user of inconsistencies.

4.7 Iterative Refinement

The system supports re-extraction with enriched context:

1. User reviews the initial extraction and identifies gaps
2. User provides additional context (e.g., "Focus on the approval workflows described in section 4")
3. System appends the refinement instructions to the original user notes
4. The full pipeline re-runs with the enriched prompt, producing an improved genome

5. The original upload is reused (no re-upload needed) since the document ID is preserved

This enables progressive improvement without starting from scratch or consuming additional upload bandwidth.

4.8 Real-Time Progress Streaming

The pipeline streams agent progress to the client using Server-Sent Events (SSE):

```
data: {"agent": "document_parser", "status": "running"}
data: {"agent": "document_parser", "status": "done", "pages": 12, "words": 3450, "sections": 8}
data: {"agent": "structure_extraction", "status": "running"}
data: {"agent": "structure_extraction", "status": "done", "objects": 5, "fields": 23, "workflows": 3}
data: {"agent": "synthesis", "status": "done", "confidence": 0.85, "validation_issues": 2}
data: {"status": "completed", "genome": {...}, "page_count": 12, "word_count": 3450}
```

Each agent reports start, completion (with metrics), or error status. The client uses these events to update a real-time progress display showing which agent is active and what it has found.

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented method for automated application genome extraction from product documentation comprising:

(a) receiving one or more unstructured document files describing an enterprise application;

(b) parsing the document files using format-specific parsers to extract structured text sections with titles, content, and positional metadata;

(c) submitting the parsed text to a large language model with a specialized extraction prompt to identify application objects, fields, workflows, and relationships;

(d) validating the extracted genome for referential integrity by verifying that field-to-object and relationship endpoint references resolve to valid entities;

(e) computing a weighted confidence score based on extraction completeness using a multi-factor formula with predetermined component weights;

(f) storing the validated genome with confidence score in a canonical format.

Claim 2. A system for extracting application genomes from unstructured documentation comprising:

(a) a document parser agent configured to parse multiple document formats (PDF, DOCX, plain text, markdown) into structured sections using format-specific parsing logic with automatic format detection;

(b) a structure extraction agent configured to analyze parsed text using a large language model to extract typed objects, fields with parent object binding, workflows with ordered steps, and relationships with typed endpoints;

(c) a synthesis and validation agent configured to verify referential integrity and compute a weighted confidence score;

(d) a pipeline orchestrator configured to execute agents sequentially with real-time progress streaming via Server-Sent Events.

Claim 3. A method for computing extraction confidence in an application genome comprising:

(a) evaluating the presence of extracted objects, fields, workflows, relationships, application name, and summary;

(b) assigning predetermined weights to each component (objects: 0.25, fields: 0.20, workflows: 0.20, relationships: 0.15, name: 0.10, summary: 0.10);

(c) summing the weights of present components;

(d) applying richness bonuses when component counts exceed predetermined thresholds;

(e) capping the total score at a maximum value of 1.0.

Dependent Claims

Claim 4. The method of Claim 1 wherein the format-specific parsers comprise: a PDF parser using text extraction with per-page sections, a DOCX parser using document object model traversal with heading-based section detection, and a text/markdown parser using heading syntax and typographic pattern detection.

Claim 5. The method of Claim 1 further comprising an iterative refinement loop wherein the user provides additional context that is appended to the extraction instructions, and the pipeline re-executes using the same uploaded document with enriched prompts.

Claim 6. The method of Claim 1 wherein the large language model input is limited to a predetermined character boundary to stay within token limits while maximizing document coverage.

Claim 7. The system of Claim 2 wherein the structure extraction agent implements robust JSON parsing comprising: markdown code fence stripping, direct JSON parsing, and fallback brace-matching extraction.

Claim 8. The method of Claim 1 further comprising committing the extracted genome to a version-controlled repository using a hierarchical taxonomy structure comprising vendor, product area, and module path components.

Claim 9. The method of Claim 1 wherein the genome comprises separate structure files for objects, fields, workflows, and relationships stored as individual JSON files alongside a canonical genome YAML file.

Claim 10. The system of Claim 2 wherein the pipeline orchestrator streams agent progress events to the client in real-time,

each event comprising agent name, status (running, done, error), and stage-specific metrics.

Claim 11. The method of Claim 1 wherein the referential integrity validation records violations as non-blocking validation notes rather than errors, permitting use of the genome while informing the user of inconsistencies.

Claim 12. The method of Claim 3 wherein the richness bonuses comprise: +0.05 when object count exceeds 5, and +0.05 when field count exceeds 10.

Claim 13. The method of Claim 1 wherein each extracted field includes: name, label, type, parent object reference, required status, and description.

Claim 14. The method of Claim 1 wherein each extracted workflow includes: name, description, trigger condition, and an ordered list of steps.

Claim 15. The system of Claim 2 further comprising an upload service that validates document format against a whitelist of supported extensions and enforces maximum file size limits.

Claim 16. The method of Claim 1 wherein the DOCX parser identifies section boundaries by detecting paragraph styles matching heading patterns (Heading 1, Heading 2, etc.) in the document object model.

Claim 17. The method of Claim 1 wherein the plain text parser identifies section boundaries using two heuristics: lines beginning with hash characters (markdown headings) and lines consisting entirely of uppercase characters within a length range.

Claim 18. The system of Claim 2 wherein the large language model configuration is resolved per-tenant, enabling different tenants to use different LLM providers and models.

Claim 19. The method of Claim 1 further comprising generating a human-readable summary markdown file alongside the canonical genome, containing the application name and LLM-generated summary text.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

---

6. Advantages Over Prior Art

1. Automation: Converts hours of manual document analysis into minutes of automated extraction
2. Multi-format support: Single pipeline handles PDF, DOCX, plain text, and markdown without user intervention

3. Machine-actionable output: Produces structured genome (objects, fields, workflows, relationships) from unstructured prose, enabling downstream automation
4. Quality quantification: Weighted confidence score provides objective measure of extraction completeness
5. Referential integrity: Automated validation catches inconsistencies that manual extraction would miss
6. Iterative improvement: Refinement loop enables progressive quality improvement without re-uploading documents
7. Real-time visibility: SSE streaming provides transparency into multi-agent processing stages
8. Vendor agnosticism: Extracted genomes use a canonical format regardless of the source platform described in the documentation
9. Scalability: Automated pipeline enables extraction from large document portfolios that would be infeasible to process manually
10. Auditability: Version-controlled storage with taxonomy structure provides complete lineage traceability

---

7. Drawings/Figures Description

Figure 1: System Architecture Overview -- Block diagram showing the three AI agents (Document Parser, Structure Extraction, Synthesis & Validation) with the pipeline orchestrator, upload service, and GitHub commit service.

Figure 2: Multi-Agent Pipeline Flow -- Sequence diagram showing document upload, agent execution order, SSE progress events, and final genome output.

Figure 3: Multi-Format Parser Architecture -- Decision tree showing format detection from file extension and dispatch to format-specific parsers, all producing identical section output structure.

Figure 4: Structure Extraction LLM Prompt -- Annotated example showing the system prompt, user prompt with document content, and expected JSON response structure.

Figure 5: Confidence Scoring Formula -- Weighted component diagram showing the six factors, their weights, and the richness bonus thresholds.

Figure 6: Referential Integrity Validation -- Flow diagram showing field-to-object validation and relationship endpoint validation with example pass/fail cases.

Figure 7: Iterative Refinement Loop -- State diagram showing initial extraction, user review, refinement input, re-extraction, and convergence to desired quality.

Figure 8: SSE Progress Event Timeline -- Timeline diagram showing agent events (running, done, error) with metrics payloads as the pipeline executes.

Figure 9: Taxonomized Repository Structure -- Directory tree showing genome.yaml, summary.md, and structure/ directory with per-type JSON files.

Figure 10: Genome Data Model -- Entity-relationship diagram showing the canonical genome structure with objects owning fields, workflows with steps, and typed relationships.

---

8. Inventors

[To be completed by filing attorney]

9. Filing Notes

- The weighted confidence scoring algorithm is a key differentiator and should be emphasized in prosecution
- The multi-format document parsing with automatic section detection is independently novel
- Consider filing as a continuation of Patent 1 (API-based Genome Extraction) to establish a patent family covering multiple extraction modalities
- The iterative refinement loop (re-extraction with enriched context) may warrant independent claims
- Prior art search should focus on: document-to-schema extraction, NLP-based data modeling, automated migration tools
- The combination of document parsing + LLM extraction + validation with confidence scoring appears novel in the enterprise migration space

---

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Application Genome Modeling, Extraction, and Portability

Title

System and Method for Modeling, Extracting, and Representing Application Genomes as Canonical Intermediate Representations of Application Intent, Structure, and Behavior from Enterprise Software Systems

1. Field of Invention

This invention relates to application genome modeling, customer data portability, intellectual property extraction, enterprise application migration, cross-platform interoperability, and vendor lock-in remediation. Specifically, it describes a system and method for:

1. Defining and producing an application genome -- a canonical representation of application intent, structure, and behavior that captures the totality of a software application's business logic, behavioral patterns, process intent, UI semantics, system interactions, data models, and operational rules in a platform-neutral, implementation-independent form.

2. Extracting application genomes from enterprise software systems -- including but not limited to configuration-driven platforms, low-code/no-code platforms, SaaS applications, and custom-built enterprise systems -- by separating customer-authored business logic independent of platform-specific implementation from the proprietary implementation framework of the originating system.

3. Maintaining application genomes as persistent, versioned, reusable systems of record that serve as the authoritative representation of an application's business intent across its lifecycle, independent of any particular deployment platform or technology stack.

The system operates on customer-provided application artifacts and at no point accesses, copies, decompiles, or reverse engineers proprietary platform execution code, platform runtime architecture, or internal vendor APIs and system logic.

2. Background and Prior Art Gaps

Enterprise customers invest significant intellectual effort in designing business processes, approval workflows, data models, catalog structures, and operational logic. These designs represent the customer's intellectual property -- the customer conceived the business requirements, designed the processes, and specified the rules. When implemented on enterprise software systems -- whether configuration-driven platforms, low-code/no-code environments, SaaS applications, or custom-built enterprise systems -- the customer's IP becomes embedded in system-specific artifacts (configuration exports, metadata packages, API definitions, UI specifications, process definitions, video demonstrations, and documentation), commingled with -- but conceptually distinct from -- the system's proprietary implementation.

Current challenges:

- Vendor lock-in: Customer-authored business logic is stored in proprietary formats that can only be interpreted by the originating system, creating dependency on a single vendor or technology stack.
- IP entanglement: The customer's intellectual property (business process designs, workflow logic, data models, UI semantics, system interaction patterns) becomes entangled with the system's proprietary implementation framework, making it difficult to determine what belongs to the customer versus the platform.
- Export limitations: System export tools produce system-specific artifacts that cannot be used on other platforms without extensive manual translation.
- Loss of business context: When customers attempt to migrate, the business intent behind configurations is often lost because it exists only in the system-specific implementation, not in a portable, canonical form.
- No canonical application model: No standard representation exists for capturing the complete intent, structure, and behavior of an application independent of its implementation platform.
- Fragmented application knowledge: Application knowledge is distributed across multiple artifact types (configurations, APIs, documentation, video walkthroughs, process diagrams) with no unified model to integrate them.
- Legal ambiguity: Without a clean separation between customer IP and platform implementation, questions of ownership and portability remain unresolved.

Gaps in prior art:

1. No existing system defines an application genome as a canonical representation of application intent, structure, and behavior encompassing behavioral patterns, process intent, UI semantics, and system interactions in a platform-neutral form.
2. No existing system extracts customer-authored business logic independent of

platform-specific implementation from enterprise software systems and produces a canonical intermediate representation suitable for cross-platform portability.

3. No existing system produces a platform-neutral portable architecture (domain model, application services, API surface, UI model, automation rules, behavioral patterns, system interaction maps) from system-specific artifacts, excluding proprietary execution code and internal APIs.

4. No existing system employs a multi-agent AI pipeline with specialized agents for multi-modal ingestion, extraction, deduplication, deep behavioral analysis, platform transformation, and validation to decompose application genome extraction into coordinated subtasks.

5. No existing system supports multi-modal extraction of application genomes from heterogeneous artifact types including configuration exports, API specifications, video demonstrations, business documents, and process diagrams.

6. No existing system merges artifacts from multiple sources and modalities into a unified application genome, handling deduplication of entities, normalization of naming, and conflict resolution across artifact types.

7. No existing system extracts reusable behavioral patterns (inventory management, approval flows, request lifecycles, status transitions, escalation chains) from application artifacts, abstracting them from system-specific terminology.

8. No existing system provides a completeness and rebuildability assessment that quantifies whether the extracted application genome is sufficient to reconstruct the application on another platform.

9. No existing system maintains the application genome as a persistent, versioned system of record that evolves with the application across its lifecycle and across platform migrations.

10. No existing system represents the application genome in both flat (tabular/document) and graph (entity-relationship/dependency) schemas simultaneously, enabling different consumption patterns for different downstream uses.

11. No existing system employs deterministic and non-deterministic processing stages in a coordinated hybrid pipeline for application genome extraction, combining rule-based parsing with AI-driven semantic analysis.

12. No existing system enforces legal boundary mechanisms that systematically remove system-specific identifiers, schemas, and execution constructs to prevent reconstruction of proprietary runtime behavior or internal APIs.

---

3. Definition of Application Genome

An application genome as defined by this invention is a canonical representation of application intent, structure, and behavior. It is the complete, platform-neutral model of what an application does, why it does it, and how users and systems interact with it -- expressed independently of how any particular platform implements it.

The application genome comprises the following facets:

3.1 Structural Facet

The structural facet captures the static architecture of the application:

- Domain Model: The entities, data objects, and their relationships that form the application's information architecture (e.g., "HR Case has many Tasks; each Task is assigned to an Agent Group")
- Data Schema: Field definitions, types, constraints, and validation rules expressed as business-level specifications independent of any database implementation
- Taxonomy: Classification hierarchies, categorization systems, and organizational structures authored by the customer

3.2 Behavioral Facet

The behavioral facet captures the dynamic logic of the application:

- Business Rules: Declarative trigger-condition-action specifications that encode the customer's operational logic (e.g., "when a case is created in the West region, assign to the Western HR team")
- Behavioral Patterns: Reusable, named abstractions of common business behaviors (e.g., "approval-chain," "inventory-decrement," "SLA-escalation") expressed in domain language
- State Machines: Status transitions and lifecycle progressions that govern how business entities evolve over time
- Event Mappings: Trigger-effect relationships that capture the customer's event-driven business logic

3.3 Process Intent Facet

The process intent facet captures the orchestrated sequences of business operations:

- Workflows: Ordered sequences of business steps with actors, actions, decision points, and outcomes
- Process Flows: End-to-end business processes that span multiple entities and actors
- Approval Chains: Multi-level authorization and delegation patterns
- Escalation Paths: Exception-handling and escalation sequences triggered by business conditions

3.4 UI Semantics Facet

The UI semantics facet captures the user experience intent without encoding visual implementation:

- Screen Specifications: Abstract descriptions of user-facing screens with purpose, audience, and functional components
- Form Definitions: Input collection specifications with field types, validation, conditional visibility, and submission behavior
- Navigation Structure: Application navigation topology and access patterns
- Component Semantics: The business purpose and behavior of UI elements independent of rendering technology (e.g., "employee lookup field" rather than "typeahead widget with REST datasource")

3.5 System Interaction Facet

The system interaction facet captures how the application connects to and communicates with external systems:

- Integration Points: External system connections with direction, purpose, and data exchange patterns
- API Surface: The customer-designed operations and queries that the application exposes or consumes, expressed as technology-stack-independent operation specifications
- Data Exchange Patterns: Import, export, synchronization, and real-time event patterns between the application and external systems
- Authentication and Authorization Intent: Access control patterns expressed as business-level role and permission specifications

3.6 Genome Representation Formats

The application genome supports dual-schema representation:

- Flat Schema (Document/Tabular): The genome is representable as a structured document (YAML, JSON) or tabular format suitable for sequential reading, code generation, and document-oriented consumption. Each genome facet is expressed as a hierarchical structure of named components with typed properties.

- Graph Schema (Entity-Relationship/Dependency): The genome is simultaneously representable as a directed graph where nodes represent genome components (entities, workflows, rules, screens, integrations) and edges represent relationships (references, triggers, dependencies, data flows). The graph schema enables dependency analysis, impact assessment, and structural comparison across genome versions.

Both representations are derived from a single canonical genome model and are guaranteed to be consistent with each other.

---

4. Summary of Invention

The present invention provides a system and method for modeling, extracting, and maintaining application genomes -- canonical representations of application intent, structure, and behavior -- from enterprise software systems. The system asserts and operationalizes the principle that customers own the business logic they designed -- the platform is merely the storage and execution vehicle.

The system accepts application artifacts from any enterprise software system -- including but not limited to configuration-driven platforms, low-code/no-code environments, SaaS applications, and custom-built enterprise systems -- through multiple ingestion modalities (configuration exports, API specifications, video demonstrations, business documents, and process diagrams) and produces a unified, portable application genome.

The extraction process operates exclusively at the business-intent level, excluding proprietary platform execution code, excluding platform runtime architecture, and excluding internal vendor APIs and system logic. The output genome represents only the customer's authored business intent in a platform-neutral, canonical form.

The pipeline employs a hybrid processing architecture that combines deterministic stages (rule-based parsing, schema validation, structural analysis) with non-deterministic stages (AI-driven semantic extraction, behavioral pattern recognition, intent inference) to maximize both reliability and depth of extraction.

The pipeline comprises specialized AI agents:

1. Artifact Ingestion Agent -- Accepts application artifacts from heterogeneous sources and modalities. For configuration exports: parses structured formats (XML, JSON, YAML, CSV), extracts records, counts components, and identifies the scope of customer-authored content. For API specifications: parses OpenAPI, GraphQL, or equivalent interface

definitions to extract customer-designed operations. For video artifacts: analyzes screen recordings and demonstrations to extract UI semantics, workflow sequences, and interaction patterns. For document artifacts: parses business requirements documents, process diagrams, and operational procedures to extract process intent and business rules. Deterministic parsing stages handle format recognition and structural decomposition; non-deterministic AI stages handle semantic classification and intent inference.

2. Genome Extraction Agent -- Analyzes ingested artifacts using large language models with specialized prompting to extract customer-authored business logic independent of platform-specific implementation. Produces a raw genome encompassing all five facets: structural (entities, data models), behavioral (rules, patterns, events), process intent (workflows, approval chains), UI semantics (screens, forms, navigation), and system interactions (integrations, API surface). All extraction operates at the business-intent level; proprietary execution code, system-specific API signatures, and internal identifiers are excluded from the output.

3. Genome Merger Agent -- When artifacts from multiple sources, exports, or modalities are provided, deduplicates entities, merges definitions that span multiple artifacts, resolves naming conflicts by preferring more complete definitions, and normalizes naming conventions to produce a single unified genome. Handles cross-modal reconciliation (e.g., reconciling a workflow extracted from a configuration export with steps observed in a video demonstration).

4. Deep Analysis Agent -- Performs second-pass analysis to extract deeper behavioral patterns from the application's artifacts:

- Logic Patterns: Reusable behavioral abstractions expressed in domain language
- Business Processes: Step-by-step process flows with actors, actions, and system behaviors
- Events: Trigger-effect mappings that capture event-driven business logic
- System Interaction Patterns: Integration archetypes and data exchange patterns

All patterns are expressed in platform-neutral, business-readable language, excluding proprietary execution code.

5. Platform Transformer Agent -- Converts the extracted genome into the canonical application genome format, with a platform abstraction layer that ensures all system-specific constructs have been replaced with generic, implementation-independent representations across all five genome facets. Produces both flat (document) and graph (entity-relationship) representations.

6. Genome Validator Agent -- Assesses the completeness and rebuildability of the extracted

application genome:

- Completeness Score (0-100): Quantifies extraction coverage across all genome facets
- Missing Components: Identifies gaps where application intent may not have been captured
- Risks: Flags areas where rebuilt applications may behave differently
- Recommendations: Suggests additional artifacts or documentation to improve coverage
- Boundary Verification: Confirms that no proprietary execution constructs, internal API references, or runtime architecture details remain in the output genome
- Cross-Modal Consistency: Verifies that genome components extracted from different modalities are consistent with each other

---

5. Detailed Description

5.1 Philosophical Foundation: Customer IP Ownership

The system is built on the principle that when a customer designs a business process and implements it on an enterprise software system, the intellectual property of that design belongs to the customer. Specifically:

- The customer conceived the business requirement (e.g., "HR case submissions should be routed by region and require manager approval")
- The customer designed the workflow (submission -> routing -> approval -> fulfillment)
- The customer defined the data model (HR Case, Employee, Task, Agent Group)
- The customer specified the business rules (auto-route by region, escalate after SLA breach)
- The customer designed the user experience (submission forms, dashboard views, navigation paths)
- The customer defined the system interactions (integrations with external HR systems, notification channels)
- The platform merely provided the storage format and execution engine

The system extracts the customer's design intent -- not the platform's proprietary code, framework, or execution engine. System-specific terminology (function names, table prefixes, schema elements, widget identifiers) is systematically stripped and replaced with business-readable, platform-neutral equivalents. The output contains no proprietary execution code, no runtime architecture details, and no internal API references.

5.2 Legal Boundary Enforcement Mechanisms

The system enforces a strict legal boundary between customer-authored business intent and proprietary platform implementation through four coordinated mechanisms:

5.2.1 System-Specific Identifier Removal

At each stage of the extraction pipeline, system-specific identifiers are identified and removed or replaced. This includes:

- Internal namespace prefixes (e.g., proprietary schema prefixes, system-generated namespace qualifiers) are stripped and replaced with customer-defined business names or semantically equivalent generic identifiers.
- Platform object identifiers (e.g., internal GUIDs, system-generated record keys, proprietary scope identifiers) are discarded entirely, as they are artifacts of the system's storage implementation and carry no customer-authored business meaning.
- System-specific field names (e.g., internal column names that encode execution semantics rather than business meaning) are mapped to descriptive, platform-neutral field names derived from the customer's labels or contextual analysis.
- Proprietary schema elements and namespaces specific to the system's export format are consumed during parsing but not propagated to the output genome.

5.2.2 Platform-Neutral Representation Conversion

All extracted artifacts are converted into platform-neutral representations that express business intent without encoding any system-specific execution semantics:

- Workflows are represented as ordered sequences of business steps (actor, action, outcome) rather than system-specific execution graph definitions, state machine encodings, or proprietary orchestration formats.
- Business rules are expressed as declarative trigger-condition-action triples in natural language rather than as system-specific scripting language, proprietary function calls, or platform API invocations.
- Data models are expressed as entity-relationship structures with business-purpose annotations rather than system-specific table schemas, proprietary field types, or platform-internal relationship encodings.
- UI specifications are expressed as abstract screen-component hierarchies (form, field, button) with business-purpose annotations rather than system-specific widget definitions, proprietary rendering directives, or platform UI framework constructs.

5.2.3 Runtime Reconstruction Prevention

The system is architecturally designed to prevent the output genome from being used to reconstruct any system's proprietary runtime, execution engine, or internal API surface:

- No execution code is extracted: The system summarizes business logic at the intent level rather than extracting or reproducing any executable code, scripts, or compiled artifacts from the platform.
- No API signatures are preserved: System-internal API endpoints, method signatures, parameter schemas, and authentication mechanisms are excluded from the output.
- No runtime architecture is encoded: The output genome contains no information about any system's execution engine, request processing pipeline, caching layers, database query patterns, or internal service architecture.
- No platform schema is reproducible: The output genome's data model represents the customer's business entities at a conceptual level; it cannot be used to reconstruct any system's internal database schema, indexing strategy, or storage implementation.

5.2.4 Business-Intent Abstraction Layer

The system employs a multi-level abstraction approach to represent all extracted customer IP as business intent:

- Level 1 -- Raw Extraction: Customer-authored content is identified and separated from platform system records. System-specific formatting is removed.
- Level 2 -- Semantic Normalization: Extracted components are normalized into a canonical genome vocabulary that is system-independent. Business names replace internal identifiers.
- Level 3 -- Behavioral Abstraction: Business logic is abstracted into reusable patterns (approval flows, lifecycle management, event-driven rules) expressed in domain language, not platform language.
- Level 4 -- Portable Architecture: The genome is transformed into a technology-stack-independent architecture specification (domain model, services, API, UI, automation) that can be implemented on any platform without reference to the originating system.

At each abstraction level, the system moves further from the system's implementation details and closer to the customer's pure business intent. The final output at Level 4 bears no structural, syntactic, or semantic resemblance to any system's proprietary architecture.

5.3 System Architecture

```
Application Artifacts (multi-modal)
  * Configuration exports (XML, JSON, YAML, CSV)
  * API specifications (OpenAPI, GraphQL, WSDL)
  * Video demonstrations (screen recordings, walkthroughs)
  * Business documents (requirements, process diagrams, SOPs)
       |
[Artifact Ingestion Agent]
  * Deterministic: format detection, structural parsing, record counting
  * Non-deterministic (AI): content classification, intent detection
  * Identify customer-authored components
  * Exclude platform infrastructure records
       | parsed artifact data (customer content only, multi-modal)
[Genome Extraction Agent (LLM)]
  * Extract customer-authored business logic independent of platform implementation
  * Map system artifacts to canonical genome facets
  * Strip system-specific terminology, identifiers, and schema elements
  * Exclude proprietary execution code
       | raw application genome (platform-neutral, five facets)
[Genome Merger Agent (LLM)] -- for multi-source/multi-modal
  * Deduplicate across sources and modalities
  * Merge partial definitions
  * Normalize naming conventions
  * Reconcile cross-modal observations
       | unified application genome
[Deep Analysis Agent (LLM)]
  * Extract reusable behavioral patterns
  * Map business processes with actors and steps
  * Identify event-driven business logic and system interaction patterns
  * Express all patterns as business intent
       | enriched genome with patterns + processes
[Platform Transformer Agent (LLM)]
  * Convert to canonical genome format via abstraction layer
  * Generate flat schema (document) and graph schema (entity-relationship)
  * Produce rebuild-ready specification
  * Verify no system-specific constructs remain
       | canonical application genome (dual-schema, system-independent)
[Genome Validator Agent (LLM)]
  * Assess completeness (0-100 score per facet)
  * Identify missing components and risks
  * Generate recommendations
  * Verify legal boundary compliance
  * Check cross-modal consistency
       | validated genome with quality report + boundary audit
[Genome Repository Service]
  * Store versioned genome with full history
  * Maintain flat + graph representations
  * Taxonomized directory structure
  * Version-controlled with audit trail
  * Persistent system of record
```

5.4 Multi-Modal Artifact Ingestion

The Artifact Ingestion Agent accepts application artifacts from heterogeneous sources and modalities through a hybrid processing pipeline:

5.4.1 Configuration Export Ingestion (Deterministic + Non-Deterministic)

Deterministic stage: Reads structured export files (XML, JSON, YAML, CSV), identifies root elements and record containers, counts records, and extracts structural metadata. Handles variations in schema across different systems and export formats.

Non-deterministic stage: Uses AI-driven analysis to classify records as customer-authored versus system-internal, identify business-relevant components within ambiguous structures, and infer component purposes from naming patterns and contextual relationships.

5.4.2 API Specification Ingestion (Deterministic + Non-Deterministic)

Deterministic stage: Parses OpenAPI, GraphQL, WSDL, or equivalent interface specifications. Extracts endpoints, operations, parameter schemas, and response definitions.

Non-deterministic stage: Uses AI-driven analysis to distinguish customer-designed API operations from platform-infrastructure endpoints, infer business intent from operation names and descriptions, and extract data exchange patterns.

5.4.3 Video Artifact Ingestion (Non-Deterministic)

Non-deterministic stage: Analyzes screen recordings and video demonstrations using multimodal AI to extract:

- UI Semantics: Screen layouts, form structures, navigation paths, and component behaviors observed during demonstrations
- Workflow Sequences: Step-by-step process flows as demonstrated by users in the recordings
- Interaction Patterns: User actions, system responses, and decision points captured in the demonstration
- Business Context: Explanatory narration or annotations that provide business rationale for observed behaviors

5.4.4 Document Artifact Ingestion (Non-Deterministic)

Non-deterministic stage: Analyzes business requirements documents, process diagrams, standard operating procedures, and operational documentation using AI to extract:

- Process Intent: Business processes, approval chains, and escalation paths described in documentation
- Business Rules: Operational rules, policies, and decision criteria specified in documents

- Data Requirements: Entity definitions, field specifications, and relationship descriptions from requirements documents
- Integration Requirements: External system connections and data exchange needs described in documentation

5.5 Customer IP Extraction via LLM

The Genome Extraction Agent uses specialized prompting that instructs the LLM to extract customer-authored business logic independent of platform-specific implementation:

1. Identify customer-authored components within the ingested artifacts, distinguishing customer-designed business logic from system infrastructure, platform boilerplate, and vendor-internal constructs.

2. Extract and normalize into the canonical genome facets, excluding proprietary execution code, excluding runtime architecture, and excluding internal APIs and system logic:
   - Structural Facet: Domain entities, data schema, taxonomy
   - Behavioral Facet: Business rules, behavioral patterns, state machines, event mappings
   - Process Intent Facet: Workflows, process flows, approval chains, escalation paths
   - UI Semantics Facet: Screen specifications, form definitions, navigation structure, component semantics
   - System Interaction Facet: Integration points, API surface, data exchange patterns

3. Strip system-specific terminology and proprietary constructs: Convert function names, table prefixes, internal identifiers, schema references, widget types, and API method names to business-readable, platform-neutral equivalents.

5.6 Multi-Source Merging and Cross-Modal Reconciliation

When artifacts from multiple sources or modalities are provided, the Genome Merger Agent:

1. Deduplicates: Identifies the same entity, workflow, or rule appearing in multiple artifacts by semantic matching
2. Merges partial definitions: Combines genome components that are partially defined across multiple artifacts
3. Resolves conflicts: When the same component appears with different definitions, prefers the more complete version or merges complementary attributes

4. Normalizes naming: Ensures consistent naming conventions across the unified genome
5. Reconciles cross-modal observations: Aligns genome components extracted from different modalities (e.g., a workflow extracted from configuration export with steps observed in a video demonstration, supplemented by business rules described in documentation)

The merger uses LLM analysis to understand semantic equivalence beyond exact string matching and to reconcile observations from fundamentally different artifact modalities.

5.7 Behavioral Pattern Extraction

The Deep Analysis Agent performs a second-pass extraction focused on reusable behavioral patterns:

Logic Patterns -- Abstracted from system-specific implementations:

```
- name: decrement_inventory_on_checkout
  trigger: checkout_created
  action: reduce inventory count
  reusable_pattern: inventory_decrement
```

Instead of system-specific function references, the system extracts the customer's business intent as a named, reusable pattern. System-specific function signatures are discarded; only the customer-authored business meaning is retained.

Business Processes -- Step-by-step with actors:

```
- name: access_provisioning
  steps:
    - step: submit_request
      actor: employee
      action: submits access request
      system_behavior: create case record
    - step: manager_approval
      actor: manager
      action: reviews and approves
      system_behavior: update status, notify fulfillment
```

Events -- Trigger-effect mappings:

```
- name: sla_breach_escalation
  trigger: case response time exceeds SLA threshold
  downstream_effects:
    - escalate to supervisor
    - send notification to management
    - update priority to critical
```

System Interaction Patterns -- Integration archetypes:

```
- name: hr_system_sync
  pattern: bidirectional_sync
  external_system: HR system of record
  data_exchanged: [employee_records, org_hierarchy]
  trigger: scheduled + event-driven
  business_purpose: "Keep employee data consistent across systems"
```

All behavioral patterns express the customer's business intent in domain language. No system-specific execution semantics, proprietary event system internals, or internal API references are included.

5.8 Platform-Neutral Transformation and Dual-Schema Output

The Platform Transformer Agent converts the application genome into the canonical format through a platform abstraction layer that replaces all remaining system-specific constructs with generic, implementation-independent representations.

Flat Schema (Document) Representation:

```
application_genome:
  metadata:
    genome_version: "2.0"
    extraction_date: "2026-03-15"
    source_system_type: "enterprise_software_system"
    artifact_modalities: [configuration_export, api_spec, video, document]

  structural:
    domain_model:
      entities:
        - name: hr_case
          fields: [id, employee_id, type, status, priority, description]
          purpose: "Tracks employee HR requests"
          relationships:
            - target: employee
              type: belongs_to
            - target: task
              type: has_many

  behavioral:
    business_rules:
      - name: auto_route_by_region
        trigger: case_created
        condition: employee.region is defined
        action: assign to regional HR team based on employee location
    patterns:
      - name: sla_escalation
        archetype: escalation_chain
        trigger: threshold_breach
        behavior: progressive_escalation

  process_intent:
```

```
    workflows:
      - name: case_lifecycle
        steps: [submit, route, approve, fulfill, resolve, close]
        actors: [employee, manager, hr_agent]

  ui_semantics:
    screens:
      - name: case_submission_form
        purpose: "Allow employees to submit HR cases"
        components: [employee_lookup, case_type_dropdown, description_field, submit_button]

  system_interactions:
    integrations:
      - name: hr_system_sync
        direction: bidirectional
        purpose: "Employee data synchronization"
    api_surface:
      - operation: submit_case
        method: POST
        purpose: "Submit new HR case"
      - operation: approve_case
        method: POST
        purpose: "Approve pending case"
```

Graph Schema (Entity-Relationship) Representation:

```
genome_graph:
  nodes:
    - id: entity:hr_case
      type: entity
      facet: structural
      properties: {name: hr_case, purpose: "Tracks employee HR requests"}
    - id: workflow:case_lifecycle
      type: workflow
      facet: process_intent
      properties: {name: case_lifecycle, steps: 6}
    - id: rule:auto_route_by_region
      type: business_rule
      facet: behavioral
      properties: {name: auto_route_by_region, trigger: case_created}
    - id: screen:case_submission_form
      type: screen
      facet: ui_semantics
      properties: {name: case_submission_form, components: 4}

  edges:
    - from: workflow:case_lifecycle
      to: entity:hr_case
      relationship: operates_on
    - from: rule:auto_route_by_region
      to: entity:hr_case
      relationship: triggered_by
    - from: screen:case_submission_form
```

```
      to: entity:hr_case
      relationship: creates
    - from: workflow:case_lifecycle
      to: screen:case_submission_form
      relationship: initiated_by
```

This output is explicitly designed to be rebuildable on any technology stack without reference to the originating system. The genome contains no system-specific identifiers, no proprietary schema elements, no execution constructs, and no internal API references. It represents exclusively the customer's authored business intent in a canonical, dual-schema form.

5.9 Completeness and Rebuildability Assessment

The Genome Validator Agent produces a quality report:

```
validation:
  completeness_score: 78
  facet_scores:
    structural: 92
    behavioral: 81
    process_intent: 74
    ui_semantics: 68
    system_interactions: 55

  missing_components:
    - type: integration_credentials
      facet: system_interactions
      description: "Integration connection details not available in provided artifacts"
    - type: complex_business_logic
      facet: behavioral
      description: "Complex scripted logic may not be fully captured in business rule summaries"

  risks:
    - description: "Approval delegation logic may behave differently on target platform"
      impact: "Medium - manual testing recommended"

  recommendations:
    - action: "Provide additional artifacts covering integration definitions"
    - action: "Provide video demonstration of approval delegation workflow"
    - action: "Review extracted business rules against original system for completeness"

  cross_modal_consistency:
    status: "PASS"
    notes: "Workflow steps from configuration export consistent with video demonstration"

  boundary_audit:
    platform_identifiers_removed: true
    vendor_api_references_removed: true
    proprietary_schema_elements_removed: true
    execution_code_excluded: true
    audit_status: "PASS - no proprietary constructs detected in output genome"
```

The completeness score enables automated decision-making: scores above 80 indicate the genome is likely sufficient for rebuilding; scores below 50 suggest additional source material is needed. Per-facet scores identify which aspects of the application are well-captured and which need additional artifact input. The boundary audit confirms that the output genome contains no proprietary constructs.

5.10 Genome as a Persistent System of Record

The application genome is designed to serve as a persistent, versioned, reusable system of record for the application's business intent throughout its lifecycle:

5.10.1 Persistence and Versioning

The genome is stored in a version-controlled repository with full change history. Each extraction, enrichment, or manual refinement creates a new genome version with:

- Immutable version snapshots: Each genome version is preserved as a complete, self-contained artifact that can be retrieved, compared, or restored at any point
- Change attribution: Each genome modification records what changed, when, why, and from which source artifact or user action
- Diff capability: Any two genome versions can be compared to produce a structured diff showing added, modified, and removed genome components across all facets
- Branch support: Multiple genome variants (e.g., representing different migration scenarios or platform-specific adaptations) can coexist as branches with merge capability

5.10.2 Lifecycle Independence

The genome persists independently of any deployment platform or technology stack:

- Pre-migration: The genome serves as the authoritative specification of what the application does, extracted from the current platform
- During migration: The genome serves as the source of truth for what must be rebuilt on the target platform, with completeness scores tracking migration progress
- Post-migration: The genome serves as living documentation of the application's business intent, updated as the application evolves on the new platform
- Cross-platform: The genome can be used to deploy the same application intent on multiple platforms simultaneously, or to compare implementations across platforms against the canonical genome

5.10.3 Reusability

The genome's canonical representation enables reuse across multiple contexts:

- Code generation: The genome can be consumed by code generation tools to produce application scaffolding, API implementations, database schemas, and UI components on any target technology stack
- Compliance and audit: The genome provides a business-readable specification of what the application does, suitable for regulatory review, compliance auditing, and risk assessment without requiring access to platform-specific implementation details
- Application portfolio analysis: Multiple application genomes can be compared, analyzed for redundancy, and consolidated at the business-intent level
- Knowledge transfer: The genome serves as platform-independent documentation that enables teams to understand application behavior without expertise in any specific platform
- Transformation recipes: Reusable transformation patterns (translation recipes) can be applied to the genome to produce target-platform-specific implementations, and these recipes themselves become versioned, reusable assets

5.10.4 Genome Evolution

The genome evolves through multiple mechanisms:

- Re-extraction: New artifacts from the source system can be ingested to update the genome with changes made on the originating platform
- Manual enrichment: Domain experts can add, refine, or correct genome components based on knowledge not captured in source artifacts
- Cross-pollination: Behavioral patterns, process templates, and architectural patterns extracted from one application genome can be applied to other applications
- Translation application: Transformation recipes applied to the genome produce new genome facets (e.g., target-platform-specific implementations) that are stored as genome extensions

5.11 Hybrid Pipeline Architecture

The extraction pipeline employs a coordinated hybrid architecture that combines deterministic and non-deterministic processing stages:

5.11.1 Deterministic Stages

Deterministic stages provide reliable, repeatable processing for well-structured inputs:

- Format detection and parsing: File type identification, XML/JSON/YAML/CSV parsing, schema validation
- Record counting and metadata extraction: Structural analysis of export files
- Schema mapping: Mapping known export schemas to canonical genome structures using predefined rules
- Deduplication by identity: Matching identical records across multiple exports by primary key or canonical identifier
- Validation: Schema validation of genome output, completeness scoring arithmetic, boundary audit checks

5.11.2 Non-Deterministic Stages

Non-deterministic stages provide AI-driven semantic analysis for ambiguous, unstructured, or cross-modal inputs:

- Content classification: Distinguishing customer-authored components from system infrastructure using contextual analysis
- Semantic extraction: Interpreting business intent from configuration patterns, script summaries, and naming conventions
- Behavioral pattern recognition: Identifying reusable business patterns across heterogeneous artifacts
- Cross-modal reconciliation: Aligning observations from different artifact modalities (configuration, video, document)
- Intent inference: Inferring the business purpose of components when explicit documentation is absent
- Natural language summarization: Producing business-readable descriptions of complex logic patterns

5.11.3 Coordination

The hybrid pipeline coordinates deterministic and non-deterministic stages through:

- Deterministic-first processing: Where possible, deterministic stages run first to provide structured context that improves non-deterministic stage accuracy
- Confidence scoring: Non-deterministic stages produce confidence scores that determine whether human review is recommended
- Fallback chains: When non-deterministic stages produce low-confidence results, alternative extraction strategies are attempted

- Consistency verification: Deterministic validation stages verify that non-deterministic outputs are structurally valid and internally consistent

5.12 Non-Derivative Transformation Guarantee

The system's output -- the portable application genome -- is not a derivative work of the originating enterprise software system for the following reasons:

1. No Protected Expression Is Reproduced. The system does not copy, reproduce, or adapt any protectable expression of the platform vendor. Platform source code, compiled binaries, proprietary frameworks, and vendor-authored UI components are never accessed, read, or extracted. The system operates exclusively on customer-provided application artifacts, which themselves are records of the customer's design decisions stored in the system's format.

2. The Transformation Operates at the Idea Level, Not the Expression Level. Copyright protects expression, not ideas. The customer's business intent -- "route HR cases by region," "require manager approval before fulfillment," "escalate when SLA is breached" -- constitutes unprotectable ideas, processes, and methods of operation. The system extracts these ideas and re-expresses them in an entirely new canonical form that shares no protectable expression with any platform.

3. Systematic Elimination of System-Specific Expression. The legal boundary enforcement mechanisms (Section 5.2) ensure that all system-specific identifiers, schema elements, API signatures, scripting constructs, and execution semantics are systematically removed during extraction. What remains is the customer's business intent expressed in a new, independently structured canonical format.

4. No Substantial Similarity to Platform Implementation. The output genome bears no structural, syntactic, or organizational resemblance to any platform's proprietary implementation. The four-level abstraction process (Section 5.2.4) ensures progressive divergence from the source format at each stage.

5. Independent Utility Without Platform Reference. The portable genome is designed to be consumed by development teams and code generation tools that have no access to and no knowledge of the originating platform. The genome is self-contained and can be implemented on any technology stack without referencing any platform's documentation, APIs, or architecture.

---

6. Key Claims

Independent Claims

Claim 1. A computer-implemented method for extracting customer-authored business logic independent of platform-specific implementation from an enterprise software system and producing an application genome comprising:

(a) receiving one or more application artifacts containing customer-authored business logic embedded in system-specific format, wherein the artifacts originate from one or more enterprise software systems including configuration-driven platforms, low-code platforms, SaaS applications, or custom-built enterprise systems;

(b) parsing the application artifacts to identify and separate customer-authored components from system infrastructure artifacts, excluding proprietary execution code, runtime architecture, and internal system APIs;

(c) extracting customer-authored business logic independent of platform-specific implementation using one or more language models, wherein the extraction produces a canonical representation encompassing structural components, behavioral patterns, process intent, UI semantics, and system interactions;

(d) converting system-specific terminology, identifiers, and proprietary schema elements into platform-neutral, business-readable equivalents, thereby removing all system-specific execution constructs;

(e) producing an application genome -- a canonical representation of application intent, structure, and behavior -- that represents the customer's intellectual property independently of the originating system, wherein the genome contains no proprietary execution code, no runtime architecture details, and no internal API references;

(f) assessing the completeness of the extracted application genome through automated validation with a quantified completeness score and a boundary audit confirming the absence of proprietary constructs in the output.

Claim 2. A system for producing application genomes from enterprise software system artifacts comprising:

(a) an artifact ingestion agent configured to accept application artifacts from heterogeneous sources and modalities, and to identify customer-authored components, excluding system infrastructure records and internal metadata;

(b) a genome extraction agent configured to extract customer-authored business logic independent of platform-specific implementation using one or more language models with specialized prompting, producing a genome encompassing structural, behavioral, process intent, UI semantic, and system interaction facets;

(c) a genome merger configured to deduplicate and unify customer IP across multiple artifacts and modalities;

(d) a deep analysis agent configured to extract reusable behavioral patterns from customer artifacts, abstracting system-specific implementations into business-readable logic patterns;

(e) a platform transformer configured to convert the extracted genome into a canonical, platform-neutral representation through a platform abstraction layer that replaces all system-specific constructs with generic, implementation-independent representations;

(f) a genome validator configured to assess extraction completeness, identify gaps, and verify that no proprietary constructs remain in the output genome.

Claim 3. A method for producing a platform-neutral portable application architecture from enterprise software system artifacts comprising:

(a) extracting customer-authored business logic independent of platform-specific implementation from application artifacts, excluding proprietary execution code, runtime architecture, and internal system APIs;

(b) transforming extracted entities into domain model objects expressed as platform-independent data structures;

(c) transforming extracted business rules into automation rules expressed as declarative trigger-action pairs in business-readable language;

(d) transforming extracted UI components into screen specifications expressed as abstract screen-component hierarchies with business-purpose annotations;

(e) transforming extracted operations into API specifications expressed as technology-stack-independent operation definitions;

(f) transforming extracted workflows into service orchestration specifications expressed as ordered business step sequences;

(g) transforming extracted integration points into system interaction specifications expressed as platform-neutral data exchange patterns;

(h) producing a unified portable genome document suitable for rebuilding the application on a different technology stack, wherein the document contains exclusively customer-authored business intent and no proprietary constructs.

Claim 4. A computer-implemented method for converting application artifacts from any enterprise software system into a canonical intermediate representation for cross-platform portability comprising:

(a) receiving application artifacts from one or more enterprise software systems, wherein the artifacts encode customer-authored application logic in one or more system-specific formats;

(b) analyzing the application artifacts to identify components that represent customer-authored business intent as distinct from system infrastructure, platform framework code, and vendor-internal constructs;

(c) extracting the identified customer-authored components and transforming them into a canonical intermediate representation -- an application genome -- that is independent of any specific enterprise software system, technology stack, or implementation framework;

(d) wherein the canonical intermediate representation encompasses at minimum: a structural facet representing the application's data model and entity relationships, a behavioral facet representing the application's business rules and operational logic, and a process facet representing the application's workflows and orchestration sequences;

(e) wherein the canonical intermediate representation is consumable by downstream systems to produce implementations on target technology stacks without reference to the originating system;

(f) wherein the transformation systematically removes all system-specific identifiers, proprietary schema elements, and execution constructs such that the canonical intermediate representation cannot be used to reconstruct the originating system's proprietary architecture.

Dependent Claims

Claim 5. The method of Claim 1 wherein the customer-authored components are identified by distinguishing customer configuration records from system infrastructure records within the application artifacts.

Claim 6. The method of Claim 1 wherein converting system-specific terminology comprises replacing internal function names, table prefixes, system identifiers, and proprietary schema references with business-readable names derived from the

customer's labels and descriptions.

Claim 7. The system of Claim 2 wherein the genome merger resolves conflicts between duplicate component definitions appearing in multiple artifacts by preferring the more complete definition.

Claim 8. The system of Claim 2 wherein the deep analysis agent extracts three categories of behavioral patterns: logic patterns (trigger-action abstractions), business processes (step-by-step with actors), and events (trigger-effect mappings), all expressed as customer business intent independent of system-specific execution semantics.

Claim 9. The method of Claim 1 further comprising extracting reusable behavioral patterns including: inventory management patterns, approval flow patterns, request lifecycle patterns, status transition patterns, escalation chain patterns, and data mutation patterns, wherein each pattern represents the customer's authored business logic independent of platform-specific implementation.

Claim 10. The method of Claim 3 wherein the portable architecture specification comprises: domain model entities, application services with typed inputs/outputs, ordered workflow steps, API operation definitions, UI screen specifications with components, automation rules as trigger-action pairs, and system interaction specifications, and wherein the specification excludes all proprietary execution code, runtime architecture, and internal system APIs.

Claim 11. The method of Claim 1 wherein the completeness score is computed on a 0-100 scale based on the presence and richness of extracted components across each genome facet: structural, behavioral, process intent, UI semantics, and system interactions.

Claim 12. The system of Claim 2 further comprising a genome repository service that stores the extracted genome as a persistent, versioned system of record with full change history, diff capability, and branch support.

Claim 13. The method of Claim 1 wherein the agents execute with real-time progress streaming via Server-Sent Events, each agent reporting start, completion with metrics, or error status.

Claim 14. The method of Claim 1 wherein the genome merger uses one or more language models to identify semantic equivalence between components beyond exact name matching.

Claim 15. The method of Claim 3 wherein the portable architecture specification is structured as YAML or JSON and can be consumed by code generation tools to produce application scaffolding on target technology stacks.

Claim 16. The method of Claim 1 further comprising a validation report that identifies: missing components (with facet, type, and description), risks (with impact assessment), recommendations (actionable steps to improve coverage), cross-modal consistency status, and a boundary audit confirming the absence of proprietary constructs.

Claim 17. The system of Claim 2 wherein the artifact ingestion agent supports multiple artifact formats and handles variations in schema across different enterprise software systems and export tools.

Claim 18. The method of Claim 1 wherein the extraction is performed without accessing the originating system's runtime environment, operating solely on exported or documented application artifacts provided by the customer.

Claim 19. The system of Claim 2 wherein the genome extraction agent uses a system prompt that explicitly instructs the model to summarize business intent rather than reproducing proprietary code, and to exclude proprietary execution code, runtime architecture, and internal APIs from all output.

Claim 20. The method of Claim 1 further comprising support for multi-source ingestion wherein artifacts from multiple systems, exports, or modalities are combined into a single unified genome, enabling recovery of customer IP that spans multiple systems, scopes, or artifact types.

Claim 21. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

Claim 22. The method of Claim 1 further comprising a platform abstraction layer that converts all system-specific constructs into generic, implementation-independent representations at four progressive levels of abstraction: raw extraction, semantic normalization, behavioral abstraction, and portable architecture, wherein each successive level increases the distance from the originating system's implementation and moves closer to the customer's pure business intent.

Claim 23. The method of Claim 1 further comprising a system-specific artifact stripping process that systematically identifies and removes system-specific identifiers including internal namespace prefixes, platform object identifiers, system-specific field names, and proprietary schema elements, replacing them with customer-defined business names or semantically equivalent generic identifiers, such that the output genome contains no artifacts traceable to the originating system's proprietary implementation.

Claim 24. The method of Claim 1 wherein the extraction produces an intent-level reconstruction of the customer's business logic rather than an implementation-level reproduction of the system's execution artifacts, such that the output genome expresses what the customer's application does in business terms without encoding how the originating system implements it, and wherein the output genome bears no structural, syntactic, or organizational resemblance to the originating system's implementation.

Claim 25. The method of Claim 1 further comprising multi-modal extraction wherein the application artifacts include two or more of: configuration exports in structured data formats, API specifications, video demonstrations or screen recordings, and business documents or process diagrams, and wherein the genome extraction pipeline ingests, analyzes, and reconciles observations from the multiple modalities into a unified application genome.

Claim 26. The method of Claim 1 wherein the application genome is represented simultaneously in both a flat schema suitable for document-oriented consumption and code generation, and a graph schema suitable for dependency analysis and structural comparison, wherein both representations are derived from a single canonical genome model and are guaranteed to be mutually consistent.

Claim 27. The method of Claim 1 wherein the extraction pipeline comprises both deterministic processing stages that perform

rule-based parsing, schema validation, and structural analysis, and non-deterministic processing stages that perform AI-driven semantic extraction, behavioral pattern recognition, and intent inference, coordinated through a hybrid pipeline architecture that sequences deterministic stages before non-deterministic stages where possible and employs confidence scoring to determine when human review is recommended.

Claim 28. The method of Claim 4 wherein the canonical intermediate representation further encompasses a UI semantics facet representing the application's user interface intent and a system interaction facet representing the application's integration patterns and external communication specifications.

Claim 29. The method of Claim 4 wherein the canonical intermediate representation is maintained as a persistent, versioned system of record with immutable version snapshots, change attribution, diff capability across versions, and branch support for representing multiple migration or adaptation scenarios.

Claim 30. The system of Claim 2 further comprising a genome evolution mechanism wherein the application genome is updated through re-extraction from updated source artifacts, manual enrichment by domain experts, cross-pollination of patterns from other application genomes, and application of reusable transformation recipes.

---

7. Advantages Over Prior Art

1. Category-defining genome model: Establishes the application genome as a canonical representation of application intent, structure, and behavior -- a new category of software artifact distinct from source code, configuration, and documentation
2. Customer IP sovereignty: Explicitly separates customer-authored business logic from platform implementation, asserting customer ownership of their designs
3. Vendor lock-in remediation: Produces portable genomes that free customer IP from dependence on any single vendor or technology stack
4. Platform-neutral output: Canonical genome specification can be rebuilt on any modern technology stack without reference to the originating system
5. Multi-modal extraction: Accepts application artifacts from heterogeneous sources (configuration exports, APIs, video, documents) enabling genome extraction even when no single artifact type captures the complete application
6. Multi-source unification: Handles the real-world case where customer IP is distributed across multiple systems, exports, or artifact types
7. Behavioral abstraction: Extracts reusable patterns (approval flows, inventory management, request lifecycles) in business-readable language, independent of system-specific function names

8. Dual-schema representation: Provides both flat (document) and graph (entity-relationship) representations of the same genome, enabling different consumption patterns for code generation, dependency analysis, and portfolio management
9. Hybrid pipeline: Combines deterministic reliability with AI-driven depth, using rule-based stages for structured inputs and AI stages for semantic analysis, coordinated through confidence scoring
10. Quality quantification: Per-facet completeness scoring and gap analysis enable informed decision-making about extraction sufficiency
11. No runtime access required: Operates solely on exported or documented application artifacts -- no access to any system's live environment is needed
12. Legal defensibility: Systematic removal of all system-specific constructs and four-level abstraction process ensures the output is demonstrably not a derivative work of the originating platform
13. Persistent system of record: The genome serves as a versioned, reusable asset throughout the application's lifecycle -- not a one-time migration artifact but a living representation of business intent
14. Scalability: Automated pipeline enables extraction from large application portfolios that would be infeasible to recover manually
15. Audit trail: Version-controlled storage with change attribution provides complete provenance of extracted customer IP
16. Legal boundary enforcement: Four-layer mechanism provides auditable proof that no proprietary constructs are present in the output
17. Cross-platform applicability: System is not specific to any particular enterprise software vendor or platform type -- applicable to configuration-driven platforms, low-code environments, SaaS applications, and custom-built systems

---

8. Drawings/Figures Description

Figure 1: Application Genome Model -- Conceptual diagram showing the five genome facets (structural, behavioral, process intent, UI semantics, system interactions) as a unified canonical model with relationships between facets.

Figure 2: Customer IP vs. Platform IP -- Conceptual diagram showing the boundary between customer-authored business intent and platform-provided infrastructure, with the extraction boundary clearly marked and explicit exclusion zones for proprietary execution code, runtime architecture, and internal APIs.

Figure 3: Multi-Agent Pipeline Architecture -- Block diagram showing the six agents (Artifact Ingestion, Genome Extraction, Genome Merger, Deep Analysis, Platform Transformer, Genome Validator) with data flow arrows, deterministic/non-deterministic stage indicators, and legal boundary enforcement checkpoints.

Figure 4: Multi-Modal Ingestion Flow -- Diagram showing how four artifact types (configuration export, API specification, video demonstration, business document) are ingested through modality-specific processing pipelines and converge into a unified extraction input.

Figure 5: Multi-Source Merge and Cross-Modal Reconciliation Flow -- Diagram showing how artifacts from multiple sources and modalities are parsed, individually extracted, and then merged into a unified genome with deduplication and cross-modal consistency verification.

Figure 6: Behavioral Pattern Extraction -- Example showing a system-specific script reference being transformed into a business-readable logic pattern, process, and event definition, with system-specific constructs being stripped at each transformation stage.

Figure 7: Platform-Neutral Transformation -- Side-by-side comparison showing system-specific artifacts (left) and the canonical genome (right) with mapping arrows between corresponding components.

Figure 8: Dual-Schema Representation -- Side-by-side view of the same genome in flat (document/YAML) and graph (entity-relationship/dependency) formats with correspondence annotations.

Figure 9: Completeness Scoring Model -- Radar chart showing per-facet completeness scores (structural, behavioral, process intent, UI semantics, system interactions) with example scores for a typical extraction.

Figure 10: Genome as System of Record Lifecycle -- Timeline diagram showing the genome's evolution from initial extraction through migration, post-migration updates, and ongoing maintenance as a persistent system of record.

Figure 11: Hybrid Pipeline Architecture -- Diagram showing the coordination of deterministic (rule-based) and non-deterministic (AI-driven) processing stages with confidence scoring and fallback chains.

Figure 12: Portable Genome Structure -- Entity-relationship diagram showing domain model, application services, API surface, UI model, automation rules, and system interactions with their relationships.

Figure 13: Repository Output Structure -- Directory tree showing versioned genome files, flat and graph representations, validation reports, and audit trail.

Figure 14: Legal Boundary Enforcement Architecture -- Four-layer diagram showing progressive abstraction from system-specific artifacts to platform-neutral business intent, with artifact stripping at each layer and boundary audit verification.

Figure 15: Non-Derivative Transformation Flow -- Structural comparison demonstrating progressive divergence from source format at each abstraction level.

Figure 16: Cross-Platform Genome Applicability -- Diagram showing the same genome extraction process applied to multiple enterprise software system types (configuration-driven, low-code, SaaS, custom-built) producing a unified canonical genome.

---

9. Inventors

[To be completed by filing attorney]

10. Filing Notes

- Category-defining framing is critical: This patent should be positioned as defining the category of "Application Genome Modeling" -- the first system to define, extract, and maintain a canonical representation of application intent, structure, and behavior. The five-facet genome model (structural, behavioral, process intent, UI semantics, system interactions) is the category definition.
- Broad applicability: The claims are deliberately not limited to any specific enterprise software vendor, platform type, or artifact format. Claim 4 in particular covers ANY system that converts application artifacts into a canonical intermediate representation for portability. This breadth should be preserved during prosecution.
- IP ownership framing: The core legal argument that customers own the business logic they design should be emphasized throughout prosecution.
- No reverse engineering of platform code: The system explicitly does NOT extract or reproduce proprietary execution code, runtime architecture, or internal APIs. It extracts only customer-authored business logic independent of platform-specific implementation. This distinction is essential for avoiding vendor IP claims.
- Non-derivative work argument: The four-level abstraction process (Section 5.2.4) and the Non-Derivative Transformation Guarantee (Section 5.12) provide a structured legal argument that the output genome is not a derivative work.
- Multi-modal extraction as differentiation: The ability to extract application genomes from heterogeneous artifact types (configuration, API, video, document) is a significant differentiator from any prior art focused solely on configuration migration.
- Dual-schema representation as differentiation: The simultaneous flat and graph representations of the same genome model, with guaranteed consistency, is independently novel.
- Hybrid pipeline as differentiation: The coordinated deterministic + non-deterministic pipeline

architecture with confidence scoring is independently novel.

- Genome as system of record: The persistent, versioned, reusable genome concept is independently novel and distinguishes this from one-time migration tools.
- Data portability regulations: Reference relevant data portability rights (GDPR Article 20, proposed US data portability legislation) as additional support for the right to extract customer-authored configurations.
- Configuration artifacts vs. source code: Emphasize that application artifacts are customer data (business process definitions), not platform source code. The customer created the configurations; the platform stores them.
- Claims 25-30 future-proof against competitors: These dependent claims cover multi-modal extraction (Claim 25), dual-schema representation (Claim 26), and hybrid pipelines (Claim 27) -- anticipated competitive approaches.
- Consider filing as a continuation of Patent 1 to establish a patent family, or as an independent application given the expanded scope.
- Prior art search should focus on: data portability tools, application migration utilities, model-driven engineering, configuration management databases, vendor lock-in remediation systems, application portfolio management tools, digital twin platforms.

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Progressive Context Hydration

Title

System and Method for LLM-Directed Iterative Context Hydration with Programmatic Enforcement of Minimum Context Acquisition Enabling Retrieval-Driven Application Reconstruction with Sublinear Token Scaling

1. Field of Invention

This invention relates to artificial intelligence systems, dynamic context management, retrieval-driven generation, computational resource optimization, and enterprise application reconstruction. Specifically, it describes a system and method wherein a large language model (LLM) actively directs its own context acquisition through a structured iterative retrieval protocol with programmatic enforcement of minimum context acquisition -- rather than receiving static pre-loaded context or passively accepting retrieval system selections -- enabling efficient, high-fidelity generation from structured repositories and source documents with token consumption that scales sublinearly relative to source application size.

2. Background and Prior Art Gaps

2.1 The Context Problem in LLM-Mediated Generation

Complex generation tasks -- including but not limited to enterprise application reconstruction, cross-platform migration, document synthesis, and code generation -- require the LLM to understand a substantial body of source material before producing output. The source material may comprise hundreds to thousands of interrelated artifacts: data models, workflows, business rules, UI definitions, integration configurations, API specifications, and documentation.

The fundamental challenge is: how does the LLM acquire sufficient context to produce high-fidelity output without exceeding computational resource limits or degrading generation quality?

Prior approaches to providing this context fall into three categories, each with fundamental limitations that the present invention overcomes:

2.2 Comparison with Static Prompting

In static prompting, the entire source material is concatenated into a single LLM prompt prior to generation.

Limitations:

- Context window overflow: Large applications exceed model context limits (e.g., 128K-200K tokens), causing hard failure or silent truncation
- Attention degradation: Even within context limits, LLM attention quality degrades on long contexts -- elements early in the prompt receive diminished attention by the time the model processes the end, producing output that favors later context elements over earlier ones
- Wasteful token consumption: The entire source is tokenized regardless of relevance to the specific generation task. For a 200KB source where only 20KB is relevant, 90% of tokens are wasted
- No adaptive retrieval: The context is fixed before generation begins. The model cannot discover that it needs additional information partway through processing
- Linear cost scaling: Token consumption scales linearly with source size. A 10x larger application consumes 10x more tokens

How the present invention differs: The present invention provides LLM-directed retrieval with enforced minimum context acquisition. The LLM examines a lightweight index of the source material, decides which specific artifacts it needs, retrieves them incrementally across multiple rounds, and is programmatically prevented from generating output before acquiring sufficient context. Token consumption is bounded by a configurable context budget and scales sublinearly with source size.

2.3 Comparison with Retrieval-Augmented Generation (RAG)

In RAG, a retrieval system selects documents based on semantic similarity to the user's query and provides them to the LLM as context.

Limitations:

- System-directed, not LLM-directed: The retrieval system (embedding model + vector store) decides what to retrieve; the LLM passively receives the result. The LLM cannot correct retrieval errors or request additional context
- Single-pass retrieval: One query produces one retrieval set. The system cannot adapt its retrieval strategy based on what the LLM discovers in the retrieved documents
- Semantic similarity mismatch: RAG retrieves by semantic similarity to the query, but structured reconstruction tasks require dependency-aware retrieval. When building an application from a genome, the LLM needs "the approval workflow" not because it's semantically similar to the user's query, but because the catalog form references it. Semantic similarity does not capture

structural dependencies

- No minimum-retrieval enforcement: Nothing prevents the LLM from generating output with zero or insufficient retrieved context. The system has no contract that generation must be preceded by adequate retrieval
- No iterative refinement: The LLM cannot say "I've read the workflow, and now I realize I also need the approval table." RAG retrieves once; the present invention retrieves iteratively until the LLM is satisfied or a budget is reached
- No retrieval planning: The LLM has no mechanism to articulate or reason about its retrieval strategy before executing it

How the present invention differs: The present invention inverts the RAG control flow. Instead of a retrieval system selecting context for the LLM, the LLM directs the retrieval system. The LLM examines an index, formulates a retrieval plan, requests specific artifacts, examines them, discovers dependencies, requests additional artifacts, and iterates until it determines it has sufficient context. A programmatic guard layer enforces that the LLM must retrieve at least some source material before generating output, preventing hallucinated generation. The LLM's retrieval plan is explicitly articulated in each round, providing both reasoning audit trails and adaptive strategy refinement.

2.4 Comparison with Agentic Tool Calling (ReAct, Function Calling)

In agentic tool calling frameworks (ReAct, OpenAI function calling, Anthropic tool use), LLMs invoke tools through a general-purpose action-observation loop.

Limitations:

- No structured retrieval protocol: Tool calling provides a general mechanism for the LLM to invoke functions, but does not define a retrieval-specific protocol with typed request/response shapes, retrieval plans, readiness signals, and context budgets
- No minimum-retrieval enforcement: Tool calling frameworks do not enforce that the LLM must retrieve context before generating output. The LLM can freely choose to generate without invoking any tools
- No domain-specific index: Tool calling provides generic file system access or API access. It does not provide a lightweight, pre-computed entity manifest that maps application components to file paths with summaries, enabling the LLM to plan retrieval efficiently
- No context budget management: Tool calling frameworks do not track accumulated context size or enforce budget limits. The LLM may retrieve unbounded amounts of data, exceeding context limits
- No source-agnostic abstraction: Each source type (file repository, XML document, parsed PDF) requires a separate tool implementation. The present invention abstracts the retrieval

interface across multiple source types while sharing all control logic, guard validation, and context accumulation

- No retrieval-to-synthesis handoff: Tool calling frameworks do not package accumulated retrieval context into a structured handoff object (HydrationContext) that can be consumed by a downstream synthesis step while preserving the LLM's working memory
- No deduplication: Tool calling frameworks do not prevent the LLM from fetching the same file multiple times across rounds

How the present invention differs: The present invention provides a domain-optimized retrieval protocol layered above general LLM capabilities. While tool calling provides the mechanism (the LLM can invoke functions), the present invention provides the structure: a typed retrieval protocol with plan/request/ready signals, a pre-computed entity index for retrieval planning, a programmatic guard that enforces minimum context acquisition, a context budget that bounds accumulation, a deduplication layer that prevents redundant retrieval, and a structured handoff (HydrationContext) that enables retrieval-to-synthesis composition. These structural elements are absent from general tool calling frameworks and constitute the core innovation.

2.5 Summary of Differentiation

Capability	Static Prompting	RAG	Agentic Tool Calling	Present Invention (PCH)
Who directs retrieval	N/A (no retrieval)	Retrieval system	LLM (unrestricted)	**LLM (with guard enforcement)**
Retrieval rounds	0	1	Unbounded	**Multi-round with budget**
Minimum-context enforcement	None	None	None	**Programmatic guard layer**
Retrieval planning	None	None	Implicit	**Explicit plan field per round**
Domain-specific index	None	Embedding index	None	**Entity manifest with paths**
Context budget	None	Top-K limit	None	**Configurable char/token budget**
Source abstraction	N/A	Vector store	Per-tool	**Shared protocol, source-specific fetch
Token scaling	Linear	Sub-linear (fixed K)	Unbounded	**Sublinear with budget enforcement**
Deduplication	N/A	N/A	None	**Path-based deduplication**
Synthesis handoff	N/A	Injected context	None	**HydrationContext object**

2.6 Gaps in Prior Art

1. No existing system allows an LLM to actively direct its own context acquisition through a structured multi-turn protocol with explicit retrieval planning, readiness signaling, and minimum-context enforcement.

2. No existing system produces a lightweight entity index (entity manifest with file paths and summaries, no content) as a first-class artifact that initializes the retrieval loop, enabling the LLM to plan retrieval efficiently without loading source content.

3. No existing system implements a source-agnostic retrieval abstraction that applies the same LLM-directed loop protocol to multiple source types: version-controlled file repositories, structured XML documents, and parsed document sections.

4. No existing system enforces a minimum-retrieval contract (guard validation) that programmatically rejects premature generation before sufficient context has been acquired, with automatic retry and correction injection.

5. No existing system provides a programmatic enforcement layer preventing premature generation that is mechanically enforced (not merely instructed), stateful (tracking files fetched per round), and self-correcting (injecting specific correction messages and retrying).

6. No existing system applies iterative context hydration to the extraction phase itself -- allowing the LLM to selectively load record types from large source documents rather than receiving the entire document at once.

7. No existing system combines: entity index generation, LLM-directed retrieval, guard enforcement, progressive context accumulation, context budget management, and downstream synthesis handoff as a unified, composable pipeline component.

8. No existing system achieves sublinear token scaling relative to source application size through the combination of index-based retrieval planning, selective content loading, and context budget enforcement.

---

3. Summary of Invention

The present invention provides a system and method called Progressive Context Hydration (PCH) -- an LLM-directed retrieval protocol with programmatic enforcement of minimum context acquisition that achieves sublinear token scaling for generation tasks over large structured sources.

The system comprises:

1. Entity Index Generation: At source commit time, a deterministic entity index is generated alongside source files. The index maps all application entities (forms, workflows, tables, UI components, integrations) to their repository file paths with short summaries -- providing a lightweight manifest of the entire application structure without file content. The index typically represents 1-5% of the full source size in tokens, enabling the LLM to comprehend the entire application structure before committing to retrieve any content.

2. Hydration Loop Initialization: When an LLM-mediated operation is requested (transformation, translation, extraction), the system initializes the loop by:
   - Listing available source files (file tree or record type index)
   - Fetching and embedding the entity index as the first message
   - Presenting a structured retrieval protocol to the LLM
   - Establishing the context budget for the session

3. LLM-Directed Retrieval with Enforced Minimum Context Acquisition: The LLM responds with a structured JSON request identifying which specific files or record types it needs, along with its explicit retrieval plan. The protocol enforces two response shapes:
   - Retrieval request: {"plan": [...], "required_files": ["path/to/file"]}
   - Ready signal: {"plan": [...], "ready": true, "reason": "..."}

The LLM iterates through multiple retrieval rounds, each time examining what it has already loaded, reasoning about what additional context it needs, and requesting specific artifacts. This is fundamentally different from RAG (where the retrieval system selects context) and from tool calling (where the LLM can freely skip retrieval).

4. Programmatic Enforcement Layer (Guard Validation): A stateful guard layer programmatically prevents premature generation. If the LLM attempts to produce output before fetching any source material, the response is mechanically rejected -- not merely discouraged through prompt instructions -- and a correction message is injected into the conversation, forcing a retry. This enforcement layer prevents "hallucinated generation" -- plausible-looking but contextually unfounded output -- regardless of the LLM's inclination, prompt injection attempts, or configuration errors.

5. Incremental Context Accumulation with Budget Enforcement: Retrieved file contents are appended to the conversation context. The LLM sees accumulated context growing across rounds, enabling it to discover dependencies iteratively (e.g., "now that I see the workflow, I also need the approval table definition"). A configurable context budget caps total accumulated context, forcing the LLM to produce output with available context when the budget is reached.

6. Ready-to-Build Transition: When the LLM signals readiness (or maximum rounds are reached, or context budget is exhausted), the accumulated context is packaged into a HydrationContext object containing: loaded files, conversation history, accumulated retrieval plan, and token usage statistics.

7. Downstream Synthesis Handoff: The HydrationContext is passed to a synthesis step that generates the final output (filesystem plan, genome YAML, application spec) using only the

retrieved context -- never the full source. The conversation history is preserved, enabling the synthesis step to continue the same LLM conversation with the accumulated working memory intact.

8. Source-Agnostic Retrieval Abstraction: The same loop logic is applied to multiple source types through a shared base abstraction:
   - File repository retrieval (for transformation/translation)
   - Structured record retrieval (for configuration export extraction)
   - Document section retrieval (for documentation extraction)

Each source type implements source-specific index building and content fetching while sharing all control logic, guard validation, context accumulation, budget enforcement, and deduplication.

---

4. Detailed Description

4.1 System Architecture

```
+-----------------------------------------------------------------+
|                 PROGRESSIVE CONTEXT HYDRATION                    |
|                                                                  |
|  +-----------------------------------------------------------+   |
|  |  Step 1: INDEX INITIALIZATION                              |   |
|  |  - Fetch entity index (entity manifest, no content)        |   |
|  |  - List available source files / record types              |   |
|  |  - Establish context budget for session                    |   |
|  |  - Construct initial prompt with index + user request      |   |
|  +---------------------------+-----------------------------+   |
|                              v                                   |
|  +-----------------------------------------------------------+   |
|  |  Step 2: LLM CALL (Round N)                                |   |
|  |  - Send: conversation history + accumulated context        |   |
|  |  - Receive: JSON response (retrieval request OR ready)     |   |
|  +---------------------------+-----------------------------+   |
|                              v                                   |
|  +-----------------------------------------------------------+   |
|  |  Step 3: GUARD VALIDATION (Enforcement Layer)              |   |
|  |  - REJECT if: generation output + 0 files fetched          |   |
|  |  - REJECT if: ready signal + 0 files fetched               |   |
|  |  - REJECT if: empty request + 0 files fetched              |   |
|  |  - Inject correction message + retry (max N retries)       |   |
|  |  - ACCEPT if: valid retrieval request or guarded ready     |   |
|  |  - Mechanically enforced -- cannot be bypassed by LLM       |   |
|  +---------------------------+-----------------------------+   |
```

```
|                              v                                    |
|  +-------------------------------------------------------------+   |
|  |  Step 4a: RETRIEVAL (if required_files present)           |   |
|  |  - Fetch each requested file/record via source adapter    |   |
|  |  - Deduplicate (skip already-loaded paths)                |   |
|  |  - Append content to conversation context                 |   |
|  |  - Update context budget tracking                         |   |
|  |  - Force ready if budget exceeded                         |   |
|  +---------------------------+-----------------------------+   |
|                              |                                    |
|                   [repeat until ready, max rounds, or budget]    |
|                              v                                    |
|  +-------------------------------------------------------------+   |
|  |  Step 4b: READY TRANSITION                                |   |
|  |  - Package HydrationContext                               |   |
|  |  - Include: loaded_files, messages, plan, token usage     |   |
|  |  - Preserve conversation history for synthesis handoff    |   |
|  +-------------------------------------------------------------+   |
+-------------------------------------------------------------------+
                              v
+-------------------------------------------------------------------+
|                 SOURCE-AGNOSTIC RETRIEVAL LAYER                   |
|                                                                   |
|  FileRepoHydration          RecordSetHydration   DocHydration    |
|  - list_tree()              - records_by_type    - doc_sections  |
|  - get_file()               - auto-include HPT   - page index    |
|  - path normalization       - type filtering      - section fetch|
|  [Shared: guard, budget, dedup, context accumulation, handoff]   |
+-------------------------------------------------------------------+
```

4.2 Entity Index Generation

At source commit time, the system generates a deterministic entity index alongside all source files. This index is the critical initialization artifact for the hydration loop.

Structure:

```
{
  "entities": {
    "forms": [
      {"id": "form_rental_request", "name": "Rental Request", "file_path": "structure/catalog.json"}
    ],
    "workflows": [
      {"id": "wf_approval", "name": "Standard Approval", "file_path": "structure/workflows.json"}
    ],
    "tables": [
      {"id": "tbl_assets", "name": "Asset Registry", "file_path": "structure/data_model.json"}
    ],
    "ui_components": [
      {"id": "ui_dashboard", "name": "Rental Dashboard", "file_path": "structure/ui.json"}
    ]
```

```
  },
  "relationships": {
    "form_rental_request": ["wf_approval", "tbl_assets"]
  },
  "summaries": {
    "form_rental_request": "Catalog form for requesting rental equipment. Contains 12 variables.",
    "wf_approval": "Two-stage approval workflow triggered on form submission."
  },
  "metadata": {
    "application_name": "Rental Management",
    "entity_counts": {"forms": 4, "workflows": 8, "tables": 12, "ui_components": 3}
  }
}
```

Key properties:

- No file content: Only entity names, IDs, file paths, and 1-2 sentence summaries
- Deterministic: Generated by rule-based extraction from source structure, no LLM required, same input produces identical output
- Relationship graph: Explicit entity-to-entity dependencies enable the LLM to plan multi-step retrieval ("the form depends on the workflow and the table")
- File path mapping: Each entity maps to the exact path where its full definition resides

Design rationale: The index answers the question "what exists and where is it?" in ~2-5KB, compared to the 50-200KB of the full source. This is the key efficiency driver -- the LLM understands the application structure before committing to retrieving any content. The index enables the LLM to make intelligent retrieval decisions (requesting only the artifacts relevant to the current task) rather than requiring the system to load everything or guess what might be relevant.

4.3 LLM-Directed Retrieval Protocol

The system presents a two-shape JSON protocol to the LLM:

Shape 1 -- Retrieval Request:

```
{
  "plan": [
    "Load rental_request form to understand field structure",
    "Load associated approval workflow to map trigger conditions"
  ],
  "required_files": ["structure/catalog.json", "structure/workflows.json"]
}
```

Shape 2 -- Ready Signal:

```
{
  "plan": ["Form structure, workflow, and data model fully loaded"],
```

```
  "ready": true,
  "reason": "All entities needed for the target app spec are loaded"
}
```

The plan field is required in both shapes. It serves three purposes:

1. Retrieval reasoning: Forces the LLM to reason explicitly about its retrieval strategy before requesting files, improving retrieval quality
2. Multi-round planning: Enables the LLM to articulate a multi-step retrieval plan across rounds ("first I need the primary entities, then I'll check dependencies")
3. Audit trail: Provides a human-readable record of the LLM's decision process, surfaced in the UI as progress updates

Round progression:

- Round 0 (initialization): Index fetched, first prompt constructed -- no LLM call
- Round 1: LLM analyzes index, formulates retrieval plan, requests primary files (forms, workflows)
- Round 2: LLM reads primary files, discovers dependencies, updates plan, requests secondary files (tables, configs)
- Rounds 3+: LLM requests additional context as dependencies are discovered, or signals ready
- Budget enforcement: If context budget is reached, system injects a directive requiring output with available context
- Force-ready: On final round, system injects a directive requiring ready signal

The critical difference from RAG and tool calling: In each round, the LLM's retrieval request is informed by:

1. The entity index (what exists)
2. All previously retrieved content (what it already knows)
3. The dependencies it has discovered in prior rounds (what it now realizes it needs)
4. The explicit retrieval plan it articulated in prior rounds (its evolving strategy)

This enables adaptive, dependency-driven retrieval that cannot be achieved by similarity-based retrieval (RAG) or unstructured tool calling.

4.4 Programmatic Enforcement Layer (Guard Validation)

The guard validation layer is a programmatic enforcement mechanism that prevents premature generation -- the most critical differentiator between the present invention and all prior art (static prompting, RAG, tool calling).

A stateful Guard object is instantiated per hydration loop run. It maintains retry counts per round, tracks cumulative files fetched, and enforces the minimum-retrieval contract.

4.4.1 Why Programmatic Enforcement Is Necessary

LLMs are probabilistic systems. Even with explicit instructions to "retrieve context before generating output," LLMs may:

- Generate plausible-looking output without retrieving any source material (hallucinated generation)
- Prematurely signal readiness after retrieving insufficient context
- Respond with empty retrieval requests that advance the round counter without acquiring context
- Be manipulated by prompt injection to skip retrieval

Prompt instructions alone cannot guarantee retrieval-first behavior. The guard provides mechanical enforcement -- the LLM's output is intercepted and validated before the system acts on it, regardless of the LLM's intent.

4.4.2 Rejection Conditions

Condition	Detection	Correction Injected
Premature generation	`output`/`filesystem_plan` key present A	"You must retrieve source files before g
Premature ready	`ready: true` AND `files_fetched == 0`	"You cannot signal readiness without fir
Empty request	`required_files: []` AND `files_fetched	"Your retrieval request is empty and no

4.4.3 Retry Logic

- Maximum retries per round is configurable (default: 2)
- After maximum retries, the guard accepts the response to prevent infinite loops
- Each rejection injects a specific correction message explaining the violation and providing guidance
- The correction message becomes part of the conversation history, ensuring the LLM incorporates the feedback in subsequent attempts

4.4.4 Guard as Enforcement Layer

The guard is not advisory -- it is a programmatic enforcement layer preventing premature generation:

- Mechanically enforced: The guard intercepts every LLM response before the system acts on it.

A response that violates the minimum-retrieval contract is never passed to the retrieval or synthesis stages.

- Stateful: The guard tracks cumulative files fetched across all rounds. Once the minimum retrieval threshold is met, the guard allows ready signals and generation output.
- Self-correcting: Each rejection injects a correction message that becomes part of the ongoing conversation, guiding the LLM toward correct behavior.
- Tamper-resistant: Because the guard operates at the system level (outside the LLM's conversation), it cannot be bypassed by prompt injection, instruction overriding, or LLM misconfiguration.
- Composable: The guard is a standalone component that can be applied to any LLM-directed retrieval loop, independent of the specific source type, index format, or downstream synthesis step.

4.5 Context Budget Enforcement

The system enforces a configurable context budget that caps total accumulated context across all retrieval rounds:

Budget parameters:

- MAXFILECHARS: Maximum characters loaded per individual file/record
- MAXTOTALCHARS: Maximum total characters accumulated across all rounds
- MAX_ROUNDS: Maximum number of retrieval rounds before forced ready

Enforcement mechanism:

1. After each retrieval round, the system computes total accumulated context size
2. If totalchars > MAXTOTAL_CHARS, the system injects a budget-exceeded directive: "Context budget reached. You must now produce output using the context you have accumulated."
3. The system forces a ready transition, preventing further retrieval
4. Individual files exceeding MAXFILECHARS are truncated at the file level with a truncation marker

Significance for sublinear scaling: The context budget is what enables sublinear token scaling. Regardless of source application size (100 files or 10,000 files), the total context loaded is bounded by MAXTOTALCHARS. Combined with the entity index (which gives the LLM a complete structural overview without loading content), the LLM can make intelligent retrieval decisions within the budget, loading only the most relevant artifacts for the specific task.

4.6 Source-Agnostic Loop Abstraction

The hydration loop is abstracted over multiple source types through a shared base:

4.6.1 File Repository Hydration (Transformation/Translation)

```
Source:  Version-controlled file repository
Index:   entity index (entity manifest) + directory tree
Request: required_files: ["structure/entities.json"]
Fetch:   get_file(tenant, base_path + "/" + path)
Budget:  MAX_FILE_CHARS per file, MAX_TOTAL_CHARS total
```

Used when applying translation recipes or user transformations to committed genomes.

4.6.2 Structured Record Hydration (Configuration Export Extraction)

```
Source:  Parsed structured record sets (XML, JSON)
Index:   Record type summary {type: {count, sample_names[]}}
Request: required_files: ["catalog_item", "workflow"]
Fetch:   filter records_by_type[type_name][:MAX_CHARS_PER_TYPE]
Budget:  MAX_CHARS_PER_TYPE per type, MAX_TOTAL total

Auto-include: High-priority types loaded first within half budget
```

Used during configuration export extraction. Replaces the prior approach of sending all records in a single prompt. The loop is bypassed for small record sets below a configurable threshold via direct auto-include without LLM overhead.

4.6.3 Document Section Hydration (Documentation Extraction)

```
Source:  Parsed document sections (PDF, DOCX, TXT)
Index:   Table of contents + section titles + word counts
Request: required_files: ["section_3", "appendix_b"]
Fetch:   return pre-parsed section text
Budget:  MAX_SECTION_CHARS per section, MAX_TOTAL total
```

Used when extracting genomes from large documentation files. Replaces full-document prompting.

Shared components across all source types:

- Guard validation (identical enforcement logic)
- Context budget enforcement
- HydrationContext data structure
- Conversation history management
- Token usage tracking

- Round limit enforcement
- Path-based deduplication

4.7 HydrationContext Data Structure

The output of every hydration loop run is a HydrationContext object:

```
HydrationContext:
  source_path:          source identifier
  user_request:         original user intent

  loaded_files:         path -> content map (deduplicated)
  iterations:           per-round retrieval records
  accumulated_plan:     LLM's stated reasoning across rounds

  total_chars:          total context accumulated
  total_input_tokens:   LLM input tokens consumed during hydration
  total_output_tokens:  LLM output tokens consumed during hydration
  total_latency_ms:     total time spent in hydration

  ready:                true = LLM signalled sufficient context
  ready_reason:         LLM's stated reason for readiness

  index:                parsed entity index (if present)
  file_tree:            available files/records
  messages:             full conversation history for handoff
  system_prompt:        for downstream synthesis use
```

The messages field enables a critical capability: the downstream synthesis step can continue the same conversation, receiving the accumulated context as if it were a single coherent session. This preserves the LLM's working memory across the retrieval and synthesis phases -- the LLM does not need to re-process already-loaded context or re-derive conclusions it reached during the retrieval phase.

4.8 Sublinear Context Scaling

The present invention achieves sublinear token scaling relative to source application size -- a fundamental efficiency advantage over static prompting (linear scaling), and a bounded-cost advantage over RAG (fixed top-K, potentially insufficient) and tool calling (unbounded retrieval).

4.8.1 Token Scaling Model

Static prompting (linear scaling):

Application size	Source chars	Tokens consumed	Context window	Outcome
Small (20 artifacts)	~10K chars	~2,500 tokens	Fits	Acceptable
Medium (200 artifacts)	~100K chars	~25,000 tokens	Fits	Degraded attention
Large (2,000 artifacts)	~1M chars	~250,000 tokens	Overflow	Fails / truncated
Very large (10,000 artifacts)	~5M chars	~1,250,000 tokens	Overflow	Impossible

Token consumption: O(N) where N = source size

Progressive Context Hydration (sublinear scaling):

Application size	Index size	Hydration tokens	Synthesis tokens	Total	vs. static
Small (< 50 artifacts)	~1KB	0 (auto-include)	~2,500	~2,500	Same
Medium (200 artifacts)	~3KB	~3,000	~6,000	~9,000	**64% reduction**
Large (2,000 artifacts)	~8KB	~5,000	~8,000	~13,000	**95% reduction**
Very large (10,000 artifacts)	~15KB	~7,000	~10,000	~17,000	**99% reduction**
Any size	Bounded	Bounded	Bounded	Bounded	**No overflow**

Token consumption: O(log N) approaching a constant bound set by MAXTOTALCHARS

4.8.2 Why Sublinear Scaling Is Achievable

Sublinear scaling is achievable because of three properties:

1. Index compression: The entity index represents the entire application structure in 1-5% of the source token count. The LLM can comprehend a 10,000-artifact application from a 15KB index (~3,750 tokens) rather than a 5MB source (~1,250,000 tokens).

2. Selective retrieval: For any specific generation task, the LLM typically needs only 5-15% of the source artifacts. The entity index enables the LLM to identify and retrieve exactly the relevant artifacts without loading the rest.

3. Context budget enforcement: The budget caps total accumulated context regardless of source size. A 10x larger application does not produce a 10x larger context -- the LLM simply makes more selective retrieval choices within the same budget.

4.8.3 Scaling Implications

The sublinear scaling property has several important implications:

- No application is too large: Applications with 10,000+ artifacts can be processed with bounded token consumption, where static prompting would fail entirely

- Predictable cost: Token consumption is bounded by the context budget, enabling accurate cost prediction regardless of application size
- Constant-quality output: Because the LLM selectively retrieves the most relevant artifacts (rather than receiving diluted attention across a massive context), output quality is maintained or improved as application size grows
- Energy efficiency: Sublinear token scaling translates directly to sublinear energy consumption, reducing the environmental impact of processing large applications

4.9 Integration with Translation Recipes (Patent 3 Enhancement)

Progressive Context Hydration composes with the Translation Recipe system (Patent 3):

Prior approach (Patent 3):

- Recipe loaded -> full repository context loaded (file tree + limited files) -> single LLM call

Enhanced approach:

- Recipe loaded -> hydration loop initializes with recipe instructions injected into system prompt -> LLM retrieves exactly the files relevant to the recipe's output requirements -> synthesis step produces output

The recipe instructions are injected into the system prompt of the hydration loop, causing the LLM to make recipe-informed retrieval decisions:

```
extra_system = translation.instructions  # injected into hydration system prompt
```

This produces higher-quality recipe output because the LLM retrieves the exact source sections the recipe needs, rather than receiving an arbitrary fixed set of files.

Updated token efficiency (with hydration):

Step	Tokens (Patent 3 alone)	Tokens (with PCH)	Reduction
Recipe application (per genome)	~12,000	~8,000	33%
Portfolio of 100 genomes	~1,659,000	~1,107,000	33% additional

Combined with Patent 3's recipe amortization, the system achieves an 83-87% total token reduction compared to unguided interactive transformation.

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented method for LLM-directed iterative context hydration comprising:

(a) generating a lightweight entity index at source commit time, the index mapping application entities to their file paths with summaries but without file content;

(b) initializing a hydration loop by presenting an LLM with the entity index and a structured retrieval protocol defining two response shapes: a retrieval request shape comprising a retrieval plan and a list of requested files, and a ready signal shape comprising a readiness declaration with reason;

(c) receiving from the LLM a structured retrieval request identifying specific files and an explicit retrieval plan articulating the LLM's retrieval strategy;

(d) validating the LLM response via a programmatic guard layer that rejects generation output or readiness signals if no source files have been fetched, injecting a correction message into the LLM conversation and retrying;

(e) fetching the requested files from a source system and appending their contents to the LLM conversation context, with deduplication to prevent redundant retrieval of already-loaded files;

(f) repeating steps (c)-(e) across multiple retrieval rounds until the LLM signals readiness, a maximum round limit is reached, or a context budget is exhausted;

(g) packaging accumulated context into a HydrationContext object comprising loaded files, full conversation history, accumulated retrieval plan, and token usage statistics;

(h) passing the HydrationContext to a downstream synthesis step that continues the same LLM conversation with the accumulated working memory intact.

Claim 2. A system for LLM-directed retrieval with enforced minimum context acquisition comprising:

(a) an index generator configured to produce a deterministic entity index mapping application entities to file paths with summaries at commit time;

(b) a hydration loop controller configured to manage multi-turn LLM dialogue for iterative file retrieval with explicit retrieval planning per round;

(c) a programmatic guard layer configured to intercept every LLM response and mechanically reject responses that violate the minimum-retrieval contract, injecting correction messages and retrying, wherein the guard operates at the system level outside the LLM conversation and cannot be bypassed by prompt injection, instruction overriding, or LLM misconfiguration;

(d) a context budget enforcer configured to track accumulated context size and force ready transitions when the budget is exceeded;

(e) a source-agnostic retrieval abstraction configured to apply the same loop protocol to file repositories, structured record sets, and document sections through source-specific adapters sharing common control logic;

(f) a HydrationContext accumulator configured to package loaded context, conversation history, and token usage for downstream synthesis.

Claim 3. A method for source-agnostic iterative context retrieval comprising:

(a) abstracting a retrieval loop over multiple source types including: version-controlled file repositories, structured record collections, and parsed document sections;

(b) for each source type, implementing a source-specific index builder and content fetcher while sharing loop control logic, guard validation, context budget enforcement, deduplication, and context accumulation;

(c) thereby enabling the same LLM-directed retrieval protocol to apply to transformation, translation, and extraction operations without modification to the loop controller.

Claim 4. A method for programmatic enforcement of minimum context acquisition in LLM-mediated generation comprising:

(a) maintaining a stateful guard object tracking files fetched per retrieval round and cumulatively across all rounds;

(b) intercepting every LLM response before the system acts on it;

(c) detecting premature generation defined as: generation output present AND cumulative files fetched equals zero;

(d) detecting premature readiness defined as: ready signal present AND cumulative files fetched equals zero;

(e) rejecting detected premature responses by injecting type-specific correction messages into the LLM conversation and initiating a retry;

(f) accepting the response after a configurable maximum number of retries per round to ensure loop termination;

(g) wherein the guard operates as a programmatic enforcement layer outside the LLM's conversational context and cannot be bypassed by prompt content, prompt injection, or LLM instruction following.

Claim 5. A computer-implemented method for LLM-mediated generation wherein the LLM must retrieve context from a source system before generating output, comprising:

(a) presenting an LLM with a structured index of available source material and a retrieval protocol, wherein the index describes the source material's structure and content without including the source content itself;

(b) requiring the LLM to issue one or more structured retrieval requests identifying specific source artifacts to load, each request accompanied by an explicit retrieval plan;

(c) programmatically enforcing that the LLM retrieves at least a minimum quantity of source material before the system accepts any generation output, wherein enforcement is performed by a guard layer that mechanically intercepts and rejects premature generation attempts;

(d) accumulating retrieved source material across multiple retrieval rounds with deduplication and budget enforcement;

(e) upon the LLM signaling readiness or a termination condition being met, synthesizing output using only the accumulated retrieved context;

(f) wherein the total source material retrieved is a subset of the available source material, selected by the LLM based on

relevance to the generation task, and the subset size is bounded by a configurable context budget independent of total source size.

Dependent Claims

Claim 6. The method of Claim 1 wherein the entity index is generated deterministically by rule-based extraction from source structure without LLM inference, ensuring identical inputs produce identical indexes.

Claim 7. The method of Claim 1 wherein the LLM retrieval request includes a plan field comprising one or more natural language statements of the LLM's retrieval strategy, and wherein the plan is surfaced in the user interface as a real-time progress indicator, and wherein the plan evolves across rounds as the LLM discovers dependencies in retrieved content.

Claim 8. The method of Claim 1 wherein fetched file contents are deduplicated by path, preventing redundant retrieval of already-loaded files across rounds, and wherein the LLM is informed of already-loaded paths to prevent redundant requests.

Claim 9. The method of Claim 1 wherein the system enforces a context budget expressed as maximum accumulated characters, injecting a budget-exceeded directive into the conversation when the budget is reached, forcing the LLM to produce output with available context regardless of whether the LLM has signaled readiness.

Claim 10. The system of Claim 2 wherein the source-agnostic retrieval abstraction for structured record sources implements an auto-include pass that loads high-priority record types within a reserved portion of the context budget before the LLM-directed loop begins.

Claim 11. The system of Claim 2 wherein for structured record sources, the loop is bypassed entirely for small record sets below a configurable threshold, applying direct auto-include without LLM overhead.

Claim 12. The method of Claim 1 wherein the HydrationContext includes the full multi-turn conversation history, enabling the downstream synthesis step to continue the same LLM conversation without re-initializing context or re-processing previously loaded files.

Claim 13. The method of Claim 1 wherein the hydration loop is composed with a translation recipe system such that recipe instructions are injected into the hydration system prompt, causing the LLM to selectively retrieve only the source artifacts relevant to the recipe's required output.

Claim 14. The method of Claim 4 wherein the guard maintains per-round retry counts and the maximum retries is configurable, with the guard accepting any response after the maximum is reached to ensure loop termination, and wherein each rejection injects a type-specific correction message that becomes part of the ongoing conversation.

Claim 15. The system of Claim 2 wherein the entity index maps application entities into canonical categories each with an entity identifier, display name, and repository file path, and wherein the index includes an explicit relationship graph mapping entity-to-entity dependencies to enable multi-step retrieval planning.

Claim 16. The method of Claim 1 wherein token consumption is sublinear relative to source application size: processing an application 10x larger requires no more than 2x additional tokens due to context budget enforcement and selective LLM-directed retrieval.

Claim 17. The method of Claim 1 wherein for transformation operations, the hydration loop retrieves files from a version-controlled repository using: (i) repository tree listing as the initial index, (ii) per-file content fetching via authenticated API, and (iii) path normalization to handle relative vs. absolute path formats.

Claim 18. The method of Claim 3 wherein document section hydration parses source documents into sections indexed by title, page range, and word count, enabling the LLM to request specific sections without receiving the full document.

Claim 19. The system of Claim 2 wherein the guard validator distinguishes between three premature response types: (i) premature generation with filesystem plan or specification output, (ii) premature readiness signal without prior retrieval, and (iii) empty retrieval request without readiness signal, applying a specific correction message to each type.

Claim 20. The method of Claim 1 further comprising streaming server-sent events (SSE) to a client interface reporting: retrieval phases, files fetched with sizes, LLM plan updates, context budget utilization, and round completion.

Claim 21. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

Claim 22. The method of Claim 5 wherein the LLM formulates a multi-round retrieval plan during the first retrieval round, wherein the plan identifies primary artifacts to retrieve immediately and secondary artifacts to retrieve after examining the primary artifacts, and wherein the plan is revised in subsequent rounds based on dependencies discovered in retrieved content.

Claim 23. The method of Claim 5 wherein the context budget comprises a maximum total character count independent of total source size, and wherein the LLM's retrieval selections within the budget are informed by the structured index such that the LLM retrieves only the artifacts most relevant to the generation task, and wherein the bounded budget combined with index-informed retrieval produces sublinear token scaling relative to source size.

Claim 24. The method of Claim 5 wherein the source system is abstracted through a retrieval interface that exposes: (i) an index listing operation returning available source artifacts with metadata and summaries, (ii) a content fetching operation returning the full content of a specific artifact by identifier, and (iii) a deduplication check preventing retrieval of already-loaded artifacts, and wherein the retrieval interface is source-agnostic such that the LLM-directed retrieval protocol operates identically regardless of whether the source is a file repository, structured record set, or document collection.

---

6. Advantages Over Prior Art

vs. Static Prompting

1. No context window overflow: Context budget enforcement prevents any application size from exceeding model limits
2. Proportional attention: LLM only sees relevant files; attention is not diluted by irrelevant artifacts
3. Predictable token cost: Cost is bounded by context budget, not application size
4. Sublinear scaling: 10x larger application costs ~1.5x more tokens, not 10x
5. Adaptive retrieval: LLM can discover and request dependencies mid-process; static prompting cannot

vs. Naive RAG

1. LLM drives retrieval: The LLM determines what it needs, not a semantic similarity function -- critical for structured tasks where dependencies are logical, not semantic
2. Multi-turn adaptive retrieval: Each round can request different files based on what the prior round revealed
3. Minimum-retrieval enforcement: Guard prevents generation before retrieval; RAG has no such contract
4. Structured protocol: JSON shapes with plan/required_files/ready fields provide a typed interface; RAG uses unstructured similarity scores
5. Retrieval planning: LLM explicitly articulates and revises its retrieval strategy; RAG provides no planning mechanism
6. Dependency-aware: LLM follows structural dependencies ("the form references this workflow, which references this table"); RAG retrieves by semantic similarity

vs. General Agentic Tool Calling

1. Domain-specific index: Entity index provides application-aware entity mapping, not generic file system access
2. Guard enforcement: Programmatic minimum-context contract has no equivalent in general tool-use frameworks
3. Context budget: Bounded accumulation prevents unbounded retrieval; tool calling has no budget concept
4. Source-agnostic abstraction: Same protocol for file repos, record sets, documents -- tool calling requires custom tool implementations per source

5. Composable with recipes: Hydration loop integrates as a pre-step to recipe execution, amplifying token savings of both systems
6. Structured handoff: HydrationContext provides a typed interface between retrieval and synthesis; tool calling has no equivalent
7. Deduplication: Path-based deduplication prevents redundant retrieval; tool calling does not track what has been fetched

---

7. Relationship to Prior OverYonder Patents

Patent 1 Modifications (Genome Extraction)

The entity index artifact described in Section 4.2 should be added to Patent 1's claims as:

- A new artifact type committed alongside genome.yaml and graph.yaml
- A new dependent claim covering deterministic index generation
- An addition to the repository structure diagrams

The structured record hydration described in Section 4.6.2 should be added to Patent 1 as:

- A new pipeline stage inserted between parsing and genome extraction
- A dependent claim covering threshold-based loop activation

Patent 3 Modifications (Translation Recipes)

Patent 3 currently states "full repository context" is loaded for recipe application. This should be updated to reference the hydration loop as the context loading mechanism:

- Replace fixed file loading with "initiates hydration loop with recipe instructions injected as system context"
- Add updated token efficiency numbers incorporating hydration savings (Section 4.9)
- Add a new claim covering recipe-hydration composition

Patent 5 Modifications (Application Genome Modeling)

The entity index format described in Section 4.2 should be referenced in Patent 5 as an additional genome representation format alongside flat and graph schemas.

---

8. Drawings/Figures Description

Figure 1: System Architecture -- Block diagram showing the hydration loop stages (Index Init -> LLM Call -> Guard Enforcement -> Retrieval -> Context Accumulation -> Budget Check -> Ready Transition) with the downstream synthesis step and source-agnostic retrieval layer.

Figure 2: Comparison with Prior Art -- Four-column diagram comparing Static Prompting, RAG, Agentic Tool Calling, and Progressive Context Hydration across dimensions: control flow, retrieval mechanism, enforcement, and token scaling.

Figure 3: Guard Validation State Machine -- State diagram showing guard states (waiting, rejected-premature-gen, rejected-premature-ready, rejected-empty, accepted, max-retries) with transitions for each rejection condition and correction message injection.

Figure 4: Source-Agnostic Abstraction -- Three-column diagram showing File Repository, Structured Records, and Document sources with shared loop controller, guard, budget enforcement, and source-specific index/fetch implementations.

Figure 5: Entity Index Structure -- Entity-relationship diagram of the entity index with entities, relationships, summaries, and metadata sections, annotated with token size comparison to full source.

Figure 6: Round-by-Round Context Growth -- Chart showing context accumulated per round for a sample application, with entity index, round-1 retrieval, round-2 dependency discovery, round-3 retrieval, and ready signal annotated.

Figure 7: Sublinear Token Scaling -- Graph showing token consumption vs. application size for Static Prompting (linear O(N)), RAG (fixed K), Tool Calling (unbounded), and PCH (sublinear O(log N) with budget ceiling), with the PCH budget ceiling annotated.

Figure 8: Token Efficiency Comparison -- Bar chart comparing single-pass, naive RAG, and Progressive Context Hydration token consumption for small/medium/large/very large applications.

Figure 9: HydrationContext Data Flow -- Sequence diagram showing how HydrationContext is produced by the loop and consumed by downstream synthesis and recipe execution, with conversation history preservation annotated.

Figure 10: Guard Enforcement Flow -- Detailed sequence diagram showing a premature generation attempt being intercepted, rejected, corrected, and retried by the guard layer.

Figure 11: Record Type Index -- Sample formatted index showing record types, counts, sample names, auto-included vs. available sections.

Figure 12: Integration with Translation Recipe System -- Diagram showing how recipe instructions are injected into the hydration system prompt, how hydration retrieves recipe-relevant files, and how hydration output feeds the synthesis step.

Figure 13: Context Budget Enforcement -- Diagram showing budget tracking across rounds, with budget-exceeded directive

injection and forced ready transition.

Figure 14: Multi-Round Retrieval Planning -- Diagram showing the LLM's retrieval plan evolving across three rounds: initial plan based on index -> revised plan after discovering dependencies -> final plan with all required context identified.

Figure 15: Sublinear Scaling Proof -- Mathematical derivation showing that token consumption is bounded by indextokens + MAXTOTALCHARSTOKENS + synthesis_tokens, all of which are independent of source size N, producing O(1) scaling with respect to N (sublinear).

---

9. Inventors

[To be completed by filing attorney]

10. Filing Notes

- Strongest novel claim: The programmatic guard enforcement layer (Claims 4 and 5) is the most mechanically novel element -- no prior art enforces minimum-retrieval as a programmatic contract with retry injection that operates outside the LLM's conversational context. This is the primary differentiator from RAG, ReAct, and tool calling.
- Broadest claim: Claim 5 covers any LLM system that must retrieve context before generating output, with programmatic enforcement, structured index, and budget-bounded selective retrieval. This is deliberately broad and not limited to enterprise applications or genome-specific use cases.
- Key differentiation from RAG: RAG is system-directed retrieval (the retrieval system selects context). PCH is LLM-directed retrieval (the LLM selects context). RAG is single-pass. PCH is multi-round adaptive. RAG has no minimum-retrieval enforcement. PCH has a programmatic guard. These are fundamental architectural differences, not incremental improvements.
- Key differentiation from ReAct/tool calling: Tool calling provides a mechanism (the LLM can invoke functions). PCH provides structure: typed protocol, entity index, guard enforcement, budget management, deduplication, and synthesis handoff. Tool calling has no minimum-retrieval contract, no budget, no deduplication, and no structured handoff.
- Sublinear scaling claim: The O(log N) token scaling property (Section 4.8) is mathematically demonstrable from the budget enforcement mechanism. Validate with production data across application sizes before filing Claim 16.
- File as continuation: Patent 3 (Translation Recipes) and Patent 5 (Application Genome

Modeling) should be cited as prior art from the same family; file as a continuation-in-part.

- Prior art risk areas: ReAct (Yao et al., 2022), iterative RAG (FLARE, Self-RAG), LLM function calling (OpenAI, Anthropic), agentic frameworks (LangChain, AutoGen) -- claims must be differentiated on: programmatic guard enforcement, structured retrieval protocol with planning, context budget enforcement, source-agnostic abstraction, and sublinear scaling.
- International filing: PCT filing recommended for global enterprise market.
- Continuation opportunity: Document section hydration (Claim 18) may warrant a separate continuation once fully implemented and production-validated.

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Multi-Stage Enterprise Application Transformation Orchestration

Title

System and Method for Orchestrating Multi-Stage Enterprise Application Transformation Using Agent-Based Execution, Deterministic-AI Hybrid Pipelines, and Skill-Chained Workflow Graphs

1. Field of Invention

This invention relates to enterprise application transformation orchestration, hybrid deterministic-AI pipeline execution, multi-system coordination, and computational workflow management. Specifically, it describes a system and method for orchestrating the end-to-end lifecycle of enterprise application transformation -- encompassing extraction, normalization, enrichment, transformation, validation, and deployment -- through a control plane that coordinates deterministic services (parsers, normalizers, validators, schema mappers) and AI agents (large language models) within a directed acyclic execution graph, with dynamic skill selection, real-time event streaming, observability, and fault-tolerant retry and fallback mechanisms.

The system serves as the control plane for enterprise application portability -- the central orchestration layer that coordinates all upstream extraction systems, transformation engines, validation agents, and downstream deployment targets into a unified, observable, and fault-tolerant pipeline.

2. Background and Prior Art Gaps

2.1 The Orchestration Problem in Enterprise Application Transformation

Enterprise application transformation -- migrating applications from one platform to another, modernizing legacy systems, or extracting portable representations from vendor-locked environments -- involves a complex sequence of interdependent operations:

1. Extraction: Ingesting application artifacts from source systems (configuration exports, API specifications, video demonstrations, documentation) and producing structured

representations

2. Normalization: Converting source-specific formats into canonical, platform-neutral representations (application genomes)
3. Enrichment: Deepening the canonical representation through behavioral pattern extraction, dependency analysis, and cross-reference resolution
4. Transformation: Converting the canonical representation into target-platform-specific implementations using translation recipes, code generation, or AI-mediated synthesis
5. Validation: Assessing the completeness, correctness, and fidelity of the transformation output
6. Deployment: Committing transformation outputs to target repositories, platforms, or environments

Each of these operations may involve deterministic processing (XML parsing, schema validation, file system operations), AI-mediated processing (LLM-driven extraction, pattern recognition, code generation), or both. The operations have complex dependencies: enrichment requires extraction to complete; transformation requires enrichment; validation requires transformation; some operations can execute in parallel while others must execute sequentially.

2.2 Current Approaches and Their Limitations

Manual orchestration: Developers manually invoke each transformation step, monitor progress, handle errors, and coordinate between systems. This is slow, error-prone, and does not scale to portfolio-level migrations.

Script-based pipelines: Shell scripts or CI/CD pipelines execute transformation steps sequentially. These are brittle (no error recovery), opaque (no observability), and cannot dynamically adapt to different application types or transformation requirements.

General-purpose workflow engines (Airflow, Temporal, Step Functions): These provide DAG execution and retry logic but are not designed for hybrid deterministic-AI workloads. They lack:

- AI-specific execution semantics (token budgets, context management, model selection)
- Skill-based composition (dynamically selecting the right tool for each step)
- Domain-specific observability (extraction completeness, transformation fidelity, genome coverage metrics)
- Feedback loops that route AI output back through validation and correction cycles

LLM agent frameworks (LangChain, AutoGen, CrewAI): These provide multi-agent AI orchestration but are not designed for enterprise transformation pipelines. They lack:

- Deterministic execution stages (parsing, validation, file operations must be reliable and

reproducible)

- Multi-system coordination (extracting from one platform, transforming, deploying to another)
- Production-grade fault tolerance (retry with exponential backoff, fallback to alternative models or strategies, circuit breakers)
- Portfolio-level execution (running the same pipeline across hundreds of applications with aggregated observability)

2.3 Gaps in Prior Art

1. No existing system provides an orchestration control plane purpose-built for enterprise application transformation that coordinates both deterministic services and AI agents within a unified execution framework.

2. No existing system implements a skill registry that enables dynamic selection and composition of transformation capabilities (extractors, normalizers, enrichers, transformers, validators, deployers) based on source type, target platform, and application characteristics.

3. No existing system executes enterprise transformation workflows as a directed acyclic execution graph (DAG) with typed edges representing data dependencies, conditional branching based on intermediate results, and dynamic subgraph expansion.

4. No existing system combines deterministic processing stages (parsers, schema validators, file system operations) with AI agent stages (LLM-mediated extraction, pattern recognition, code generation) in a single coordinated pipeline with unified observability and error handling.

5. No existing system provides real-time event streaming of transformation pipeline progress with domain-specific events (extraction phases, files fetched, patterns discovered, transformation completeness, validation scores) consumable by client interfaces and monitoring systems.

6. No existing system implements feedback loops within the transformation pipeline that route validation results back to transformation stages for automated correction and re-execution.

7. No existing system provides portfolio-level orchestration that executes the same transformation pipeline across multiple applications with aggregated metrics, progress tracking, and resource optimization.

8. No existing system implements fault-tolerant hybrid execution with AI-specific retry

semantics (model fallback, prompt modification, context reduction) alongside deterministic retry semantics (exponential backoff, idempotent re-execution).

---

3. Summary of Invention

The present invention provides an orchestration control plane for enterprise application portability -- a system that coordinates the end-to-end lifecycle of enterprise application transformation through a hybrid deterministic-AI pipeline with skill-chained workflow execution.

The system comprises:

3.1 Orchestration Engine

The central coordinator that manages pipeline execution. The engine:

- Accepts transformation requests specifying source system, target platform, and transformation parameters
- Resolves the appropriate execution graph from the skill registry based on source type, target platform, and application characteristics
- Executes the graph by dispatching each node to the appropriate skill (deterministic service or AI agent)
- Manages data flow between nodes, passing outputs of upstream nodes as inputs to downstream nodes
- Handles errors through configurable retry and fallback strategies
- Streams real-time execution events to client interfaces and monitoring systems
- Tracks resource consumption (tokens, API calls, compute time) across all pipeline stages

3.2 Skill Registry

A catalog of composable transformation capabilities registered with the orchestration engine. Each skill is a self-describing unit of work with:

- Skill identifier: Unique name and version
- Skill type: Deterministic (parser, normalizer, validator, deployer) or AI (LLM-mediated extractor, enricher, transformer, generator)

- Input schema: Typed specification of required inputs
- Output schema: Typed specification of produced outputs
- Execution requirements: Resource needs (model type, token budget, API credentials, compute tier)
- Compatibility metadata: Source types, target platforms, and application categories the skill supports
- Quality metrics: Historical performance data (success rate, average latency, output quality scores)

Skills are registered declaratively and can be added, updated, or deprecated without modifying the orchestration engine. The registry enables:

- Dynamic skill selection: The engine selects the best available skill for each pipeline stage based on the specific transformation context
- Skill chaining: Skills are composed into multi-step workflows where the output of one skill feeds the input of the next
- Skill versioning: Multiple versions of a skill can coexist, enabling gradual rollout and A/B comparison
- Skill discovery: New skills can be discovered and integrated without pipeline reconfiguration

3.3 Execution Graph (DAG)

Transformation pipelines are represented as directed acyclic graphs where:

- Nodes represent individual transformation operations (extraction, normalization, enrichment, transformation, validation, deployment), each bound to a skill from the registry
- Edges represent typed data dependencies between operations, specifying which output fields of an upstream node feed which input fields of a downstream node
- Conditional branches enable dynamic routing based on intermediate results (e.g., if extraction completeness score is below threshold, route to a supplementary extraction stage rather than proceeding to transformation)
- Parallel lanes enable concurrent execution of independent operations (e.g., extracting from multiple source exports simultaneously)
- Subgraph expansion: A single node can expand into a subgraph at execution time based on runtime conditions (e.g., a "multi-export extraction" node expands into N parallel extraction nodes based on the number of exports provided)
- Feedback edges: Validation outputs can be routed back to transformation stages through

feedback edges that trigger re-execution with corrective context

Example execution graph for a standard transformation pipeline:

```
[Source Artifact Ingestion]
       |
       +-- [Config Export Parser] ---- (deterministic)
       +-- [API Spec Parser] -------- (deterministic)
       +-- [Video Analyzer] ---------- (AI agent)
              |
       [Genome Extraction] ----------- (AI agent)
              |
       [Genome Merger] --------------- (AI agent, conditional: multi-source only)
              |
       [Deep Analysis / Enrichment] -- (AI agent)
              |
       [Genome Validation] ------------ (AI agent)
              |
         +----+----+
    [score >= 70]  [score < 70]
         |              |
         |         [Supplementary Extraction] -> [Re-validation]
         |              |
         +------+-------+
                |
       [Translation Recipe Application] -- (AI agent + hydration)
                |
       [Output Validation] ------------ (deterministic + AI)
                |
         +------+------+
    [valid]        [invalid]
         |              |
         |         [Correction Loop] -> [Re-validation]
         |              |
         +------+-------+
                |
       [Repository Commit / Deploy] -- (deterministic)
                |
       [Portfolio Metrics Update] ---- (deterministic)
```

3.4 Event Streaming Layer

The system provides real-time event streaming throughout pipeline execution:

- Pipeline-level events: Pipeline started, pipeline completed, pipeline failed, pipeline paused/resumed
- Stage-level events: Stage started, stage completed, stage failed, stage retrying, stage skipped
- Domain-specific events: Files fetched (with paths and sizes), patterns discovered, extraction

completeness scores, transformation file counts, validation scores, context budget utilization

- Resource events: Token consumption per stage, API call counts, compute time, estimated cost

Events are streamed via Server-Sent Events (SSE) to client interfaces and are simultaneously published to a persistent event log for monitoring, alerting, and post-execution analysis.

Event consumers include:

- Client UI: Real-time progress indicators, stage-by-stage status, file-level detail
- Monitoring systems: Pipeline health dashboards, SLA tracking, anomaly detection
- Analytics: Historical performance analysis, resource optimization, skill quality tracking
- Feedback loops: Downstream stages can subscribe to upstream events to make adaptive decisions

3.5 Observability and Metrics

The system provides multi-level observability:

Per-stage metrics:

- Execution duration and latency breakdown
- Token consumption (input, output, total) for AI stages
- Error counts and retry counts
- Output quality scores (extraction completeness, transformation fidelity, validation pass/fail)
- Resource utilization (context budget, API rate limits)

Per-pipeline metrics:

- End-to-end duration
- Total token consumption across all AI stages
- Total cost (compute + API)
- Stage success/failure/retry counts
- Quality scores at each validation checkpoint
- Feedback loop iteration counts

Portfolio-level metrics:

- Applications processed (total, in-progress, completed, failed)

- Aggregate token consumption and cost
- Average transformation quality scores
- Recipe reuse rates and efficiency gains
- Pipeline throughput (applications per hour/day)
- Skill utilization and performance comparison

Observability infrastructure:

- Structured logging with correlation IDs linking all events within a pipeline execution
- Distributed tracing across multi-system operations (source extraction -> GitHub commit -> deployment)
- Metric aggregation with configurable retention periods
- Alerting on quality score degradation, error rate spikes, and resource consumption anomalies

3.6 Retry and Fallback Mechanisms

The system implements fault-tolerant execution with strategies differentiated by skill type:

Deterministic skill retry:

- Exponential backoff with configurable base interval and maximum retries
- Idempotent re-execution (same inputs always produce same outputs)
- Circuit breaker pattern: after N consecutive failures of a deterministic skill, the pipeline pauses and alerts rather than continuing to retry
- Transient error detection: distinguish between retryable errors (network timeout, rate limit) and permanent errors (invalid input, permission denied)

AI agent retry:

- Model fallback: If the primary LLM fails or produces low-quality output, the system falls back to an alternative model (e.g., primary model -> fallback model -> smaller/faster model)
- Prompt modification: On retry, the system can modify the prompt strategy (e.g., simplify instructions, reduce context, add examples)
- Context reduction: If the AI stage fails due to context length, the system reduces context (e.g., fewer files, shorter summaries) and retries
- Temperature adjustment: On retry, the system can adjust generation temperature to produce more or less deterministic output
- Partial result recovery: If an AI stage produces partial output before failing, the system can

extract and preserve the partial result for manual review or supplementary processing

Pipeline-level fallback:

- Alternative path execution: If a primary transformation path fails, the pipeline can route to an alternative skill chain (e.g., if AI-mediated transformation fails, fall back to template-based transformation)
- Graceful degradation: If a non-critical stage fails (e.g., enrichment), the pipeline can continue with reduced output quality rather than failing entirely, with the degradation noted in the output quality report
- Human-in-the-loop escalation: For failures that cannot be resolved automatically, the pipeline pauses and creates a review task with full context (inputs, error details, partial results) for human intervention

---

4. Detailed Description

4.1 Orchestration Engine Architecture

The orchestration engine is the central coordinator of the transformation pipeline. It operates as follows:

4.1.1 Request Processing

When a transformation request arrives, the engine:

1. Validates the request: Confirms that source artifacts are accessible, target platform is supported, and required credentials are available
2. Resolves the execution graph: Queries the skill registry to determine the appropriate skill chain for the given source type, target platform, and transformation parameters. The graph may be resolved from:
   - A predefined pipeline template (e.g., "standard extraction + transformation + deployment")
   - A dynamically composed graph based on source analysis (e.g., detecting that the source contains video artifacts triggers inclusion of the video extraction skill)
   - A user-specified custom graph with manual skill selection
3. Allocates resources: Reserves token budgets for AI stages, establishes rate limit quotas for external API calls, and assigns compute resources for deterministic stages

4. Initializes execution state: Creates a persistent execution record with unique pipeline ID, stage status tracking, event log, and metric accumulators

4.1.2 Graph Execution

The engine executes the graph using a topological traversal:

1. Stage dispatch: For each node whose dependencies are satisfied, the engine dispatches execution to the bound skill:
   - Deterministic skills are executed directly (function call, service invocation)
   - AI agent skills are executed through the LLM interface with context management (including progressive context hydration when applicable)
2. Data routing: The engine routes output data from completed stages to downstream stages according to the typed edge specifications in the graph
3. Conditional evaluation: At conditional branch points, the engine evaluates branch conditions against intermediate results and routes execution accordingly
4. Parallel execution: Independent stages (nodes with no mutual dependencies) are dispatched concurrently, with the engine managing concurrency limits and resource contention
5. Subgraph expansion: When a stage is marked for dynamic expansion, the engine generates the subgraph at runtime and integrates it into the execution plan
6. Feedback loop execution: When a validation stage produces a below-threshold result, the engine routes the result back to the appropriate transformation stage with corrective context, re-executes the transformation, and re-validates

4.1.3 State Management

The engine maintains persistent execution state:

- Stage states: pending, running, completed, failed, retrying, skipped, waiting-for-feedback
- Data artifacts: Intermediate outputs stored at each stage for auditability and restart capability
- Checkpoint mechanism: The engine periodically checkpoints execution state, enabling restart from the last successful stage after a system failure
- Concurrent pipeline management: Multiple pipelines can execute concurrently with resource isolation and priority-based scheduling

4.2 Skill Registry Architecture

4.2.1 Skill Definition

Skills are registered with the following specification:

```
skill:
  id: "genome-extraction-v2"
  version: "2.1.0"
  type: "ai_agent"

  description: "Extracts application genome from configuration exports using LLM analysis"

  input_schema:
    required:
      - name: "parsed_records"
        type: "structured_record_set"
        description: "Parsed configuration records from source system"
    optional:
      - name: "hydration_context"
        type: "HydrationContext"
        description: "Pre-loaded context from progressive hydration"

  output_schema:
    - name: "genome"
      type: "application_genome"
      description: "Extracted application genome in canonical format"
    - name: "completeness_score"
      type: "float"
      range: [0, 100]
      description: "Extraction completeness assessment"

  execution:
    model: "claude-sonnet"
    fallback_model: "gpt-4o"
    token_budget: 50000
    max_retries: 2
    timeout_seconds: 120
    supports_hydration: true

  compatibility:
    source_types: ["servicenow_export", "salesforce_metadata", "generic_xml"]
    target_platforms: ["*"]
    min_record_count: 1
    max_record_count: 10000

  quality:
    avg_completeness_score: 82.4
    success_rate: 0.96
    avg_latency_seconds: 45
    total_executions: 1247
```

4.2.2 Skill Selection Algorithm

When the engine needs to select a skill for a pipeline stage, it queries the registry with:

- Required skill type (extractor, normalizer, enricher, transformer, validator, deployer)
- Source type and target platform
- Application characteristics (size, complexity, artifact types)

The registry returns a ranked list of compatible skills, ordered by:

1. Compatibility score (how well the skill matches the specific source/target pair)
2. Quality metrics (historical success rate, average quality scores)
3. Resource efficiency (token consumption per quality unit)
4. Availability (current load, rate limit headroom)

The engine selects the top-ranked skill, with the second-ranked skill designated as the fallback.

4.2.3 Skill Chaining

Skills are composed into chains where the output schema of one skill satisfies the input schema of the next. The registry validates chain compatibility at graph resolution time:

```
[Config Parser]          -> structured_record_set
[Genome Extractor]       <- structured_record_set -> application_genome
[Deep Analyzer]          <- application_genome -> enriched_genome
[Translation Applicator] <- enriched_genome + translation_recipe -> filesystem_plan
[Output Validator]       <- filesystem_plan -> validation_report
[Repository Deployer]    <- filesystem_plan + validation_report -> deployment_result
```

Type mismatches between chained skills are detected at graph resolution time (before execution begins), preventing runtime failures from schema incompatibility.

4.3 Hybrid Deterministic-AI Pipeline Execution

The system's core architectural innovation is the unified execution of deterministic services and AI agents within a single pipeline, with execution semantics appropriate to each:

4.3.1 Deterministic Stage Execution

Deterministic stages are operations that produce identical outputs for identical inputs:

- Configuration parsers: XML, JSON, YAML parsing with schema validation

- Schema normalizers: Converting source-specific schemas to canonical formats
- File system operations: Reading, writing, and organizing files in repositories
- Validation checkers: Schema compliance, completeness scoring arithmetic, boundary audit checks
- Deployment operations: Repository commits, branch creation, file uploads

Deterministic stages execute with:

- Guaranteed reproducibility: Same inputs always produce same outputs
- Fast execution: No LLM inference latency
- Predictable resource consumption: Compute and memory requirements are deterministic
- Simple retry: Failed executions can be retried with identical parameters

4.3.2 AI Agent Stage Execution

AI agent stages are operations that use LLM inference for semantic analysis, pattern recognition, and generation:

- Genome extraction: Analyzing configuration records to extract business logic
- Behavioral enrichment: Discovering reusable patterns and process flows
- Translation application: Applying transformation recipes to produce target-platform output
- Quality assessment: Evaluating extraction completeness and transformation fidelity
- Context hydration: LLM-directed iterative context acquisition

AI agent stages execute with:

- Context management: Token budgets, context windows, progressive hydration
- Model selection: Primary and fallback models per stage
- Non-deterministic output: Results may vary across executions; quality scoring validates acceptability
- Structured output parsing: AI output is parsed into typed data structures for downstream consumption
- Guard enforcement: For stages requiring context acquisition, programmatic guards prevent premature generation

4.3.3 Hybrid Coordination

The orchestration engine coordinates deterministic and AI stages through:

- Deterministic-first sequencing: Where possible, deterministic stages execute first to provide structured, validated input to AI stages. This improves AI stage quality by ensuring the AI receives clean, well-structured data.
- AI-output validation: AI stage outputs pass through deterministic validation before being routed to downstream stages. This catches malformed output, schema violations, and quality threshold failures before they propagate.
- Adaptive pipeline modification: Based on AI stage results, the engine can dynamically modify the remaining pipeline (e.g., if AI extraction identifies additional source artifacts, the engine adds extraction stages for those artifacts).
- Resource balancing: The engine balances resource allocation between deterministic and AI stages, reserving token budgets for AI stages while allowing deterministic stages to consume compute resources freely.

4.4 Multi-System Coordination

The orchestration engine coordinates operations across multiple external systems:

4.4.1 Source System Adapters

The engine interfaces with source systems through typed adapters:

- Configuration-driven platforms: Adapters for ingesting configuration exports, metadata packages, and update sets from enterprise platforms
- API-based systems: Adapters for querying API specifications, schema definitions, and endpoint inventories
- File-based sources: Adapters for reading documents, video files, and process diagrams from file storage systems
- Version control systems: Adapters for reading source code, configuration files, and deployment artifacts from repositories

Each adapter handles authentication, pagination, rate limiting, and error recovery specific to its source system.

4.4.2 Target System Adapters

The engine interfaces with target systems through deployment adapters:

- Repository deployment: Creating branches, committing files, and managing pull requests in version control systems
- Platform deployment: Provisioning applications, configurations, and data on target enterprise

platforms

- CI/CD integration: Triggering build, test, and deployment pipelines on target infrastructure

4.4.3 Cross-System Coordination

For transformations spanning multiple systems, the engine coordinates:

- Ordered operations: Extract from source system A, transform, deploy to target system B
- Credential management: Securely managing and rotating credentials for each external system
- Rate limit coordination: Distributing API calls across systems to avoid rate limit violations
- Transactional semantics: Where possible, implementing compensating actions if a downstream deployment fails after upstream operations have succeeded

4.5 Feedback Loops

The system implements closed-loop feedback to improve transformation outcomes:

4.5.1 Validation-Driven Correction

When a validation stage produces a below-threshold quality score:

1. The validation output (score, identified issues, recommendations) is packaged as corrective context
2. The corrective context is routed back to the relevant transformation stage via a feedback edge in the execution graph
3. The transformation stage re-executes with the original input plus the corrective context, enabling the AI agent to address specific identified issues
4. The re-executed output passes through validation again
5. The loop continues until the quality threshold is met or a maximum iteration count is reached

4.5.2 Cross-Pipeline Learning

Across multiple pipeline executions, the system aggregates quality metrics to improve future executions:

- Skill quality tracking: Skills that consistently produce below-threshold output are flagged for review or deprecated in favor of higher-quality alternatives
- Recipe refinement: Translation recipes that produce low validation scores are flagged for

refinement, with the specific validation failures provided as context for recipe improvement

- Pipeline optimization: Stage orderings that produce better outcomes (e.g., running enrichment before a specific transformation type) are identified and promoted to default pipeline templates

4.6 Portfolio-Level Orchestration

The system supports executing transformation pipelines across portfolios of applications:

4.6.1 Portfolio Execution

- Batch dispatch: Multiple applications are submitted as a portfolio, and the engine creates individual pipeline instances for each
- Priority scheduling: Applications are prioritized based on business criticality, complexity, or dependency order
- Resource pooling: Token budgets and API rate limits are managed at the portfolio level, preventing any single application from consuming all available resources
- Progress tracking: Portfolio-level dashboards show aggregate progress, completion rates, and quality distributions

4.6.2 Portfolio-Level Optimization

- Recipe discovery: As applications in the portfolio are transformed, successful transformation patterns are automatically captured as recipes (per Patent 3) and offered for subsequent applications
- Skill selection refinement: The system refines skill selection based on which skills produce the best results for the specific application types in the portfolio
- Resource forecasting: Based on completed applications, the system projects remaining resource consumption and completion time for the portfolio

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented system for orchestrating multi-stage enterprise application transformation comprising:

(a) an orchestration engine configured to receive transformation requests, resolve execution graphs, dispatch operations to registered skills, manage data flow between operations, handle errors through configurable retry and fallback strategies, and stream real-time execution events;

(b) a skill registry containing a catalog of composable transformation skills, each skill having a defined type (deterministic service or AI agent), input schema, output schema, execution requirements, and compatibility metadata, the registry enabling dynamic selection of skills based on source type, target platform, and application characteristics;

(c) an execution graph representing the transformation pipeline as a directed acyclic graph wherein nodes represent transformation operations bound to registered skills and edges represent typed data dependencies between operations, the graph supporting conditional branching, parallel execution lanes, dynamic subgraph expansion, and feedback edges;

(d) an event streaming layer configured to publish real-time pipeline-level, stage-level, and domain-specific events to client interfaces and monitoring systems;

(e) an observability module configured to collect and aggregate per-stage, per-pipeline, and portfolio-level metrics including execution duration, token consumption, quality scores, and error rates;

(f) a retry and fallback module configured to execute fault-tolerant recovery with strategies differentiated by skill type: deterministic retry with exponential backoff and idempotency for deterministic skills, and model fallback, prompt modification, and context reduction for AI agent skills.

Claim 2. A computer-implemented method for executing a hybrid deterministic-AI pipeline for enterprise application transformation comprising:

(a) resolving a transformation pipeline from a skill registry, the pipeline comprising an ordered sequence of deterministic stages and AI agent stages connected by typed data dependencies;

(b) executing deterministic stages that perform reproducible, non-AI operations including parsing, schema normalization, validation, and file system operations, each producing identical outputs for identical inputs;

(c) executing AI agent stages that perform LLM-mediated operations including extraction, enrichment, transformation, and quality assessment, each using context management, model selection, and structured output parsing;

(d) coordinating deterministic and AI stages within a unified execution framework wherein deterministic stages execute first where possible to provide structured input to AI stages, and AI stage outputs pass through deterministic validation before routing to downstream stages;

(e) managing data flow between stages by routing typed outputs of completed stages to the input schemas of dependent downstream stages;

(f) handling failures through type-appropriate retry strategies: idempotent re-execution for deterministic stages and model fallback, prompt modification, or context reduction for AI stages;

(g) streaming real-time execution events and collecting observability metrics across all stages.

Claim 3. A computer-implemented method for dynamically selecting and chaining transformation skills for enterprise application processing comprising:

(a) maintaining a skill registry comprising a plurality of transformation skills, each skill having a defined input schema, output schema, skill type, compatibility metadata, and historical quality metrics;

(b) receiving a transformation request specifying a source type, target platform, and application characteristics;

(c) querying the skill registry to identify compatible skills for each stage of the transformation pipeline, ranking compatible skills by compatibility score, quality metrics, resource efficiency, and availability;

(d) composing the selected skills into a directed acyclic execution graph by matching output schemas of upstream skills to input schemas of downstream skills, validating type compatibility at graph resolution time;

(e) executing the composed skill chain by dispatching each skill in topological order, routing typed outputs between skills, and handling errors through skill-type-appropriate retry and fallback to alternative skills.

Dependent Claims

Claim 4. The system of Claim 1 wherein the skill registry supports skill versioning such that multiple versions of a skill can coexist with independent quality metrics, and the orchestration engine can select between versions based on quality performance or conduct A/B comparisons.

Claim 5. The system of Claim 1 wherein the execution graph supports conditional branching based on intermediate results, such that a validation stage producing a quality score below a configurable threshold causes the pipeline to route execution to a supplementary processing stage rather than proceeding to the next primary stage.

Claim 6. The system of Claim 1 wherein the execution graph supports feedback edges that route validation results back to upstream transformation stages with corrective context, causing re-execution of the transformation with specific identified issues, and wherein the feedback loop iterates until a quality threshold is met or a maximum iteration count is reached.

Claim 7. The method of Claim 2 wherein deterministic-first sequencing ensures that deterministic parsing and normalization stages execute before AI agent stages to provide structured, validated input to the AI, improving AI stage output quality.

Claim 8. The method of Claim 2 wherein AI stage outputs are validated by deterministic validation stages before routing to downstream stages, catching malformed output, schema violations, and quality threshold failures before propagation.

Claim 9. The system of Claim 1 wherein the event streaming layer publishes domain-specific events including: files fetched with paths and sizes, behavioral patterns discovered, extraction completeness scores, transformation file counts, validation scores, context budget utilization, and feedback loop iteration counts.

Claim 10. The system of Claim 1 wherein the event streaming layer uses Server-Sent Events (SSE) for real-time delivery to client interfaces, and simultaneously publishes events to a persistent event log for post-execution analysis and monitoring.

Claim 11. The system of Claim 1 wherein the observability module collects metrics at three levels: per-stage metrics (duration, token consumption, quality scores, retry counts), per-pipeline metrics (end-to-end duration, total cost, stage success/failure counts), and portfolio-level metrics (applications processed, aggregate quality scores, recipe reuse rates, throughput).

Claim 12. The system of Claim 1 wherein the observability module provides distributed tracing with correlation IDs linking all events within a pipeline execution across multi-system operations.

Claim 13. The system of Claim 1 wherein the retry and fallback module implements model fallback for AI agent skills such that if a primary LLM fails or produces below-threshold output, the system automatically retries with a designated fallback model without manual intervention.

Claim 14. The system of Claim 1 wherein the retry and fallback module implements prompt modification on retry for AI agent skills, including simplifying instructions, reducing context, or adding examples to improve output quality on subsequent attempts.

Claim 15. The system of Claim 1 wherein the retry and fallback module implements context reduction on retry for AI agent skills such that if an AI stage fails due to context length constraints, the system reduces the context provided and retries with the reduced context.

Claim 16. The system of Claim 1 wherein the retry and fallback module implements circuit breaker patterns for deterministic skills such that after N consecutive failures, the pipeline pauses and escalates rather than continuing to retry.

Claim 17. The system of Claim 1 wherein the retry and fallback module supports partial result recovery for AI agent skills such that if an AI stage produces partial output before failing, the partial result is preserved for manual review or supplementary processing.

Claim 18. The method of Claim 3 wherein skill chaining validates type compatibility between upstream output schemas and downstream input schemas at graph resolution time, before pipeline execution begins, preventing runtime failures from schema incompatibility.

Claim 19. The system of Claim 1 further comprising source system adapters and target system adapters that interface with external systems for artifact ingestion and deployment, each adapter handling authentication, pagination, rate limiting, and error recovery specific to its system.

Claim 20. The system of Claim 1 further comprising cross-system coordination logic that manages ordered operations across multiple external systems (extract from system A, transform, deploy to system B), credential management, rate limit coordination across systems, and compensating actions for partial failures.

Claim 21. The system of Claim 1 further comprising portfolio-level orchestration that executes transformation pipelines across multiple applications with batch dispatch, priority scheduling, resource pooling across applications, and aggregate progress tracking.

Claim 22. The system of Claim 1 wherein the orchestration engine supports dynamic subgraph expansion such that a single node in the execution graph expands into a subgraph at runtime based on runtime conditions, enabling adaptive pipeline structure.

Claim 23. The method of Claim 2 further comprising adaptive pipeline modification wherein based on AI stage results, the orchestration engine dynamically adds, removes, or reorders stages in the remaining pipeline execution.

Claim 24. The system of Claim 1 further comprising cross-pipeline learning that aggregates quality metrics across multiple pipeline executions to refine skill selection, flag underperforming skills for review, and identify stage orderings that produce better outcomes.

Claim 25. The system of Claim 1 further comprising human-in-the-loop escalation for failures that cannot be resolved through automated retry or fallback, wherein the pipeline pauses and creates a review task with full context including inputs, error details, and partial results.

Claim 26. The system of Claim 1 wherein the orchestration engine maintains persistent execution state with checkpoint capability, enabling restart from the last successful stage after a system failure without re-executing completed stages.

Claim 27. The method of Claim 2 further comprising integration with a progressive context hydration system (as described in related Patent 6) for AI agent stages that require iterative context acquisition, wherein the orchestration engine allocates context budgets per AI stage and the hydration loop operates within the allocated budget.

Claim 28. The method of Claim 2 further comprising integration with a translation recipe system (as described in related Patent 3) for transformation stages, wherein the orchestration engine selects and applies stored recipes based on the source type and target platform, and recipe application executes as a single AI stage within the pipeline.

Claim 29. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to implement the system of Claim 1.

---

6. Advantages Over Prior Art

vs. Manual Orchestration

1. Automation: End-to-end pipeline execution without human intervention for routine transformations
2. Scalability: Portfolio-level execution across hundreds of applications
3. Consistency: Same pipeline applied uniformly to all applications of a given type
4. Observability: Real-time progress tracking and quality metrics replace manual status checking

vs. Script-Based Pipelines

1. Error recovery: Configurable retry, fallback, and feedback loops replace brittle fail-fast behavior
2. Dynamic adaptation: Conditional branching and subgraph expansion replace rigid sequential execution
3. Observability: Structured event streaming and multi-level metrics replace log file inspection
4. Skill composition: Dynamic skill selection and chaining replace hard-coded tool invocations

vs. General-Purpose Workflow Engines

1. AI-native execution: Token budgets, context management, model selection, and guard enforcement are first-class execution semantics, not bolted-on integrations
2. Domain-specific observability: Extraction completeness, transformation fidelity, and genome coverage metrics are built into the observability model
3. Hybrid coordination: Deterministic-first sequencing, AI output validation, and type-differentiated retry strategies are purpose-built for mixed workloads
4. Feedback loops: Validation-driven correction cycles are integrated into the execution graph, not implemented as external retry logic

vs. LLM Agent Frameworks

1. Deterministic reliability: Critical operations (parsing, validation, deployment) execute deterministically, not through AI inference
2. Production fault tolerance: Circuit breakers, exponential backoff, model fallback, and partial result recovery provide enterprise-grade reliability
3. Multi-system coordination: First-class source and target system adapters with credential management and rate limiting
4. Portfolio scale: Resource pooling, priority scheduling, and aggregate metrics support portfolio-level execution

Unique Advantages

1. Control plane positioning: The system is the single coordination point for all extraction, transformation, validation, and deployment operations, providing unified observability and control
2. Skill ecosystem: The registry enables an extensible ecosystem of transformation capabilities that can be contributed, versioned, and quality-tracked independently
3. Composability with patent family: The system integrates with genome extraction (Patent

1/5), translation recipes (Patent 3), and progressive context hydration (Patent 6) as first-class pipeline stages

---

7. Drawings/Figures Description

Figure 1: Orchestration Control Plane Architecture -- Block diagram showing the orchestration engine at the center, connected to: skill registry, execution graph engine, event streaming layer, observability module, retry/fallback module, source system adapters, and target system adapters.

Figure 2: Standard Transformation Pipeline DAG -- Directed acyclic graph showing the full pipeline from source artifact ingestion through extraction, normalization, enrichment, transformation, validation, and deployment, with conditional branches, parallel lanes, and feedback edges annotated.

Figure 3: Skill Registry Architecture -- Entity diagram showing skill records with input/output schemas, execution requirements, compatibility metadata, and quality metrics, with skill selection algorithm flow.

Figure 4: Hybrid Execution Model -- Sequence diagram showing interleaved execution of deterministic stages (parser, validator, deployer) and AI agent stages (extractor, enricher, transformer) within a single pipeline, with deterministic-first sequencing and AI output validation.

Figure 5: Skill Chaining and Type Validation -- Diagram showing a chain of six skills with typed output-to-input connections, type compatibility validation at resolution time, and fallback skill designation.

Figure 6: Event Streaming Architecture -- Diagram showing event flow from pipeline execution through SSE to client interfaces and persistent event log to monitoring systems.

Figure 7: Retry and Fallback Decision Tree -- Flowchart showing the retry decision process: error classification -> deterministic retry (backoff, idempotent) or AI retry (model fallback, prompt modification, context reduction) -> circuit breaker or human escalation.

Figure 8: Feedback Loop Execution -- Sequence diagram showing a transformation stage producing output -> validation stage scoring below threshold -> corrective context routing back to transformation -> re-execution -> re-validation -> acceptance or additional iteration.

Figure 9: Conditional Branching in Execution Graph -- Diagram showing a validation node with two conditional outgoing edges: one for above-threshold scores (proceeding to transformation) and one for below-threshold scores (routing to supplementary extraction).

Figure 10: Portfolio-Level Orchestration -- Dashboard wireframe showing aggregate metrics across a portfolio of applications:

completion progress, quality distribution, resource consumption, and recipe discovery rates.

Figure 11: Multi-System Coordination -- Sequence diagram showing the orchestration engine coordinating operations across a source system (extraction), internal processing (transformation), and target system (deployment) with credential management and error recovery.

Figure 12: Dynamic Subgraph Expansion -- Before/after diagram showing a single "multi-export extraction" node expanding at runtime into N parallel extraction nodes based on the number of source exports detected.

Figure 13: Observability Stack -- Layered diagram showing structured logging, distributed tracing, metric aggregation, alerting, and dashboards with correlation IDs connecting all layers.

Figure 14: Patent Family Integration -- Diagram showing how the orchestration control plane (Patent 7) coordinates Patent 1/5 (genome extraction/modeling), Patent 3 (translation recipes), and Patent 6 (progressive context hydration) as pipeline stages within the unified execution framework.

---

8. Inventors

[To be completed by filing attorney]

9. Filing Notes

- Control plane positioning is the strategic framing: This patent should be positioned as the coordination layer that ties the entire patent family together. The orchestration engine is the "operating system" for enterprise application portability -- it is the system that invokes extraction (Patent 1/5), applies recipes (Patent 3), manages context hydration (Patent 6), and coordinates the end-to-end pipeline.
- Strongest novel claim: The hybrid deterministic-AI execution model (Claim 2) with type-differentiated retry strategies and deterministic-first sequencing is the most architecturally novel element. No existing system provides unified orchestration of deterministic services and AI agents with execution semantics appropriate to each.
- Broadest claim: Claim 1 covers any orchestration system with a skill registry, execution graph, event streaming, observability, and type-differentiated retry. This is deliberately broad and should be defended on the combination of these elements, not any single one.
- Skill registry as ecosystem moat: The skill registry (Claim 3, Claims 4, 18) creates an extensible ecosystem that grows more valuable as more skills are contributed. This is a

strategic competitive advantage beyond the technical novelty.

- Feedback loops are differentiating: Validation-driven correction loops (Claim 6) are absent from general-purpose workflow engines and LLM agent frameworks. This is a key differentiator during prosecution.
- Patent family integration: Claims 27 and 28 explicitly reference Patents 6 and 3, establishing the orchestration control plane as the coordination layer for the entire patent family. This strengthens all patents in the family.
- Prior art search should focus on: Workflow orchestration engines (Airflow, Temporal, Step Functions), LLM agent frameworks (LangChain, AutoGen, CrewAI), MLOps pipeline tools (Kubeflow, MLflow), CI/CD systems (Jenkins, GitHub Actions). The claims must be differentiated on: domain specificity, hybrid deterministic-AI execution, skill-based composition, AI-specific retry semantics, and portfolio-level orchestration.
- International filing: PCT filing recommended given the global enterprise transformation market.
- Consider filing as part of the existing patent family to strengthen the overall portfolio.

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.

Patent Technical Disclosure: Incremental Enterprise Application Migration

Title

System and Method for Incremental Enterprise Application Migration Using Partial Genome Extraction, Dependency-Aware Component Selection, and Selective Reconstruction with Legacy-Modern Coexistence

1. Field of Invention

This invention relates to enterprise application migration, incremental modernization, hybrid system operation, and risk-managed technology transformation. Specifically, it describes a system and method for migrating enterprise applications incrementally -- extracting individual components (workflows, forms, data models, business rules, integrations) from a source system's application genome, resolving their dependencies, selectively reconstructing them on a target platform, and maintaining synchronized coexistence between the legacy source system and the partially migrated target system throughout the migration lifecycle.

The system enables enterprises to migrate applications component by component rather than all-at-once, reducing risk, accelerating time to value, and allowing the legacy and modern systems to operate in parallel during the transition period.

2. Background and Prior Art Gaps

2.1 The Migration Risk Problem

Enterprise application migration -- moving a business-critical application from one platform to another -- is among the highest-risk initiatives an IT organization undertakes. Current migration approaches present a binary choice:

Full migration (big-bang):

- The entire application is extracted, transformed, and deployed on the target platform in a single effort
- The legacy system is decommissioned and the new system goes live simultaneously

- Risks: Any defect in the migrated application affects all users immediately. Rollback requires reverting the entire application. The migration project takes months to years before delivering any business value. Testing must cover the complete application before cutover.
- Failure mode: If the migration fails or produces unacceptable results, the organization has invested significant resources with no incremental value delivered

Manual incremental migration:

- Developers manually identify components to migrate, manually rebuild them on the target platform, and manually maintain synchronization
- Risks: No automated dependency resolution -- migrating a workflow without its dependent data models produces broken functionality. No systematic approach to identifying which components can be safely migrated independently. Manual synchronization between legacy and new systems is error-prone and operationally expensive.

2.2 Current State of Application Genome Technology

The application genome model (as described in related Patents 1 and 5) provides a canonical representation of application intent, structure, and behavior. Existing genome extraction systems produce a complete genome -- the full application's data models, workflows, business rules, UI specifications, and integrations extracted as a unified artifact.

However, the current genome model treats the application as a monolith. There is no mechanism to:

- Extract a subset of the genome corresponding to specific application components
- Understand the dependency graph between components to determine which subsets can be safely extracted independently
- Selectively reconstruct individual components on a target platform while the remaining components continue to operate on the source system
- Synchronize data and operations between the legacy source and the partially migrated target during the coexistence period

2.3 Gaps in Prior Art

1. No existing system supports partial genome extraction -- extracting a subset of an application's genome corresponding to specific components (individual workflows, forms, data models, or business rules) rather than the complete application.

2. No existing system performs dependency-aware component selection that analyzes the genome's dependency graph to determine which components can be safely extracted

independently, which require co-extraction of dependent components, and which cannot be extracted without breaking the remaining application.

3. No existing system supports selective reconstruction -- rebuilding individual genome components on a target platform while the remaining components continue to operate on the source system.

4. No existing system provides migration wave planning -- automatically grouping genome components into ordered migration waves based on dependency analysis, business priority, risk assessment, and operational constraints.

5. No existing system maintains legacy-modern coexistence -- operating the source system and the partially migrated target system simultaneously with automated synchronization of shared data and cross-system workflow routing.

6. No existing system provides incremental migration progress tracking -- measuring and reporting migration completeness at the component level, tracking which genome components have been migrated, which are in progress, and which remain on the legacy system.

7. No existing system supports selective rollback at the component level -- reverting individual migrated components back to the legacy system without affecting other successfully migrated components.

---

3. Summary of Invention

The present invention provides a system and method for incremental enterprise application migration that enables organizations to migrate applications component by component, with automated dependency resolution, selective reconstruction, and legacy-modern coexistence throughout the migration lifecycle.

The system comprises:

3.1 Partial Genome Extraction Engine

The system extends the application genome model with the ability to extract partial genomes -- subsets of the full genome corresponding to specific application components:

- Component-level extraction: Extract individual workflows, forms, data models, business rules,

UI screens, or integration definitions as standalone genome fragments

- Dependency-aware extraction: When a component is selected for extraction, the system automatically identifies and includes all dependent components required for the extracted component to function correctly on the target platform
- Extraction boundary analysis: The system analyzes the dependency graph to determine the minimum viable extraction set -- the smallest set of components that can be extracted and reconstructed as a functioning unit
- Remaining application impact assessment: The system assesses the impact of extracting selected components on the components that remain on the source system, identifying any broken references, orphaned dependencies, or degraded functionality

3.2 Dependency Resolution Engine

The system builds and analyzes a component-level dependency graph from the application genome:

- Dependency graph construction: From the full genome's entity relationships, workflow references, business rule triggers, UI data bindings, and integration connections, the system constructs a directed graph where nodes are individual components and edges are typed dependencies (references, triggers, data flows, UI bindings)
- Dependency classification: Dependencies are classified by type:
  - Hard dependencies: The component cannot function without the dependent component (e.g., a workflow that references a data model table)
  - Soft dependencies: The component can function with degraded capability without the dependent component (e.g., a form that references an optional lookup table)
  - Cross-system dependencies: Dependencies that will require synchronization bridges during the coexistence period (e.g., a migrated workflow that needs to read data still in the legacy system)
- Extraction feasibility analysis: For any proposed extraction set, the system determines whether the set is self-contained (all hard dependencies are included), whether it creates cross-system dependencies that require bridges, and whether the remaining legacy application will continue to function
- Circular dependency detection: The system identifies circular dependency clusters that must be migrated as atomic units

3.3 Migration Wave Planner

The system automatically plans the migration as a sequence of ordered waves:

- Wave generation: The system analyzes the dependency graph, business priority assignments, and risk assessments to group components into migration waves -- ordered sets of components that can be migrated together as a unit
- Wave ordering: Waves are ordered such that:
  - Foundation components (data models, shared reference data) migrate in early waves
  - Dependent components (workflows, business rules that reference the data models) migrate in subsequent waves
  - Complex, high-risk components (multi-system integrations, cross-functional workflows) migrate in later waves
  - Each wave is independently deployable -- completing a wave produces a functioning partially-migrated system
- Wave sizing: Each wave is sized to be completable within a defined time window (e.g., 1-2 sprint cycles), controlling the blast radius of any individual migration step
- Risk scoring: Each wave receives a risk score based on: component complexity, number of cross-system dependencies created, number of users affected, and business criticality of the components
- Wave dependency validation: The system validates that each wave's extraction set is self-contained (all hard dependencies are satisfied by previously migrated waves or the current wave)

3.4 Selective Reconstruction Engine

The system reconstructs extracted genome components on the target platform:

- Component-level translation: The system applies translation recipes (per Patent 3) or AI-mediated transformation to individual genome components, producing target-platform-specific implementations for each component
- Interface generation: For components that interact with other components still on the legacy system, the system generates interface specifications (API contracts, data exchange formats, event schemas) that define the cross-system communication protocol
- Stub generation: For dependencies that remain on the legacy system, the system generates stubs -- placeholder implementations on the target platform that delegate to the legacy system via synchronization bridges
- Incremental deployment: Reconstructed components are deployed to the target platform

incrementally, with each deployment validated before proceeding to the next

3.5 Legacy-Modern Coexistence Manager

The system maintains synchronized operation between the legacy source system and the partially migrated target system:

- Data synchronization: Shared data entities (referenced by both legacy and migrated components) are synchronized between systems through configurable synchronization patterns:
  - Primary-replica: One system is the authoritative source; the other receives read-only replicas
  - Bidirectional sync: Both systems can modify shared data, with conflict resolution rules
  - Event-driven sync: Changes in one system are propagated to the other via events
- Workflow routing: For workflows that span legacy and migrated components, the system routes workflow steps to the appropriate system:
  - Steps handled by migrated components execute on the target platform
  - Steps handled by legacy components execute on the source system
  - The coexistence manager coordinates handoffs between systems at workflow transition points
- User routing: Users are routed to the appropriate system based on which components they need:
  - Users accessing migrated functionality are directed to the target platform
  - Users accessing legacy functionality are directed to the source system
  - The routing can be feature-flag-controlled for gradual user migration
- Coexistence health monitoring: The system monitors synchronization lag, cross-system error rates, and data consistency metrics, alerting when coexistence health degrades

3.6 Migration Progress Tracker

The system tracks and reports migration progress at the component level:

- Component migration status: Each genome component is tracked as: not started, extraction planned, extracted, transformation in progress, transformed, deployed to target, validated on target, live on target, legacy decommissioned
- Coverage metrics: Percentage of the application genome that has been successfully migrated,

broken down by component type (workflows, forms, data models, rules, integrations)

- Coexistence metrics: Number and type of cross-system dependencies, synchronization bridge count, data consistency scores
- Risk metrics: Current risk exposure based on which components are in which state and the number of active cross-system dependencies
- Projected completion: Based on completed wave velocity, projected completion date for full migration

---

4. Detailed Description

4.1 Partial Genome Extraction

4.1.1 Component Selection Interface

The system provides a component selection interface overlaid on the full application genome. Users can:

1. Browse the genome's component inventory: View all workflows, forms, data models, business rules, UI screens, and integrations in the genome with their names, descriptions, and dependency counts
2. Select components for extraction: Choose individual components or groups of components for inclusion in a migration wave
3. View dependency analysis: For any selected component, see:
   - Hard dependencies that must be co-extracted
   - Soft dependencies that are recommended but not required
   - Components that depend on the selected component (reverse dependencies)
   - The minimum viable extraction set (the selected component plus all hard dependencies)
4. Assess extraction impact: View the impact on the remaining legacy application:
   - Broken references that will require stubs or bridges
   - Features that will be degraded or unavailable after extraction
   - Cross-system dependencies that will be created

4.1.2 Partial Genome Extraction Process

When components are selected for extraction:

1. Dependency closure computation: The system computes the transitive closure of hard dependencies from the selected components, producing the minimum viable extraction set
2. Genome slicing: The system extracts the selected components and their dependencies from the full genome, producing a partial genome -- a self-contained genome fragment that represents only the selected components
3. Interface boundary identification: The system identifies all points where the partial genome references components not included in the extraction set, producing an interface boundary specification
4. Remaining genome analysis: The system analyzes the full genome minus the extracted components to identify broken references, orphaned components, and degraded functionality

The partial genome is a first-class genome artifact -- it has the same structure (structural, behavioral, process intent, UI semantics, system interaction facets) as a full genome, but represents only a subset of the application.

4.1.3 Extraction Granularity

The system supports extraction at multiple granularity levels:

- Individual component: A single workflow, form, data model, or rule
- Component cluster: A group of tightly coupled components identified by dependency analysis (e.g., a form + its associated workflow + the triggered business rules)
- Functional module: A complete business capability (e.g., "HR case management" comprising all related forms, workflows, rules, data models, and integrations)
- Custom selection: User-defined selection of any combination of components

4.2 Dependency Resolution

4.2.1 Dependency Graph Construction

The system constructs a component-level dependency graph from the application genome:

```
dependency_graph:
  nodes:
    - id: "wf_case_lifecycle"
      type: workflow
      name: "Case Lifecycle"
```

```yaml
    - id: "form_case_submission"
      type: form
      name: "Case Submission Form"

    - id: "tbl_hr_case"
      type: data_model
      name: "HR Case Table"

    - id: "rule_auto_route"
      type: business_rule
      name: "Auto-Route by Region"

    - id: "int_hr_system"
      type: integration
      name: "HR System Sync"

  edges:
    - from: "wf_case_lifecycle"
      to: "tbl_hr_case"
      type: hard_dependency
      reason: "Workflow operates on HR Case records"

    - from: "form_case_submission"
      to: "tbl_hr_case"
      type: hard_dependency
      reason: "Form creates HR Case records"

    - from: "form_case_submission"
      to: "wf_case_lifecycle"
      type: hard_dependency
      reason: "Form submission triggers workflow"

    - from: "rule_auto_route"
      to: "tbl_hr_case"
      type: hard_dependency
      reason: "Rule triggers on HR Case creation"

    - from: "wf_case_lifecycle"
      to: "int_hr_system"
      type: soft_dependency
      reason: "Workflow optionally syncs to HR system"
```

4.2.2 Dependency Analysis Operations

The system performs the following operations on the dependency graph:

Minimum Viable Extraction Set: For a given component selection, compute the minimum set of additional components required for the selection to function independently:

```
selected: {form_case_submission}
```

```
hard_dependencies: {tbl_hr_case, wf_case_lifecycle}
transitive_hard_deps: {tbl_hr_case}  (wf_case_lifecycle also depends on tbl_hr_case)
minimum_viable_set: {form_case_submission, tbl_hr_case, wf_case_lifecycle}
soft_dependencies_excluded: {rule_auto_route, int_hr_system}
```

Extraction Impact Analysis: For a given extraction set, compute the impact on remaining components:

```
extracting: {form_case_submission, tbl_hr_case, wf_case_lifecycle}
remaining_broken_refs:
  - rule_auto_route -> tbl_hr_case (hard dep: rule will not function)
  - int_hr_system -> tbl_hr_case (soft dep: sync will be unavailable)
recommendation: "Include rule_auto_route in extraction set or generate stub"
```

Independent Component Clusters: Identify groups of components with no cross-group hard dependencies, representing components that can be migrated independently:

```
cluster_1: {form_case_submission, tbl_hr_case, wf_case_lifecycle, rule_auto_route}
cluster_2: {form_asset_request, tbl_assets, wf_asset_approval}
cluster_3: {int_hr_system}  (soft dependencies only)
migration_order: cluster_3 -> cluster_1 -> cluster_2  (or cluster_1 and cluster_2 in parallel)
```

Circular Dependency Detection: Identify component groups with circular hard dependencies that must be migrated as atomic units:

```
circular_cluster: {wf_approval_chain, rule_escalation, wf_escalation_handler}
reason: "approval_chain triggers escalation rule, which triggers escalation_handler,
         which feeds back into approval_chain"
migration_constraint: "Must migrate as a single unit"
```

4.3 Migration Wave Planning

4.3.1 Wave Generation Algorithm

The system generates migration waves through the following process:

1. Build component dependency graph from the full genome
2. Identify foundation components: Components with no hard dependencies (typically data models, reference data, shared configurations) -- these form Wave 1
3. Topological ordering: Order remaining components by dependency depth (components that depend only on Wave 1 components form Wave 2; components depending on Wave 2 form Wave 3; etc.)
4. Business priority overlay: Within each wave, prioritize components based on business value, user impact, and urgency

5. Risk-based sizing: Split large waves into smaller sub-waves to control blast radius, ensuring no single wave affects more than a configurable percentage of users or business processes
6. Constraint satisfaction: Ensure each wave satisfies:
   - All hard dependencies are resolved by current or prior waves
   - The wave is independently deployable (completing the wave produces a stable system)
   - The wave's cross-system dependency count is within acceptable limits

4.3.2 Wave Plan Output

```
migration_plan:
  total_components: 47
  total_waves: 5
  estimated_duration: "8-12 weeks"

  waves:
    - wave: 1
      name: "Foundation Data Models"
      components:
        - {id: "tbl_hr_case", type: data_model, risk: low}
        - {id: "tbl_employees", type: data_model, risk: low}
        - {id: "tbl_assets", type: data_model, risk: low}
      new_cross_system_deps: 0
      estimated_effort: "1 sprint"
      risk_score: 12

    - wave: 2
      name: "Core Workflows and Forms"
      components:
        - {id: "form_case_submission", type: form, risk: medium}
        - {id: "wf_case_lifecycle", type: workflow, risk: medium}
        - {id: "rule_auto_route", type: business_rule, risk: low}
      dependencies_on_prior_waves: [tbl_hr_case, tbl_employees]
      new_cross_system_deps: 2
      estimated_effort: "2 sprints"
      risk_score: 34

    - wave: 3
      name: "Secondary Workflows"
      components:
        - {id: "form_asset_request", type: form, risk: medium}
        - {id: "wf_asset_approval", type: workflow, risk: medium}
      dependencies_on_prior_waves: [tbl_assets, tbl_employees]
      new_cross_system_deps: 1
      estimated_effort: "1 sprint"
      risk_score: 28

    - wave: 4
```

```
    name: "Integrations and Advanced Rules"
    components:
      - {id: "int_hr_system", type: integration, risk: high}
      - {id: "rule_sla_escalation", type: business_rule, risk: medium}
    dependencies_on_prior_waves: [wf_case_lifecycle, tbl_hr_case]
    new_cross_system_deps: 0  # resolves 2 prior cross-system deps
    estimated_effort: "2 sprints"
    risk_score: 45

  - wave: 5
    name: "UI and Reporting"
    components:
      - {id: "ui_dashboard", type: ui_screen, risk: low}
      - {id: "ui_case_detail", type: ui_screen, risk: low}
    dependencies_on_prior_waves: [all prior waves]
    new_cross_system_deps: 0
    estimated_effort: "1 sprint"
    risk_score: 15

cross_system_dep_peak: "Wave 2-3 (3 active bridges)"
risk_peak: "Wave 4 (integration migration)"
```

4.4 Selective Reconstruction

4.4.1 Component-Level Translation

For each component in a migration wave, the system:

1. Extracts the partial genome: Produces a self-contained genome fragment for the component and its dependencies
2. Selects the translation recipe: From the recipe registry (Patent 3), selects the appropriate recipe for the component type and target platform
3. Applies translation with component context: The translation recipe is applied with additional context about the component's role in the overall application, its dependencies on already-migrated components, and its interface requirements with legacy components
4. Generates target-platform implementation: Produces the target-platform-specific implementation for the component

4.4.2 Interface and Stub Generation

For components that interact with legacy system components not yet migrated:

Interface specifications:

```
cross_system_interface:
  name: "hr_case_data_bridge"
  direction: "target_reads_from_legacy"
  source_component: "tbl_hr_case"  # on legacy system
  consuming_component: "wf_case_lifecycle"  # on target platform
  data_contract:
    entity: "hr_case"
    fields: [id, employee_id, status, priority]
    sync_pattern: "event_driven"
    sync_frequency: "real_time"
    conflict_resolution: "legacy_wins"
```

Stub implementations:

```
stub:
  name: "hr_case_legacy_proxy"
  purpose: "Proxy HR Case data reads to legacy system until tbl_hr_case is migrated"
  deployed_on: "target_platform"
  delegates_to: "legacy_system.hr_case_api"
  replaces_when: "tbl_hr_case migrated in Wave 1"
  auto_retire: true
```

Stubs are automatically retired when their corresponding real component is migrated in a subsequent wave.

4.4.3 Incremental Deployment

Each wave's reconstructed components are deployed incrementally:

1. Deploy to staging: Components deployed to target platform staging environment
2. Automated validation: Extracted genome's validation scores compared against reconstructed component behavior
3. Cross-system integration testing: Stubs and synchronization bridges tested with live legacy system data
4. Canary deployment: Migrated components activated for a small percentage of users while monitoring error rates and performance
5. Full activation: Upon successful canary, migrated components activated for all users with legacy components deactivated for the migrated functionality
6. Stub cleanup: Previous wave stubs that are now satisfied by real components are retired

4.5 Legacy-Modern Coexistence

4.5.1 Coexistence Architecture

During the migration period, the legacy and target systems operate simultaneously:

```
+--------------------------------+     +--------------------------------+
|        LEGACY SYSTEM             |     |        TARGET PLATFORM           |
|                                  |     |                                  |
|  [Components NOT yet migrated]   |     |  [Migrated components]           |
|  - Remaining workflows           |     |  - Wave 1-N deployed components  |
|  - Remaining forms               |     |  - Stubs for legacy deps         |
|  - Remaining rules               |     |  - Sync bridge endpoints         |
|  - Shared data (source of truth  |     |  - Generated interfaces          |
|    for non-migrated entities)    |     |                                  |
|                                  |     |                                  |
+----------+----------------------+     +----------+----------------------+
           |                                        |
           +------------+--------------------------+
                        |
           +------------+--------------------------+
           |     COEXISTENCE MANAGER                  |
           |                                          |
           |  - Data synchronization bridges          |
           |  - Workflow routing (legacy <-> target)    |
           |  - User routing (feature flags)          |
           |  - Coexistence health monitoring         |
           |  - Stub lifecycle management             |
           +------------------------------------------+
```

4.5.2 Data Synchronization Patterns

The coexistence manager implements configurable data synchronization between systems:

Primary-Replica Pattern:

- One system is designated as the authoritative source for each data entity
- The other system receives a read-only replica, updated in near-real-time
- Used when only one system needs to write to a given entity (e.g., legacy is primary for non-migrated data models)

Bidirectional Synchronization Pattern:

- Both systems can read and write shared data entities
- Conflict resolution rules determine which write prevails when both systems modify the same record:
  - Timestamp-based: Most recent write wins
  - System-priority: Designated system's write always wins
  - Field-level merge: Non-conflicting field changes are merged; conflicting fields escalate for

review

- Used during transition periods when both systems may need to modify shared data

Event-Driven Synchronization Pattern:

- Changes in one system publish events to a shared event bus
- The other system subscribes to relevant events and applies changes
- Used for real-time synchronization of high-volume or time-sensitive data

4.5.3 Workflow Routing

For workflows that span legacy and migrated components:

1. Router configuration: The coexistence manager maintains a routing table mapping each workflow step to the system responsible for executing it
2. Cross-system handoff: When a workflow transitions from a step on one system to a step on the other, the coexistence manager:
   - Serializes the workflow context (current state, data payload, actor identity)
   - Routes the context to the target system
   - The target system deserializes the context and continues execution
3. Routing evolution: As components are migrated in successive waves, the routing table is updated to reflect the new system assignments. Eventually, all workflow steps route to the target platform and the cross-system routing is retired.

4.5.4 Selective Rollback

If a migrated component produces unacceptable results after deployment:

1. Component-level rollback: The specific component is deactivated on the target platform and reactivated on the legacy system
2. Routing update: The coexistence manager updates routing tables to direct users and workflow steps back to the legacy system for the rolled-back component
3. Stub restoration: If stubs were retired when the component was migrated, they are restored to their pre-migration state
4. Isolation: Rollback of one component does not affect other successfully migrated components -- the system maintains independent rollback capability per component

4.6 Migration Progress and Risk Management

4.6.1 Component-Level Status Tracking

Each genome component is tracked through the migration lifecycle:

```
component_status:
  - id: "tbl_hr_case"
    type: data_model
    status: "live_on_target"
    wave: 1
    migrated_date: "2026-02-15"
    validation_score: 94
    rollback_available: true

  - id: "wf_case_lifecycle"
    type: workflow
    status: "deployed_to_target"
    wave: 2
    validation_score: 87
    cross_system_deps: ["int_hr_system"]
    stubs_active: ["hr_system_legacy_proxy"]

  - id: "int_hr_system"
    type: integration
    status: "extraction_planned"
    wave: 4
    blocked_by: []
    blocks: ["retirement of hr_system_legacy_proxy stub"]
```

4.6.2 Risk Dashboard

The system provides a real-time risk dashboard showing:

- Migration progress: Percentage of genome components in each status (not started -> live on target)
- Active cross-system dependencies: Count and type of synchronization bridges currently operating
- Coexistence health: Synchronization lag, cross-system error rates, data consistency scores
- Risk exposure: Components currently in transition (deployed but not yet validated) that represent active risk
- Rollback readiness: Which components can be independently rolled back if needed
- Wave forecast: Projected dates for upcoming waves based on current velocity

---

5. Key Claims

Independent Claims

Claim 1. A computer-implemented system for incremental enterprise application migration comprising:

(a) a partial genome extraction engine configured to extract subsets of an application genome corresponding to selected application components, producing self-contained genome fragments that represent individual workflows, forms, data models, business rules, or integrations independently of the full application;

(b) a dependency resolution engine configured to construct a component-level dependency graph from the application genome, classify dependencies as hard (required for function), soft (enhancing but not required), or cross-system (requiring synchronization bridges), and compute minimum viable extraction sets for proposed component selections;

(c) a migration wave planner configured to automatically group genome components into ordered migration waves based on dependency analysis, business priority, risk assessment, and operational constraints, wherein each wave is independently deployable and produces a stable partially-migrated system;

(d) a selective reconstruction engine configured to apply translation recipes or AI-mediated transformation to individual genome components, generate interface specifications and stubs for dependencies on non-migrated legacy components, and deploy reconstructed components incrementally to a target platform;

(e) a coexistence manager configured to maintain synchronized operation between the legacy source system and the partially migrated target system through data synchronization, workflow routing, user routing, and coexistence health monitoring;

(f) a migration progress tracker configured to track component-level migration status and provide risk metrics throughout the migration lifecycle.

Claim 2. A computer-implemented method for selective reconstruction of enterprise application components on a target platform comprising:

(a) receiving a selection of one or more application components from an application genome for migration to a target platform;

(b) computing a minimum viable extraction set by analyzing the genome's dependency graph to identify all hard dependencies of the selected components, and including those dependencies in the extraction set;

(c) extracting a partial genome comprising the minimum viable extraction set as a self-contained genome fragment;

(d) generating target-platform implementations for each component in the partial genome through translation recipe application or AI-mediated transformation;

(e) for each dependency in the partial genome that references a component not included in the extraction set, generating an interface specification defining the cross-system communication protocol and a stub implementation on the target platform that delegates to the legacy system;

(f) deploying the generated implementations and stubs to the target platform;

(g) configuring the coexistence manager to route data and workflow operations between the legacy and target systems for the

migrated components.

Claim 3. A computer-implemented method for dependency-aware incremental application migration comprising:

(a) constructing a component-level dependency graph from a complete application genome, wherein nodes represent individual application components and edges represent typed dependencies between components;

(b) classifying each dependency as hard (the upstream component cannot function without the downstream component), soft (the upstream component can function with degraded capability without the downstream component), or cross-system (the dependency will require a synchronization bridge during coexistence);

(c) computing independent component clusters -- groups of components with no cross-group hard dependencies that can be migrated independently;

(d) detecting circular dependency clusters that must be migrated as atomic units;

(e) generating an ordered migration wave plan wherein foundation components with no hard dependencies are assigned to early waves, dependent components are assigned to subsequent waves in topological dependency order, and each wave is validated to be independently deployable;

(f) for each wave, computing the extraction impact on the remaining legacy application, identifying broken references, orphaned dependencies, and required synchronization bridges.

Dependent Claims

Claim 4. The system of Claim 1 further comprising a hybrid operation capability wherein the legacy source system and the target platform operate simultaneously with both systems serving active users, the coexistence manager routing each user request to the appropriate system based on which components handle the requested functionality.

Claim 5. The system of Claim 1 wherein the coexistence manager implements data synchronization between the legacy and target systems using one or more configurable patterns: primary-replica synchronization with a designated authoritative source, bidirectional synchronization with configurable conflict resolution rules, and event-driven synchronization via a shared event bus.

Claim 6. The system of Claim 1 wherein the coexistence manager implements workflow routing for workflows that span legacy and migrated components, routing each workflow step to the system responsible for executing it and coordinating cross-system handoffs by serializing and transferring workflow context between systems.

Claim 7. The method of Claim 2 further comprising selective rollback capability wherein individual migrated components can be deactivated on the target platform and reactivated on the legacy system without affecting other successfully migrated components.

Claim 8. The method of Claim 3 further comprising a phased migration capability wherein the migration wave plan is executed iteratively with each wave completing extraction, reconstruction, deployment, validation, and activation before the next wave begins, and wherein the system maintains a stable, functioning application at the end of each wave.

Claim 9. The method of Claim 3 wherein the dependency resolution engine performs extraction impact analysis that computes, for a proposed extraction set, the effects on remaining legacy components including: broken hard references that will cause functionality loss, broken soft references that will cause degraded functionality, and orphaned components that will have no remaining consumers.

Claim 10. The system of Claim 1 wherein the partial genome extraction engine supports multiple extraction granularity levels: individual component extraction, component cluster extraction for tightly coupled component groups identified by dependency analysis, functional module extraction for complete business capabilities, and custom selection of arbitrary component combinations.

Claim 11. The system of Claim 1 wherein the selective reconstruction engine generates stubs for legacy dependencies that are automatically retired when their corresponding real components are migrated in subsequent waves, without manual intervention.

Claim 12. The method of Claim 2 wherein the selective reconstruction engine generates interface specifications for cross-system communication that define: the data contract (entity, fields, types), the synchronization pattern (primary-replica, bidirectional, event-driven), the synchronization frequency, and the conflict resolution rules.

Claim 13. The method of Claim 3 wherein the migration wave planner applies a risk-based sizing constraint that limits each wave to affect no more than a configurable percentage of users or business processes, controlling the blast radius of each migration step.

Claim 14. The system of Claim 1 wherein the migration progress tracker provides component-level status tracking through the migration lifecycle states: not started, extraction planned, extracted, transformation in progress, transformed, deployed to target, validated on target, live on target, and legacy decommissioned.

Claim 15. The system of Claim 1 wherein the migration progress tracker provides a risk dashboard showing: migration progress as percentage of genome components in each lifecycle state, active cross-system dependency count and types, coexistence health metrics including synchronization lag and data consistency scores, and projected completion dates based on current wave velocity.

Claim 16. The method of Claim 2 further comprising incremental deployment of reconstructed components through a staged process: deployment to staging, automated validation against genome quality scores, cross-system integration testing with live legacy data, canary deployment to a subset of users, and full activation upon successful canary.

Claim 17. The system of Claim 1 further comprising cross-pipeline integration with the orchestration control plane (Patent 7) such that each migration wave is executed as a pipeline instance within the orchestration framework, with skill selection, event streaming, observability, and retry/fallback mechanisms applied to the incremental migration process.

Claim 18. The method of Claim 3 further comprising cross-pipeline learning wherein migration outcomes from completed waves are analyzed to refine the wave plan for subsequent waves, adjust risk scores based on observed complexity, and improve skill selection for component types that proved more or less difficult than estimated.

Claim 19. The system of Claim 1 wherein the coexistence manager supports user routing through feature flags such that individual users or user groups can be gradually transitioned from the legacy system to the target platform for specific functionality, enabling controlled rollout independent of the migration wave schedule.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to implement the system of Claim 1.

---

6. Advantages Over Prior Art

vs. Full Migration (Big-Bang)

1. Reduced risk: Each migration wave affects only a subset of the application; failures are contained to the wave scope rather than the entire application
2. Faster time to value: Users begin working on the target platform after Wave 1 (typically 1-2 sprints) rather than waiting for the full migration to complete (typically months to years)
3. Incremental validation: Each migrated component is validated independently, with issues caught and resolved before subsequent waves add complexity
4. Selective rollback: Individual components can be rolled back without reverting the entire migration, preserving value from successfully migrated components
5. Reduced business disruption: Users experience gradual transition rather than abrupt cutover; the legacy system continues operating throughout
6. Budget flexibility: Migration can be paused, accelerated, or reprioritized between waves based on business needs, without losing progress

vs. Manual Incremental Migration

1. Automated dependency resolution: The system automatically identifies which components can be safely migrated independently and which must be co-migrated, eliminating the most common source of incremental migration failures
2. Systematic wave planning: The system generates optimal migration wave sequences based on dependency analysis, replacing ad-hoc human planning that frequently misses dependencies
3. Automated coexistence management: Data synchronization, workflow routing, and user routing are managed systematically, replacing error-prone manual coordination

4. Stub lifecycle management: Stubs are automatically generated and retired as components are migrated, eliminating a common source of technical debt

Enterprise Adoption Benefits

1. Lower barrier to entry: Organizations can begin migration with a single low-risk wave (e.g., shared data models) rather than committing to a full application migration
2. Proof-of-concept capability: Wave 1 serves as a proof of concept, demonstrating feasibility and building confidence before committing to subsequent waves
3. Budget alignment: Migration cost is distributed across waves, aligning with enterprise budget cycles rather than requiring a single large capital allocation
4. Staff capacity management: Smaller waves can be executed by existing teams without requiring dedicated migration staff
5. Risk-appropriate governance: Each wave can be individually approved through change management processes, with risk scores informing approval decisions
6. Compliance continuity: The legacy system continues operating with full audit trail during migration, satisfying compliance requirements that prohibit migration-related outages

---

7. Drawings/Figures Description

Figure 1: Incremental Migration System Architecture -- Block diagram showing the six major components: Partial Genome Extraction Engine, Dependency Resolution Engine, Migration Wave Planner, Selective Reconstruction Engine, Coexistence Manager, and Migration Progress Tracker, with data flow between them.

Figure 2: Component Dependency Graph -- Example dependency graph showing 12 application components (forms, workflows, data models, rules, integrations) with hard, soft, and cross-system dependency edges, independent clusters highlighted, and a circular dependency cluster identified.

Figure 3: Migration Wave Plan -- Timeline diagram showing five migration waves with component assignments, dependency arrows between waves, cross-system dependency creation and resolution markers, and risk scores per wave.

Figure 4: Partial Genome Extraction -- Diagram showing a full application genome (47 components) with a selected extraction set highlighted, hard dependency closure computed, and the resulting partial genome (7 components) extracted as a self-contained fragment.

Figure 5: Selective Reconstruction Flow -- Sequence diagram showing: partial genome extraction -> translation recipe

application -> interface/stub generation -> staging deployment -> validation -> canary -> full activation -> stub retirement.

Figure 6: Legacy-Modern Coexistence Architecture -- Block diagram showing the legacy system and target platform operating simultaneously with the Coexistence Manager coordinating data synchronization, workflow routing, and user routing between them.

Figure 7: Data Synchronization Patterns -- Three-panel diagram showing primary-replica, bidirectional, and event-driven synchronization patterns with data flow arrows and conflict resolution decision points.

Figure 8: Workflow Routing -- Sequence diagram showing a workflow with six steps, three executing on the legacy system and three on the target platform, with cross-system handoff points managed by the Coexistence Manager.

Figure 9: Selective Rollback -- Before/after diagram showing a successfully migrated component being rolled back to the legacy system: routing table update, component deactivation on target, reactivation on legacy, and stub restoration.

Figure 10: Migration Progress Dashboard -- Wireframe showing component-level status tracking, genome coverage percentage, active cross-system dependency count, coexistence health metrics, risk exposure, and projected completion timeline.

Figure 11: Extraction Granularity Levels -- Four-panel diagram showing the same application genome with different extraction granularity selections: individual component, component cluster, functional module, and custom selection.

Figure 12: Wave Evolution -- Series of snapshots showing the application state at the end of each wave: which components are on legacy, which are on target, which stubs are active, and which cross-system dependencies exist.

Figure 13: Risk Curve -- Graph showing migration risk over time, with risk increasing during each wave's deployment window and decreasing as validation completes, overall risk declining as more components are successfully migrated.

---

8. Inventors

[To be completed by filing attorney]

9. Filing Notes

- Risk reduction framing is the enterprise sale: The primary value proposition -- reduced risk through incremental migration -- should be emphasized throughout prosecution. Enterprise decision-makers choose incremental approaches specifically because of risk management requirements.

- Dependency resolution is the strongest technical claim: The dependency graph construction, classification (hard/soft/cross-system), minimum viable extraction set computation, and circular dependency detection constitute the core technical novelty. Prior art search should focus on: dependency analysis in software migration, modular decomposition systems, and graph-based migration planners.
- Coexistence management is the most operationally novel claim: No prior system provides automated legacy-modern coexistence with data synchronization, workflow routing, user routing, and stub lifecycle management for partially migrated applications. This is a key differentiator from both big-bang migration and manual incremental approaches.
- Selective rollback is a critical enterprise requirement: The ability to roll back individual components without affecting other migrated components addresses one of the primary objections enterprise customers raise against migration projects.
- Patent family integration: Claims 17 and 18 reference Patents 7 (Orchestration) and the translation recipe system (Patent 3). The incremental migration system operates within the orchestration framework and uses recipes for component-level translation.
- Prior art search should focus on: Application migration tools (AWS Migration Hub, Azure Migrate, CloudEndure), strangler fig pattern implementations, database migration tools (Flyway, Liquibase), feature flag systems (LaunchDarkly, Split), data synchronization platforms (Debezium, Striim), and enterprise integration patterns (ESB, iPaaS).
- International filing: PCT filing recommended for global enterprise market.
- Consider filing as part of the existing patent family to strengthen the portfolio. The incremental migration system is a natural extension of the genome extraction and transformation patents.

This document is a technical disclosure for patent filing purposes. It should be reviewed by a registered patent attorney before formal claims are drafted and filed.